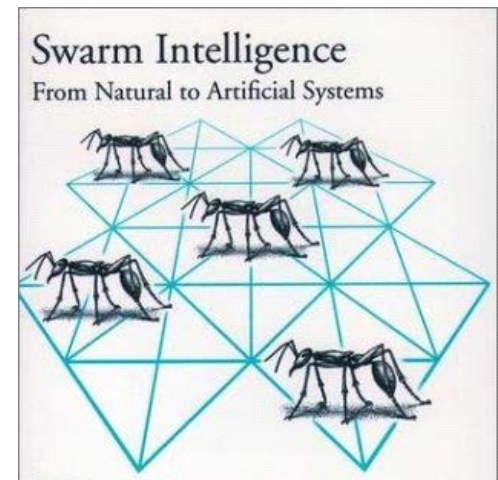


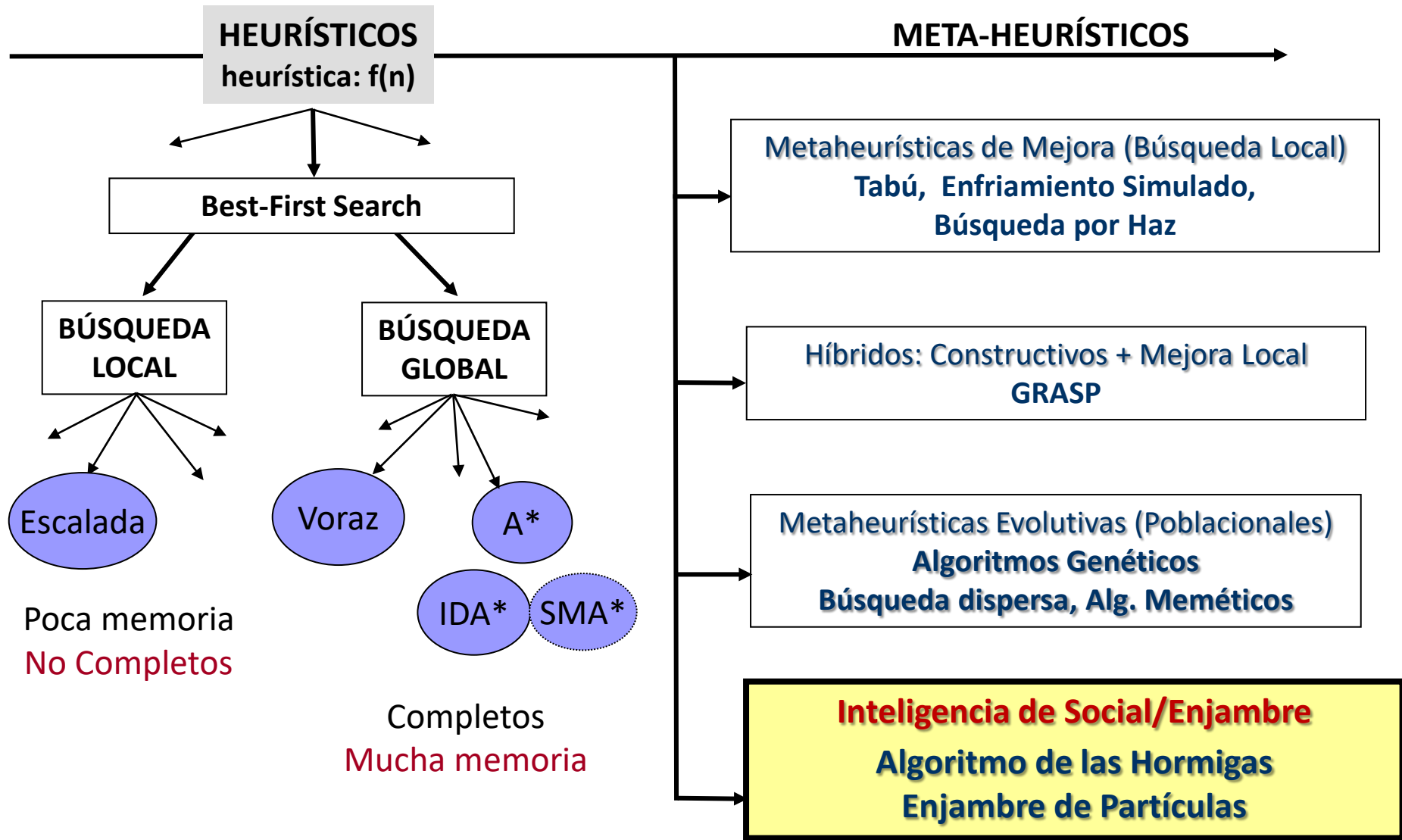
Tema- 5 Inteligencia de Enjambre (Swarm intelligence)

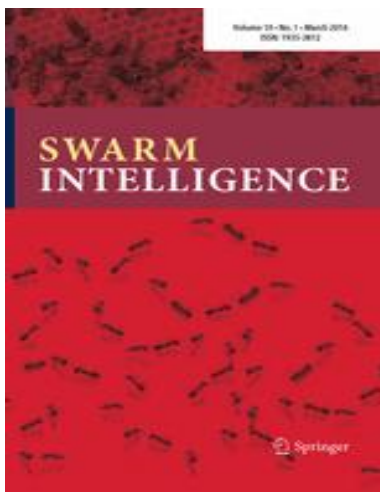
(Inteligencia Social, Inteligencia Colectiva, Metaheurísticas Bio-Inspiradas)

- Algoritmo de las Hormigas (*Ant Colony Optimization - ACO*),
- Enjambre de partículas (*Particle Swarm Optimization - PSO*)
- Otras variantes

<http://www.swarmintelligence.org/>







Artificial Intelligence Review (2020) 53:5589–5635
<https://doi.org/10.1007/s10462-020-09829-2>

Performance comparison of five metaheuristic nature-inspired algorithms to find near-OGRs for WDM systems

Artif Intell Rev (2019) 52:2191–2233
<https://doi.org/10.1007/s10462-017-9605-z>

Metaheuristic research: a comprehensive survey

Artificial Intelligence Review (2020) 53:753–810
<https://doi.org/10.1007/s10462-018-09676-2>

From ants to whales: metaheuristics for all tastes

[ACO Timetabling Tool.pdf](#)

[Ant Colony Optimization Book.pdf](#)

[Aplicaciones Algoritmo Hormigas.pdf](#)

[Aplicaciones Metaheuristica POO.pdf](#)

[APPLICATION IN POWER SYSTEM PROBLEMS.pdf](#)

[Comparison of 5 novel nature inspired metaheuristic.pdf](#)

[From ants to whalesmetaheuristics for all tastes \(2020\).pdf](#)

[Metaheuristic Review 2019.pdf](#)

[multi-objective particle swarm optimization.pdf](#)

[PARTICLE SWARM OPTIMIZATION.pdf](#)

[Particle Swarm Optimization Algorithms.pdf](#)

[PSO for engineering problems.pdf](#)

[PSO-Survey.pdf](#)

[PSO Viajante Multiobjetivo.pdf](#)

[State of the art review of intelligent scheduling.pdf](#)

[Survey PSO para timetabling.pdf](#)

Principios de la Inteligencia de Enjambre: Descentralización, Auto-organización, Estigmergía

Basada en el comportamiento colectivo de sistemas descentralizados y auto-organizados.

- Un sistema auto-organizado mantiene una estructura (funcionalidad) *independiente del exterior*, o de un *control global*, y que proviene de las *interacciones* en los *niveles más bajos*.

Múltiples interacciones

Realimentación positiva /negativa

Amplificación en la selección de opciones aleatorias



- En un sistema descentralizado, el control esta completamente distribuido entre sus elementos componentes

Estigmergía: stigma (estimulación) + ergon (trabajo)

Interacción por el entorno:

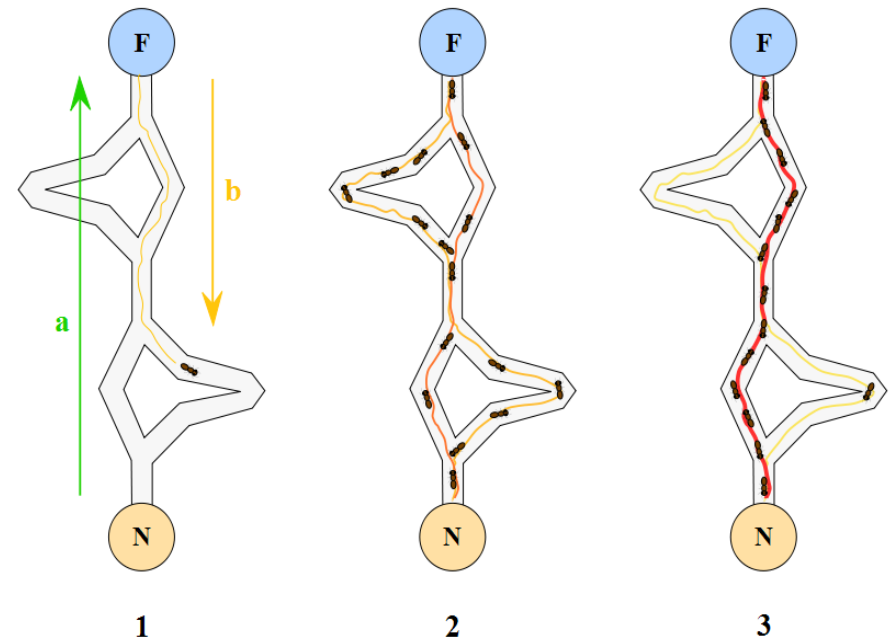
- Los agentes actúan como respuesta al entorno y a otros agentes, mediante reglas de conducta muy simples.
- Los agentes *modifican* (indirectamente) el entorno: Interaccionan (colaboran) entre ellos a través del entorno (medio físico).
- El entorno actúa como una memoria de trabajo.
- La interacción puede ser realizada por cualquier individuo.
- Las reglas simples de la respuesta al entorno pueden crear diferentes patrones de conducta.

Inteligencia de enjambre: *Basada en la observación de la naturaleza (bio-inspirada)*

- Se basa en una población (sociedad) de **agentes individuales, auto-organizados y sin control centralizado**.
- Los individuos **interactúan** entre sí y con el entorno, de forma localizada y siguiendo reglas muy simples.
- Hay un cierto seguimiento al **líder, a los buenos exploradores, o a la costumbre social**.
- A diferencia de las metaheurísticas poblacionales, los individuos no son eliminados (no hay reemplazo), no hay control centralizado y los individuos suelen estar en constante movimiento.

Resultado:

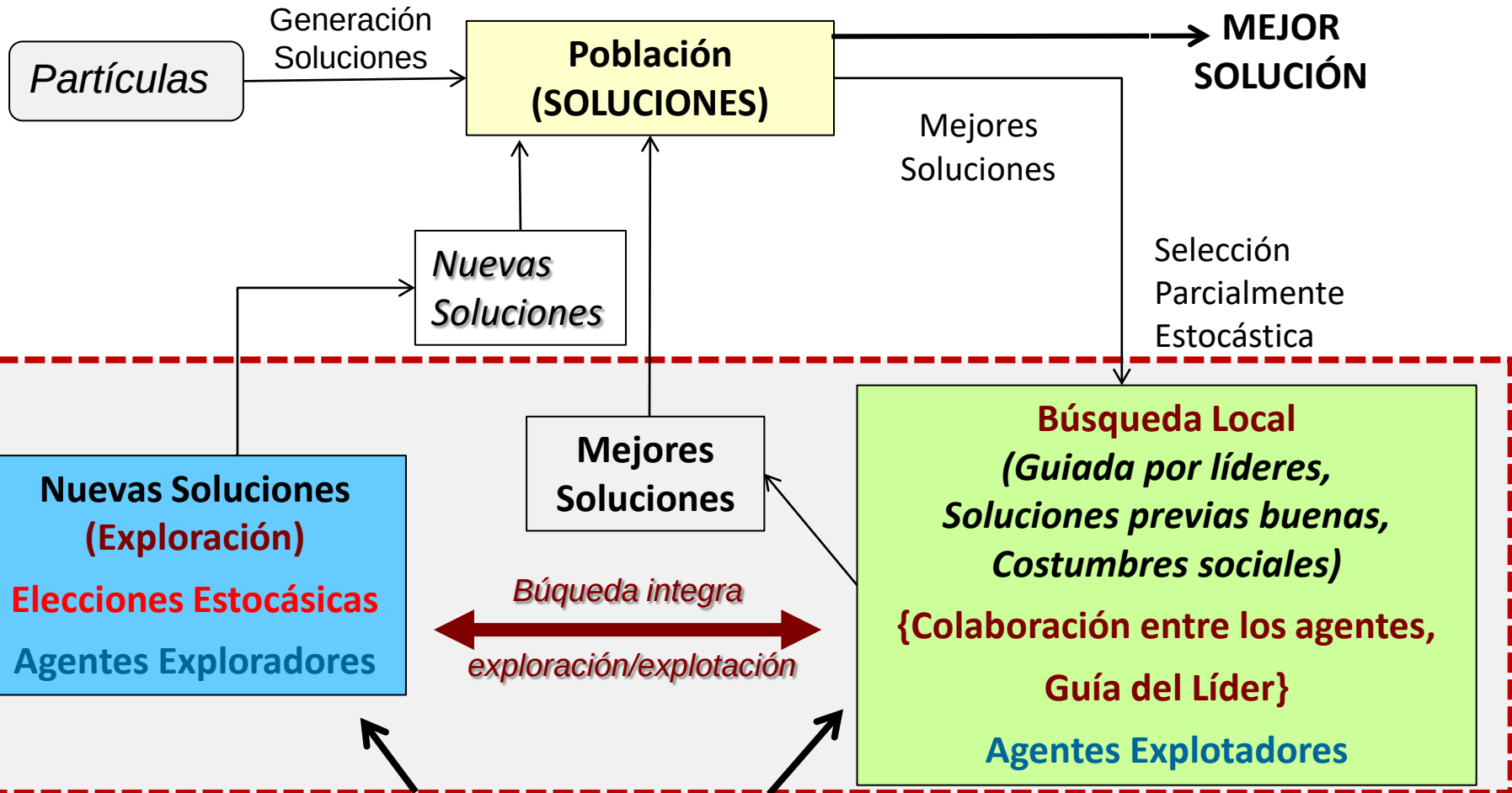
- La **interacción** entre los agentes induce una cierta **inteligencia colectiva** (desconocida incluso para los propios individuos).
- La funcionalidad del sistema global **transciende** el repertorio de las conductas individuales.
- La productividad del sistema (colonia) es mayor que el esfuerzo de sus miembros
- La respuesta conjunta del grupo es **robusta, flexible y adaptativa** con respecto a los cambios en el entorno (**sistemas auto-inmunes**)



Esquema General

Las partículas generan las soluciones (no son las soluciones)

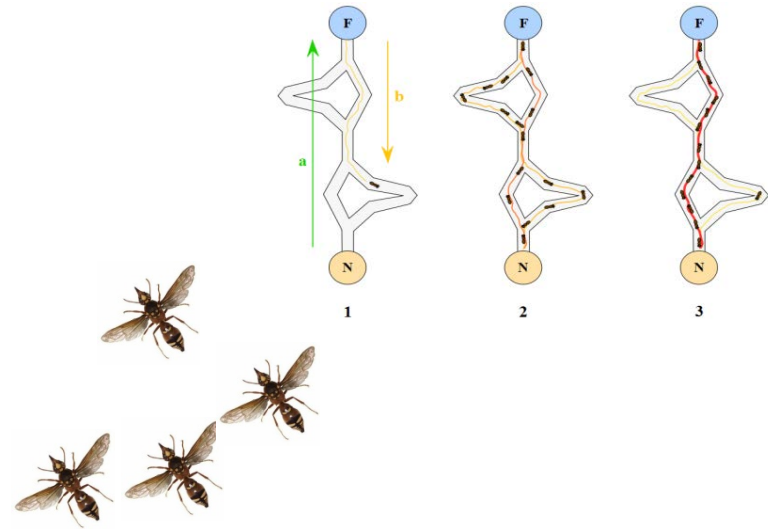
INICIO



Balance Estocástico: Exploración / Explotación

Principales Variantes:

- **Algoritmo de las Hormigas**
(Ant Colony Optimization),



- **Enjambre de partículas**
(Particle Swarm Optimization)
Bandada de Pájaros, Bancos de Peces, etc.



Otras metaheurísticas (enjambre/bioinspiradas):

Algoritmo de las abejas,
Algoritmo de los murciélagos (Bat algorithm),
Algoritmo de las luciérnagas (glowworm swarm optimization, GSO),
Caída inteligente de gotas de agua (Intelligent Water Drops, IWD),
Etc., etc., etc., etc., etc.....



Algoritmo de las Hormigas

(Ant Colony Optimization) M. Dorigo, 1992



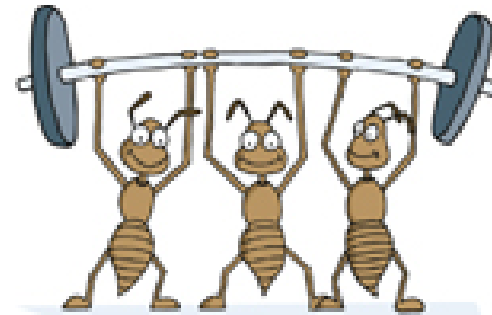
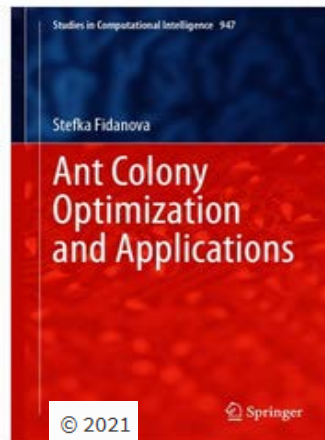
http://www.scholarpedia.org/article/Ant_colony_optimization

Google Académico

<http://www.midaco-solver.com/>

<http://www.aco-metaheuristic.org/>

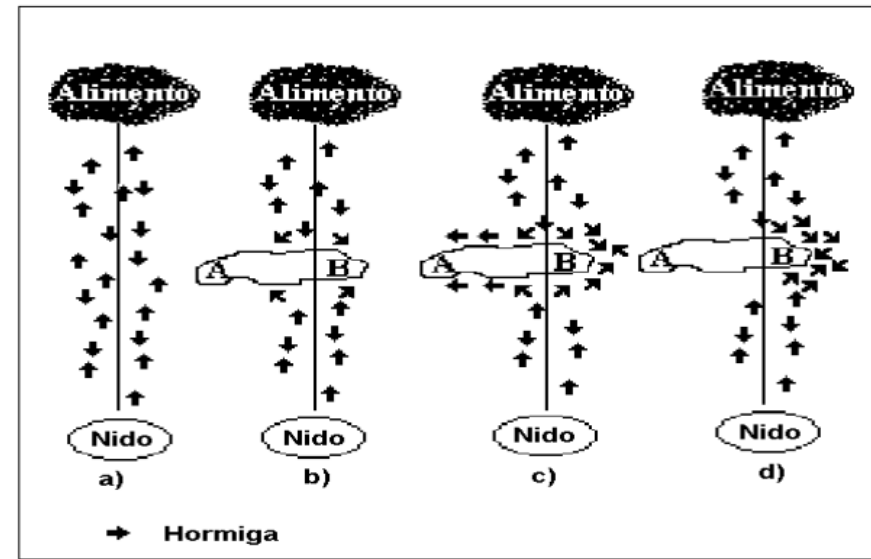
Artículos en Poliformat



"Optimization, Learning and Natural Algorithms" PhD thesis, Politecnico di Milano, 1992.

Basado en la observación de la naturaleza: Hormigas en la búsqueda de comida.

- Las hormigas recorren diversos caminos buscando el alimento. Eventualmente, encuentran (si existen) varias sendas (soluciones).
- La hormigas dejan una sustancia (feromona), marcando el rastro que recorren.
 - La feromona se evapora en el tiempo.
 - La feromona en cada punto de un camino dependerá del número (caudal) de hormigas que han dejado su rastro.
- Las hormigas, en la búsqueda del camino, siguen el rastro de las feromonas.
- El camino más corto tendrá el mayor numero de paso de hormigas: más caudal $\Rightarrow$ mayor feromona
- Al final, ***todas las hormigas siguen el camino óptimo.***
- Aplicable para obtener sendas optimizadas en grafos (y problemas equivalentes).



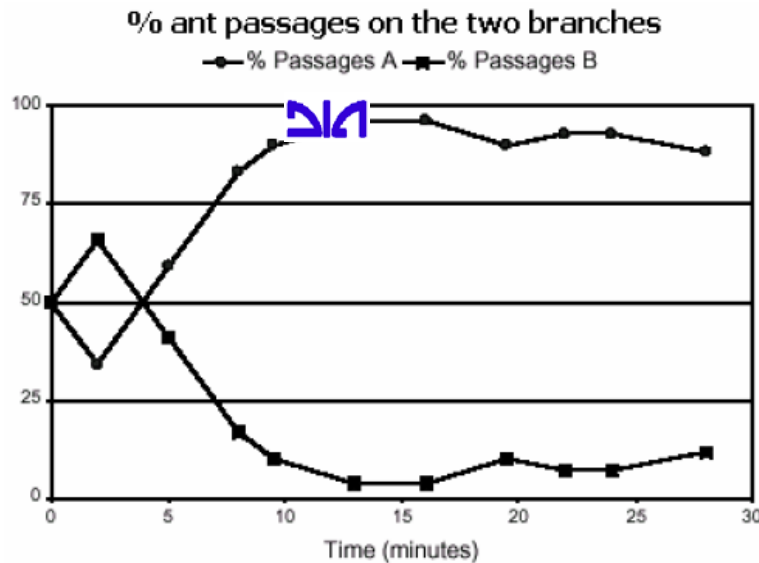
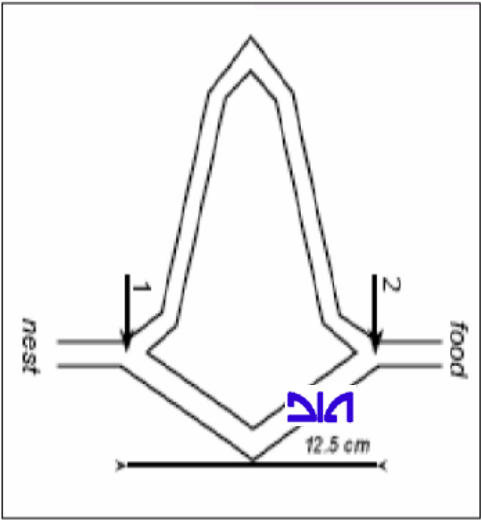
Al ser interrumpido el camino directo, la mitad de las hormigas se dirigen por A y la otra mitad por B.

- B es la menor distancia para rodear el obstáculo, luego mayor cantidad de hormigas y depositan más feromonas.

Las hormiga siguen el rastro de las feromonas y elegirán rodear el obstáculo por B, lo que a su vez depositará más feromonas en ese trayecto.

Por realimentación positiva, el camino B será seleccionado como el camino más corto.

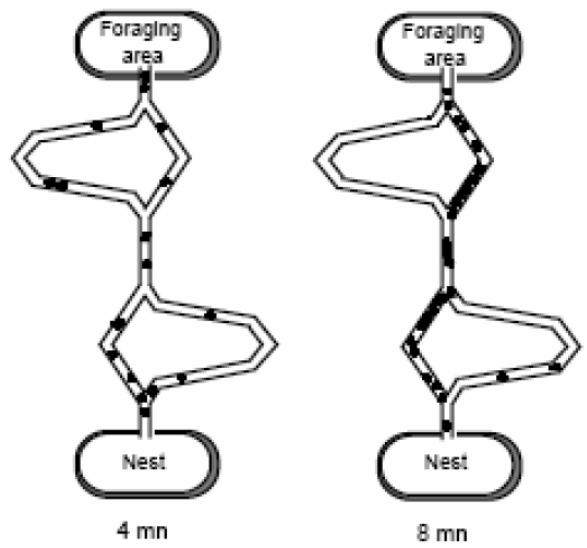
Antecedentes..... Estudio sobre las Hormigas ‘biológicas’ (Goss et al., 1989, Deneubourg et al., 1990).



Self-organized shortcuts in the Argentine ant Goss et al., 1989

Estudiaron el porcentaje de hormigas que tomaban cada camino.

Inicialmente los caminos lo elegían de manera arbitraria, pero casi todas acababan siguiendo el mismo camino.



A partir de sus experimentos, construyeron un **modelo estocástico sencillo** para describir la dinámica de las hormigas:

$P_{i,a}$: La probabilidad de que una hormiga llegue a la intersección i y seleccione la rama a en un instante t es:

$$p_{i,a} = \frac{[k + \tau_{i,a}]^{\alpha}}{[k + \tau_{i,a}]^{\alpha} + [k + \tau_{i,a'}]^{\alpha}}$$

donde

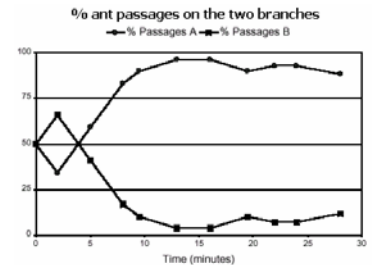
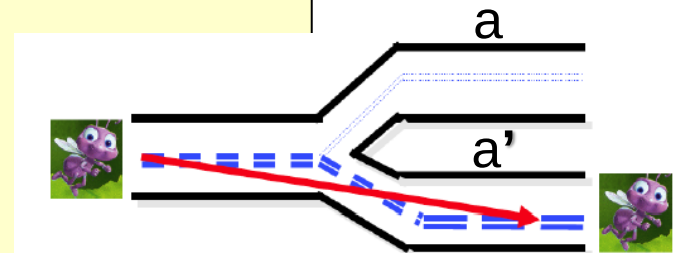
$\tau_{i,a}$ es la concentración de feromona en la rama a de la intersección i ,

$\tau_{i,a'}$ es la concentración de feromona en la otra rama de la intersección i ,

$\alpha = 2$ (obtenido de diversos experimentos), y

k es el tiempo que se tarda en atravesar la rama a .

La concentración de feromona depende del caudal de hormigas



- La **metaheurística ACO** reproduce el comportamiento de las hormigas reales en una colonia artificial de hormigas para resolver problemas complejos de camino mínimo
- Cada **hormiga artificial** (agente) es un mecanismo probabilístico de **construcción** de soluciones al problema que **combina**:
 - a) Rastros de **feromona (artificiales) τ** que cambian con el tiempo para reflejar la **experiencia** adquirida por los agentes previos en la resolución del problema (**explotación**).
 - b) Información **heurística η** sobre el problema.

Aplicación al Problema del Viajante

- El objetivo es encontrar la senda más corta que recorre una serie de ciudades.
- Cada hormiga hace una senda alternativa de las posibles:
Cada hormiga debe visitar todas las ciudades, y cada ciudad exactamente una vez,
- En cada iteración, **cada hormiga decide pasar de una ciudad a otra**, en base a:
 - a) Exploración Propia: Una ciudad lejana tiene menos posibilidades de ser elegida frente a una cercana (por **visibilidad: heurística η**),
 - b) Explotación/Colaboración: Cuanto más marcado esté el camino (más intensa es la **feromona: τ**) entre dos ciudades, mayor probabilidad de que sea elegido,
- Cada hormiga, tras completar su viaje, ha depositado más feromonas en sus trayectos si su viaje ha sido más corto (carga/longitud).
- Las feromonas se evaporan con el tiempo.

Procedure Hormigas

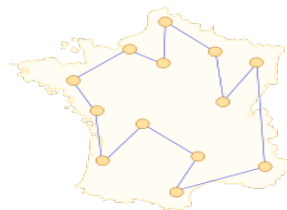
while (no-fin)

 GenerarSoluciones()

 Actualización-Feromonas()

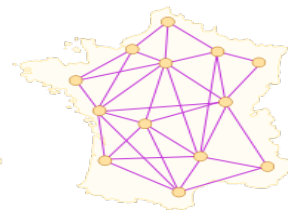
end while

end procedure



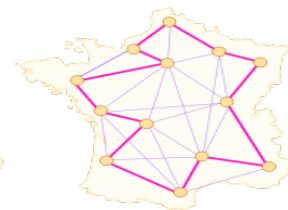
1

obtención
de una
solución
(1 hormiga)



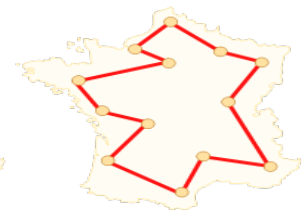
2

múltiples
hormigas
buscando



3

el mejor camino
se va definiendo
(arcos de mayor
feromona)



4

se obtiene el
mejor
camino

Algoritmo de las Hormigas-1

Procedure Hormigas

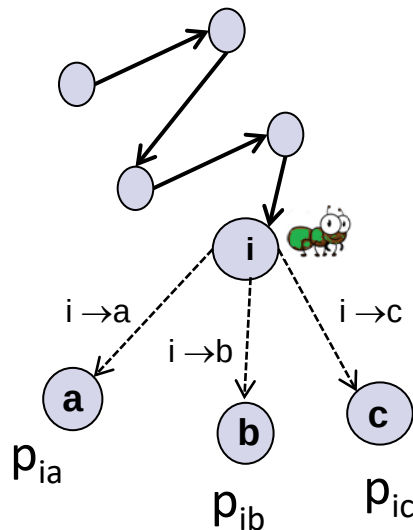
while (no-fin)

GenerarSoluciones()

Actualización-Feromonas()

end while

end procedure



GenerarSoluciones ()

;aplicación al viajante

- Cada hormiga obtendrá un camino: *Soluciones* $\equiv$ *Caminos*
- En cada iteración, las hormigas eligen un trayecto $i \rightarrow j$.
(si N ciudades, entonces N-1 iteraciones)

Para cada arco i ($i=1, N-1$) ; N: ciudades

Para cada Hormiga h ($h=1, H$) ; H: Hormigas

Hormiga h selecciona siguiente trayecto (ciudad) en su senda

- Criterio de Selección de Trayecto

Una hormiga irá de $i \rightarrow j$, con probabilidad

$$p_{i,j} = \frac{(\tau_{i,j}^{\alpha})(\eta_{i,j}^{\beta})}{\sum (\tau_{i,j}^{\alpha})(\eta_{i,j}^{\beta})}$$

← Valores del arco $i \rightarrow j$

← Sumatorio para todos los nodos del grafo

Donde,

τ_{ij} es la feromona de arco $i \rightarrow j$, con una influencia α

η_{ij} es la visibilidad del arco (típicamente, $1/\text{distancia}$), con influencia β

Algoritmo de las Hormigas-2

Procedure Hormigas

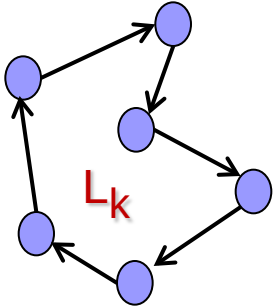
while (no-fin)

GenerarSoluciones()

Actualización-Feromonas()

end while

end procedure



- Con un buen diseño, los caminos de todas las hormigas acaban convergiendo.
- El proceso acaba tras un número dado de iteraciones, o cuando se cumple un criterio de terminación

Actualización-Feromonas ()

Las feromonas en los arcos se actualizan tras la obtención de un camino (solución) por todas las hormigas.

La feromona de cada arco $i \rightarrow j$ (τ_{ij}) se actualiza de la forma:

$$\tau_{ij} = (1-\rho) \tau'_{ij} + \Delta\tau_{ij}$$

Donde,

ρ Es la tasa de evaporación, que se aplica a la feromona previa τ'_{ij}

$\Delta\tau_{ij}$ Es la feromona que se deposita tras los caminos encontrados:

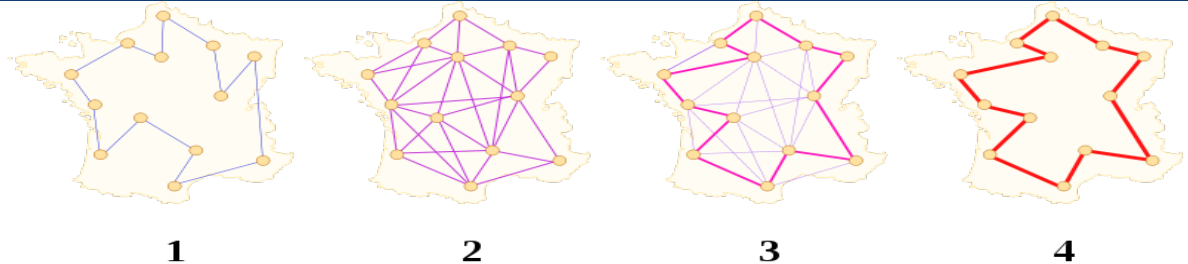
$$\Delta\tau_{ij} = \sum_{h=1, H} \Delta\tau^h_{ij}$$

siendo $\Delta\tau^h_{ij}$ la feromona dejada por hormiga h en $i \rightarrow j$:

= $1/L_h$ si la hormiga h atraviesa el arco

= 0 en otro caso

L_h es el coste (distancia) de todo el camino obtenido por la hormiga- h



Implementación Algoritmo de las Hormigas: 3 matrices

Matriz de feromonas: Feromonas del arco $i \rightarrow j$

Las feromonas de cada celda (i,j)
se reactualizan tras cada iteración:

$$\tau_{ij} = (1-\rho) \tau'_{ij} + \Delta\tau_{ij}$$

Matriz de Visibilidades:

Distancia del arco $i \rightarrow j$

Matriz de Probabilidades:

Probabilidades del arco $i \rightarrow j$

Las probabilidades de cada
celda (i,j) se reactualizan tras
cada iteración

$$p_{i,j} = \frac{(\tau_{i,j}^\alpha)(\eta_{i,j}^\beta)}{\sum (\tau_{i,j}^\alpha)(\eta_{i,j}^\beta)}$$

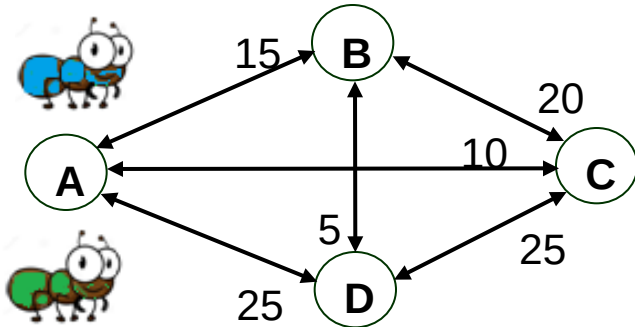
	1	2	3	4	5	...	n-1	n
1								
2								
3								
4								
....								
n-1								
n								

Parámetros: Hormigas,
 ρ (evaporación),
 α, β (pesos), τ_{INICIAL}

$\uparrow \rho, \beta, \tau_{\text{INICIAL}} \Rightarrow$ Exploración ,

$\downarrow \rho, \uparrow \alpha \Rightarrow$ Explotación ($\approx$ feromona colaborativa)

Ejemplo: Problema del Viajante



$$(\eta_{ij}=1/l_{ij})$$

	B	C	D
A	1/15	1/10	1/25
B		1/20	1/5
C			1/25

Matriz de Visibilidades

$$\tau_{ij} = (1-\rho) \tau'_{ij} + \Delta\tau_{ij}$$

	B	C	D
A	0,1	0,1	0,1
B		0,1	0,1
C			0,1

Matriz de Feromonas (inicial)

Parámetros:

Hormigas $\{h_i\}=2$, $\rho=0,5$, $\alpha=0,4$, $\beta=0,6$, $\tau_{INICIAL}=0,1$

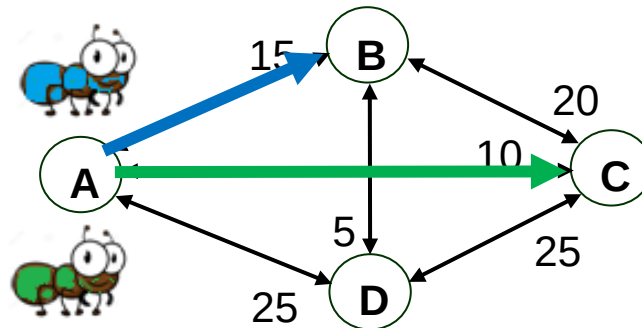
$$p_{i,j} = \frac{(\tau_{i,j}^\alpha)(\eta_{i,j}^\beta)}{\sum (\tau_{i,j}^\alpha)(\eta_{i,j}^\beta)}$$

	B	C	D
A	0,18	0,21	0,14
B		0,16	0,18
C			0,14

Matriz de Probabilidades

h1, en A elige C (p=0,21)

h2, en A elige B (p=0,18),



$$p_{i,j} = \frac{(\tau_{i,j}^\alpha)(\eta_{i,j}^\beta)}{\sum (\tau_{i,j}^\alpha)(\eta_{i,j}^\beta)}$$

	B	C	D
A	0,18	0,21	0,14
B		0,16	0,18
C			0,14

Matriz de Probabilidades

h1 (A-C) elige B (P=0,16)

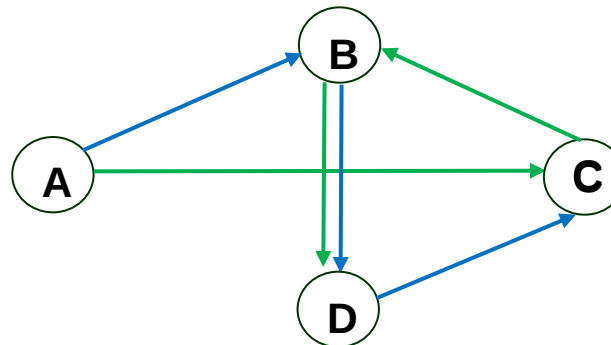
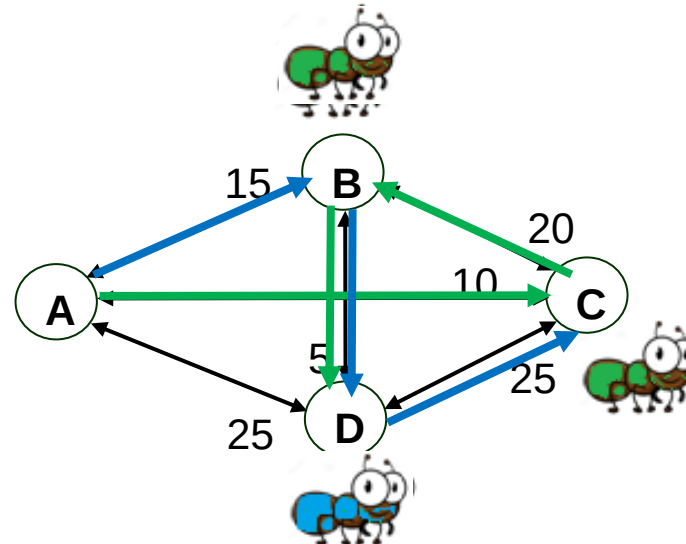
h2 (A-B) elige D (p=0,18)

h1 (A-C-B) elige D

h2 (A-B-D) elige C

Parámetros:

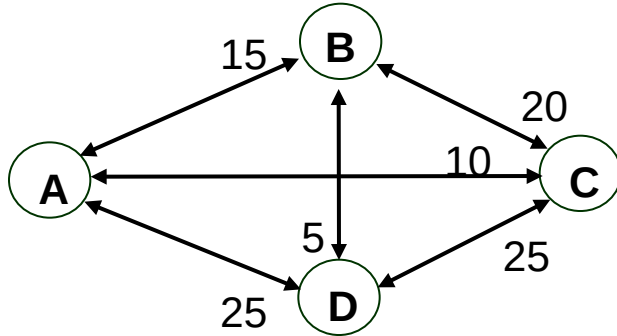
Hormigas $\{h_i\}=2$, $\rho=0,5$, $\alpha=0,4$, $\beta=0,6$, $\tau_{\text{INICIAL}}=0,1$



L1: 35+25=60

L2: 45+10=55

Actualización de feromonas



$(\eta_{ij}=1/l_{ij})$

	B	C	D
A	1/15	1/10	1/25
B		1/20	1/5
C			1/25

Matriz de Visibilidades

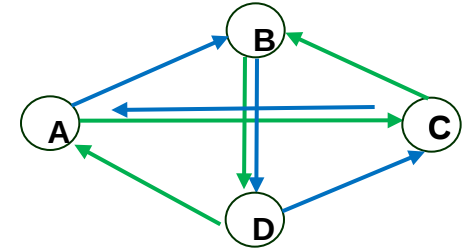
	B	C	D
A	0,07	0,08	0,07
B		0,07	0,08
C			0,05

Matriz de Feromonas (actualizada)

Hormigas=2, $\rho=0,5$, $\alpha=0,4$, $\beta=0,6$, $\tau_{\text{INICIAL}}=0,1$

L1: 60. En cada arco que pasa deja 1/60

L2: 55. En cada arco que pasa deja 1/55



	B	C	D
A	0,18	0,21	0,14
B		0,16	0,18
C			0,14

	B	C	D
A	0,17	0,23	0,14
B		0,15	0,20
C			0,12

Matriz de Probabilidades (actualizada)

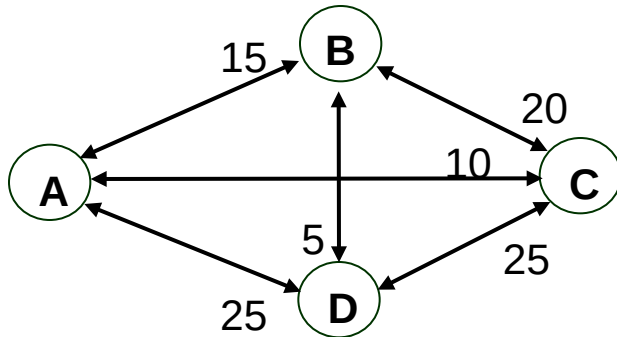
	B	C	D
A	0,1	0,1	0,1
B		0,1	0,1
C			0,1

Matriz de Feromonas (inicial)

$$p_{i,j} = \frac{(\tau_{i,j}^{\alpha})(\eta_{i,j}^{\beta})}{\sum (\tau_{i,j}^{\alpha})(\eta_{i,j}^{\beta})}$$

$$\tau_{ij} = (1-\rho) \tau'_{ij} + \Delta\tau_{ij}$$

Hormigas=2, $\rho=0,5$, $\alpha=0,4$, $\beta=0,6$, $\tau_{\text{INICIAL}}=0,1$



	B	C	D
A	0,18	0,21	0,14
B		0,16	0,18
C			0,14

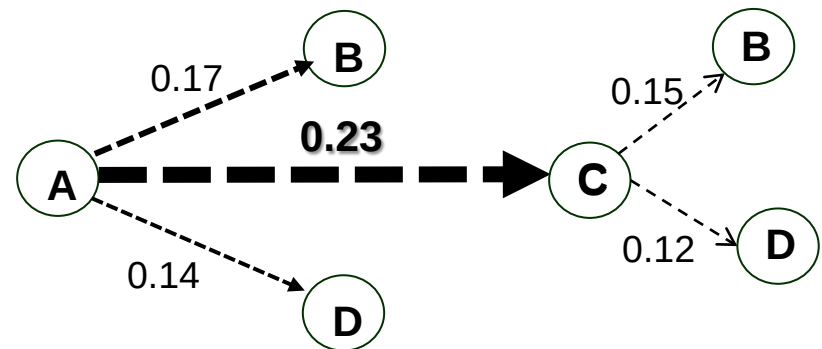
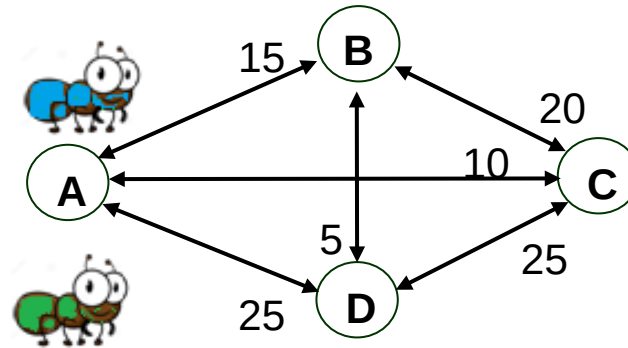


	B	C	D
A	0,17	0,23	0,14
B		0,15	0,20
C			0,12

Matriz de Probabilidades (**actualizada**)

Segunda iteración:

Hay una retro-alimentación entre las hormigas



A-C-D-B-A es un camino óptimo

Algoritmo Hormigas ()

- Inicialización Matriz de Visibilidad, Matriz de Feromonas
- Repetir hasta fin

Contrucción Soluciones:

Para cada paso i ($i=1, N$) {

Para cada Hormiga- h ($h=1, \text{Hormigas}$)

{ Hormiga- h selecciona siguiente arco (ciudad) en su senda:
Una hormiga irá de $i \rightarrow j$, con probabilidad $p_{i,j} = \frac{(\tau_{i,j}^\alpha)(\eta_{i,j}^\beta)}{\sum (\tau_{i,j}^\alpha)(\eta_{i,j}^\beta)}$

Donde,

τ_{ij} es la feromona de arco $i \rightarrow j$, con una influencia α

η_{ij} es la visibilidad del arco, con influencia β }

}

Actualización-Feromonas:

La feromona de cada arco $i \rightarrow j$ (τ_{ij}) se actualiza de la forma: $\tau_{ij} = (1-\rho) \tau'_{ij} + \Delta\tau_{ij}$

Donde, ρ Es la tasa de evaporación, que se aplica a la feromona previa τ'_{ij}

$\Delta\tau_{ij}$ Feromona depositada por todas las hormigas en los arcos que recorren:

$$\Delta\tau_{ij} = \sum_{h=1, H} \Delta\tau_{ij}^h$$

siendo $\Delta\tau_{ij}^h$ la feromona dejada por hormiga h en $i \rightarrow j$:

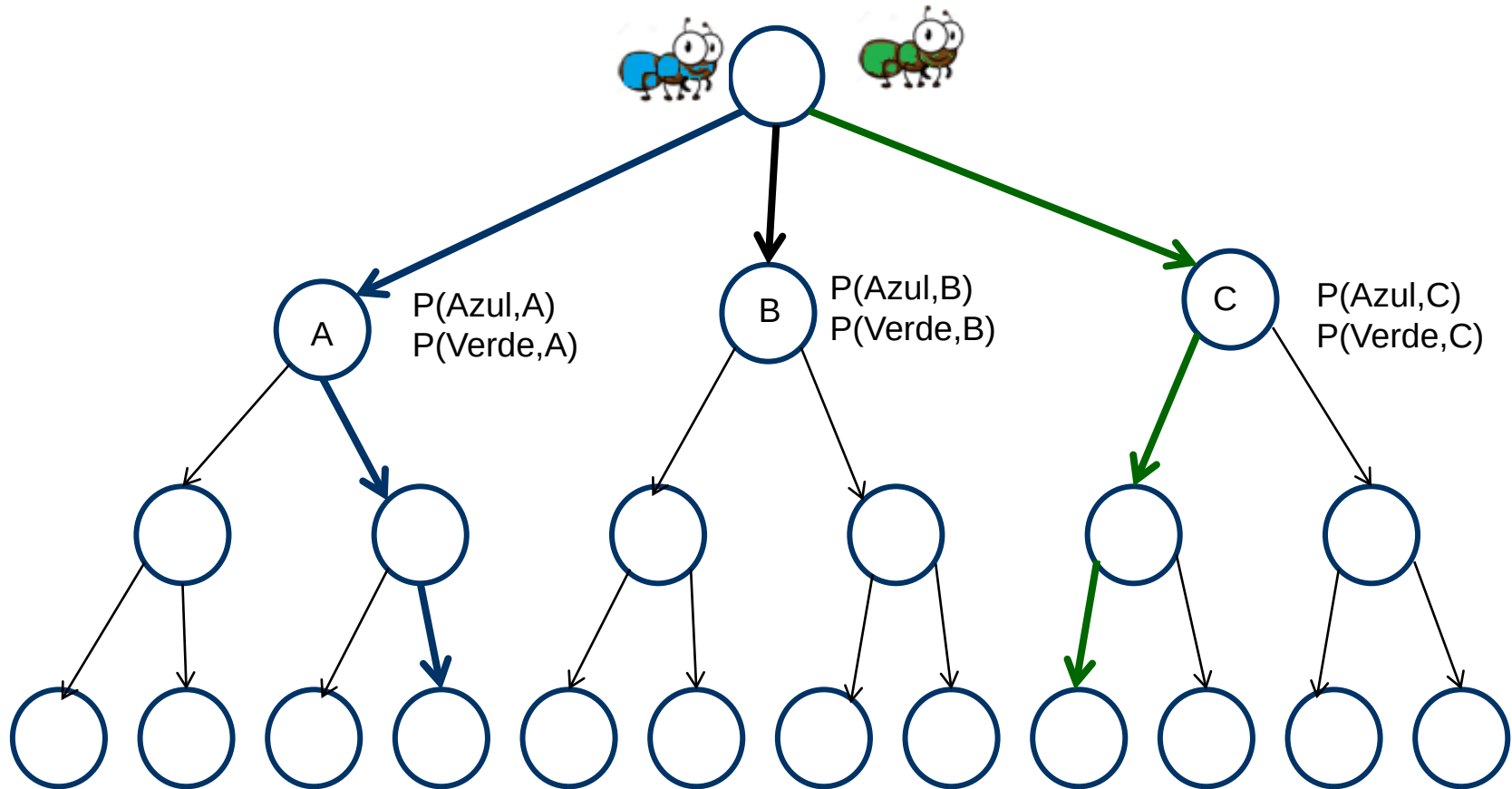
= $1/L_h$ si la hormiga h atraviesa el arco, y ha realizado una senda de coste L_h

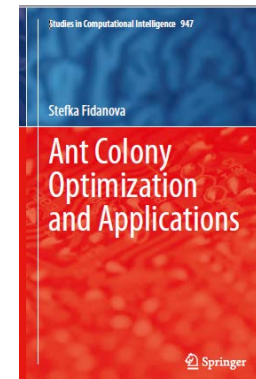
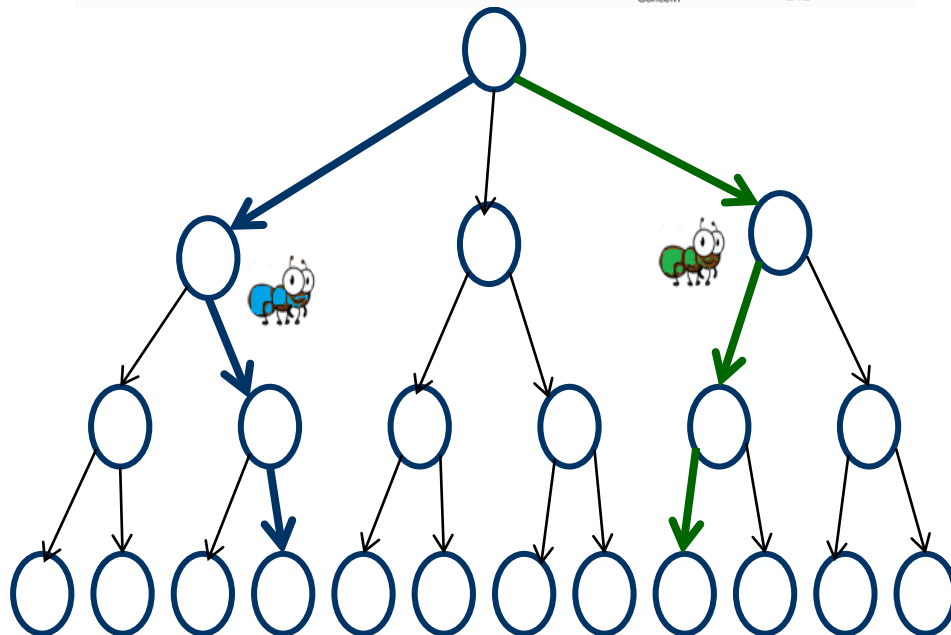
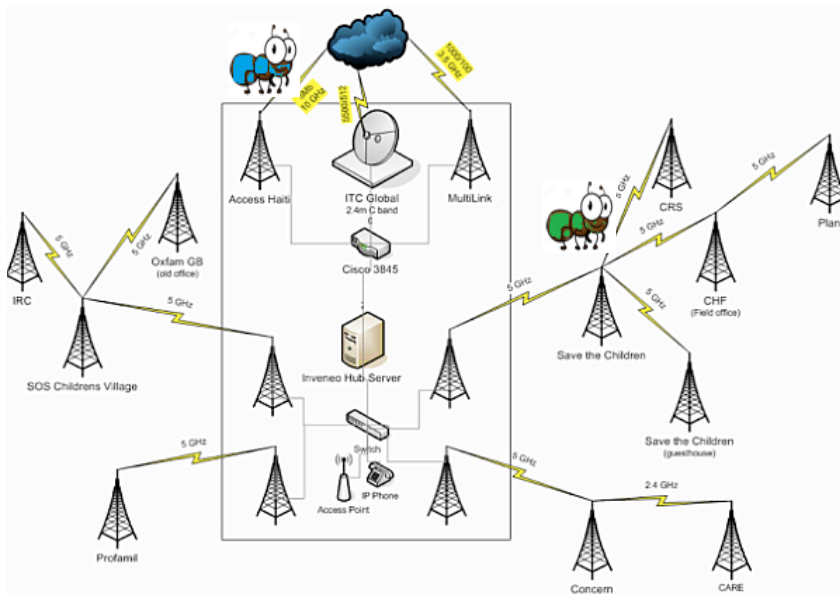
= 0 en otro caso

Actualización Matriz de Probabilidades

Los problemas que se pueden resolver con los algoritmos ACO son los que se pueden representar como un grafo ponderado $G = (N, A)$, donde A es el conjunto de aristas que conectan los nodos N

Los problemas que se pueden resolver con los algoritmos ACO son los que se pueden representar como un grafo ponderado $G = (N,A)$, donde A es el conjunto de aristas que conectan los nodos N (espacio de búsqueda con arcos de peso no uniforme)





An ANTS heuristic for the frequency assignment problem. Maniezzo V, Carbonaro A. Future Gener Comput Syst 16(8): 927–935.

Dependencia de los parámetros

↑ Feromona inicial ⇒ ↑ Exploración

Si se inicializan las **feromonas a valores muy bajos** la búsqueda estará muy influenciada por los primeros recorridos: **menos explotación**.

Si se inicializan las **feromonas a valores muy altos** deberán pasar muchas iteraciones hasta que la evaporación reduzca los suficiente los valores de feromona para que las feromonas introducidas por las hormigas empiecen a influir en la búsqueda: **más exploración**

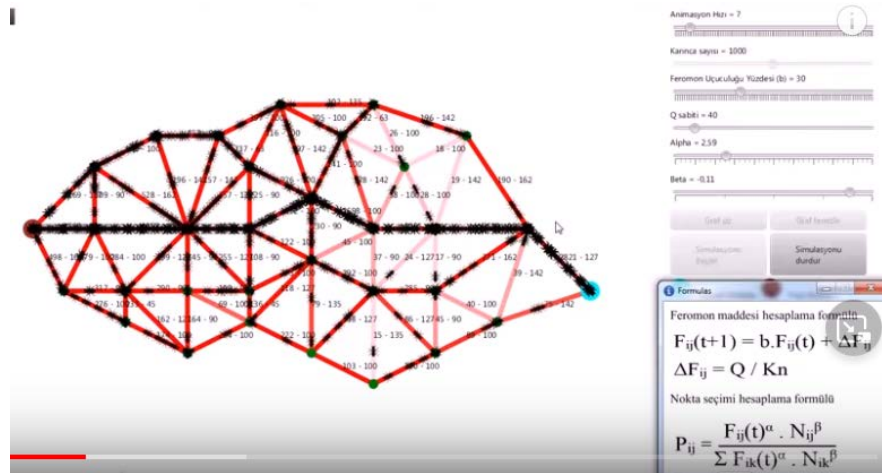
↑ Feromona depositable por hormigas ⇒ ↑ Explotación

↑H: nº de Hormigas

↑ ρ (evaporación): menos realimentación

α : Influencia visibilidad (η : heurística del problema)

β Influencia Feromona (τ : retroalimentación)



Numan Karaaslan (Requiere JavaX)

Variaciones Algoritmo de las Hormigas

Sistema Básico

En cada iteración, las hormigas depositan feromona de acuerdo a la calidad de la solución que encuentran ($1/L_k$).

Sistema Elitista

En cada iteración, la **solución global (almacenada) mejor** también deposita feromonas, junto con el resto de las hormigas.

Sistema basado en el rango

Las soluciones son ordenadas de acuerdo a su calidad (longitud).

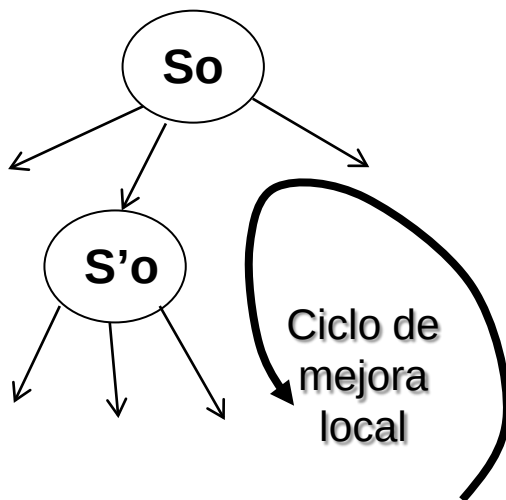
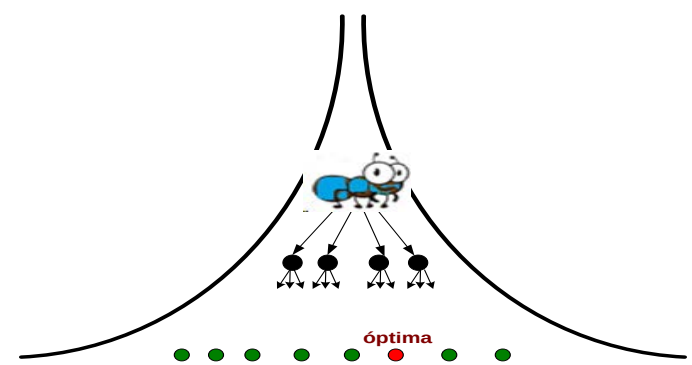
La cantidad de feromona depositada por cada hormiga **depende del rango de su solución, en vez de la longitud de su senda.**

Sistema de Colonias:

Todas las hormigas dejan la misma feromona en los arcos que recorren, independientemente de la bondad de la solución.

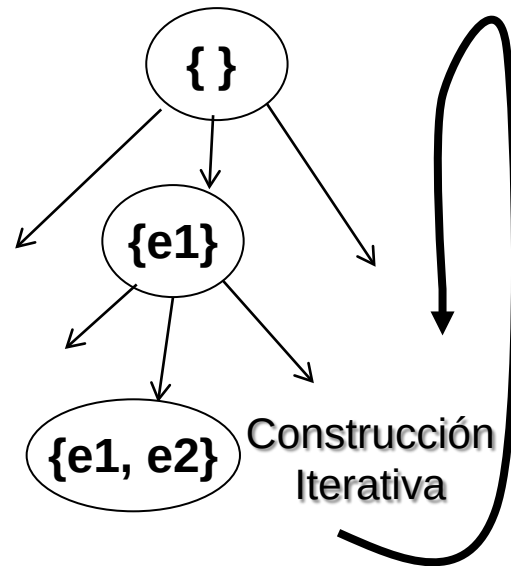
Pero las hormigas no están sincronizadas y las que acaban antes (por haber encontrado mejor camino) vuelven a comenzar una nueva búsqueda. Así, los mejores caminos tendrán más probabilidad de ser repetidos. ***Es la típica forma en la naturaleza.***

Algoritmo Hormigas *versus* Metaheurísticas Mejora y Constructivas



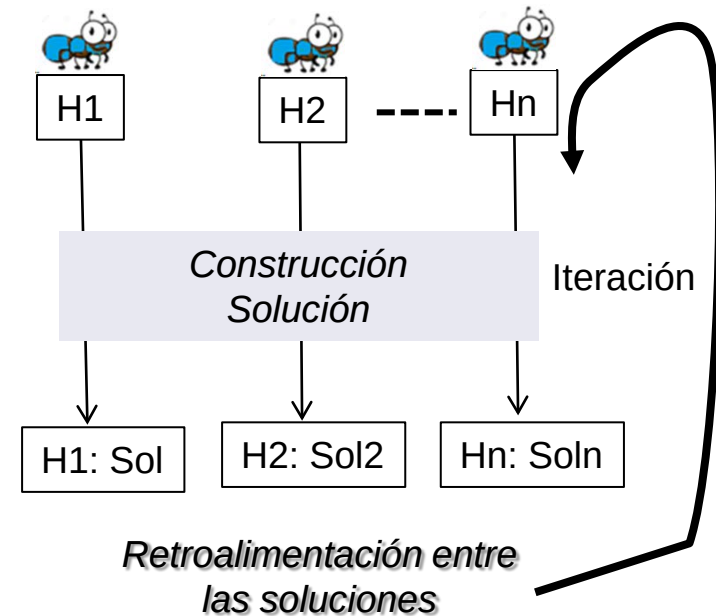
Búsqueda Local en la vecindad

- Enfriamiento Simulado, Tabú, Búsqueda en Haz.
- Función de Evaluación $f(n)$
- No hay realimentación entre soluciones, salvo la propia solución actual



Constructivo (GRASP)

- Construcción de Solución (Múltiples soluciones independientes)
- Mejora Local posterior
- Función heurística $f(n)$



Algoritmo de las Hormigas

- Construcción de las Soluciones
- Paso a paso, por múltiples hormigas
- Retro-alimentación entre las soluciones obtenidas

Aplicaciones Típicas:

- Enrutamiento de vehículos. Determinación de rutas.
- Diseño de circuitos lógicos.
- Asignación de recursos. Asignación de Horarios.
- Problemas de Recubrimiento.
- Control Inteligente.
- Etc.

Bee colony optimization part i: The algorithm overview. Tatjana Davidovic, Dusan Teodorovic, Milica Selmic (2015) DOI:10.2298/YJOR130515034A

Monografía: Ant Colony Optimization - Techniques and Applications

Edited by Helio J.C. Barbosa, InTech (Ed), 2013 .

Monografía: Ant Colony Optimization – Methods and Applications

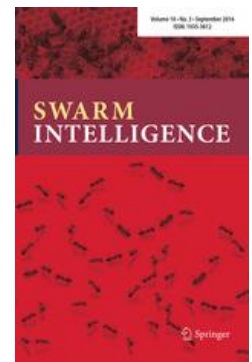
Edited by A. Ostfeld InTech (Ed), 2014

A concise overview of applications of ant colony optimization.

Stützle, López, Dorigo. U. Libre de Bruselas. Tech. Rpt

Main Ant Colony Optimization

Routing	Traveling salesman	
	Vehicle routing (VRP)	
	VRP with time windows VRPMTWMV VRP with loading con	Constraint satisfaction Course timetabling Ambulance location MAX-SAT
Scheduling	Team orienteering Sequential ordering TSP with time window Single machine	Assembly line balancing Simple assembly line balancing Supply chain management
		Machine learning
	Flow shop	Bayesian networks Classification rules
	Industrial scheduling Project scheduling Group shop Job shop	Bioinformatics
	Open shop Car sequencing	Shortest common supersequence Protein folding Docking Peak selection in biomarker identification DNA sequencing Haplotype inference
Subset	Multiple knapsack	
	Maximum independent set Redundancy allocation Weight constraint graph tree partitioning	
	Bin packing Set covering Set packing I-cardinality trees Capacitated minimum spanning tree Maximum clique Multilevel lot-sizing	
Assignment and layout	Edge-disjoint paths Feature selection Multicasting ad-hoc networks	
	Quadratic assignment	
	Graph coloring Generalized assignment Frequency assignment	



Instance	GA		ACO		SA	
	Best Distance	Time (s)	Best Distance	Time (s)	Best Distance	Time (s)
bier127	419224	~3	124651.524	~27	265289	~0.3
berlin52	11551	~1	7721.432	~4	10586	~0.2
ali535	9466	~5	1510.637	~62	6471	~0.3



Figure 12. Evolution of GA, ACO and SA results over best distance for bier127, berlin52, and ali535

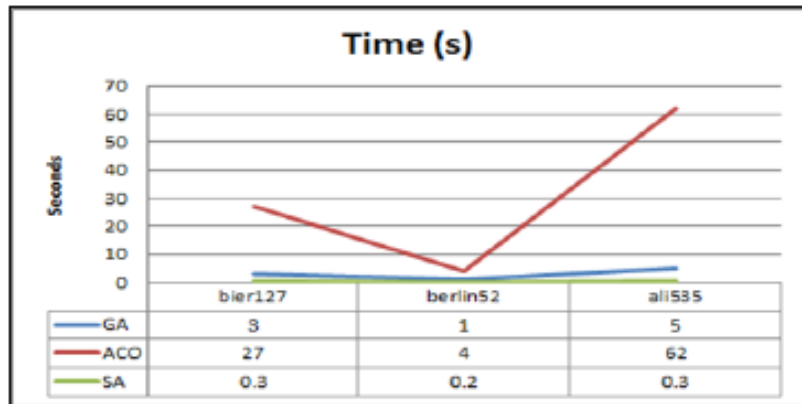


Figure 13. Evolution of GA, ACO and SA results over time(s) for bier127, berlin52 and ali535

El tipo de problemas que se pueden resolver por medio de ACO pertenece al **grupo de problemas de camino mínimo** que pueden representarse en forma de **grafo ponderado $G = (N, A)$** .

- **N:** *nodos del grafo*,
- Las *soluciones se corresponden con caminos en el grafo* (secuencias de nodos o aristas),
- Existen **costes asociados a cada arista**,
- Las conexiones tendrán asociados rastros de feromona τ y **valores heurísticos η** , que *representan* la información heurística disponible en el problema.

In: Performance Comparison of Simulated Annealing, GA and ACO Applied to TSP. Int J of Intelligent Computing Research 6, 4, Dec2015

Using Ant Colony Optimization to Solve Train Timetabling Problem of Mass Rapid Transit

Jau-Ming Su¹ Jen-Yu Huang²

¹Department of Transportation Technology and Logistic Management, Chung-Hua University

²Division of Traffic Engineering Institute of Civil Engineer, National Taiwan University

Artículos en Poliformat



ELSEVIER

International Journal of Production Economics

Volume 149, March 2014, Pages 131-144

Ant Colony System-based Applications to Electrical Distribution System Optimization

Gianfranco Chicco
Politecnico di Torino
Italy

An ant colony based timetabling tool

Thatchai Thepphakorn ^a, Pupong Pongcharoen ^a  , Chris Hicks ^b

 **Show more**

<https://doi.org/10.1016/j.ijpe.2013.04.026>

Get ri



International Journal of Innovative Research in Computer and Communication Engineering

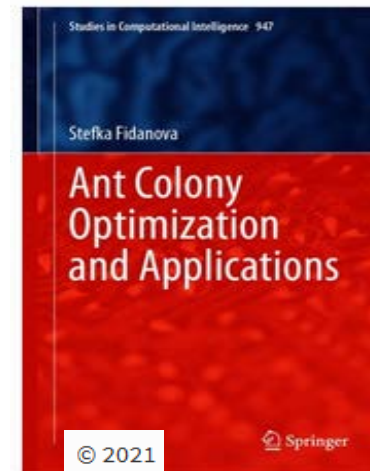
(An ISO 3297: 2007 Certified Organization)

Website: www.ijircce.com

Vol. 5, Issue 3, March 2017

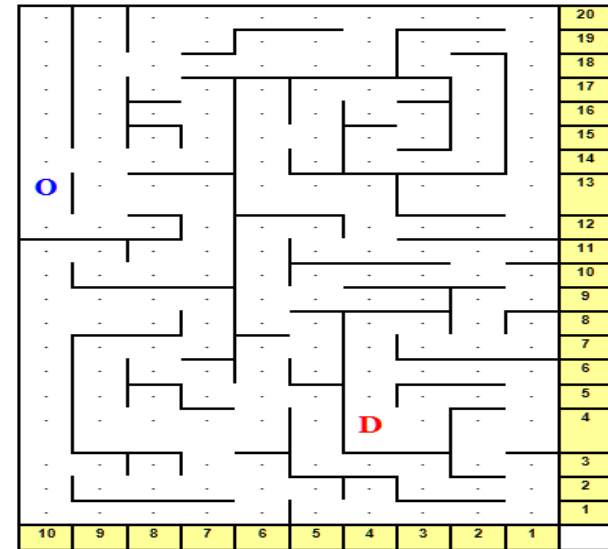
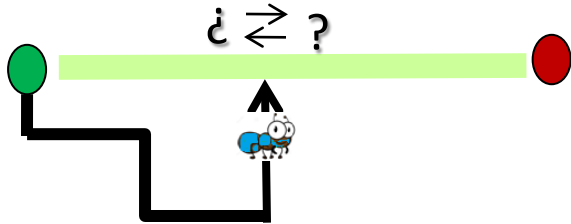
Timetable Generation Using Ant Colony Optimization Algorithm

Priyanka Gore¹, Poonam Sonawane¹, Suneha Potdar²

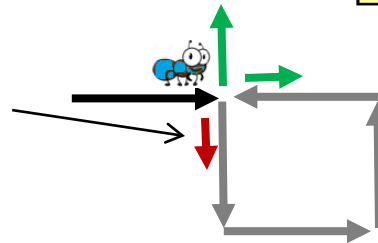


Aplicación del Algoritmo de las Hormigas en Laberintos

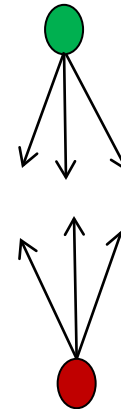
Información de feromonas: valor y dirección



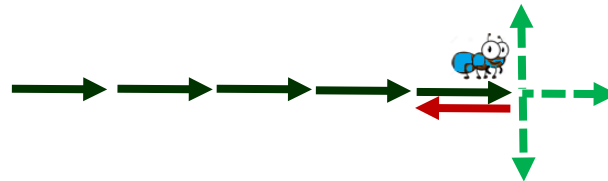
Ciclos: Olvidar ciclo, penalizar selección



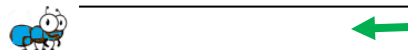
Búsqueda Bidireccional



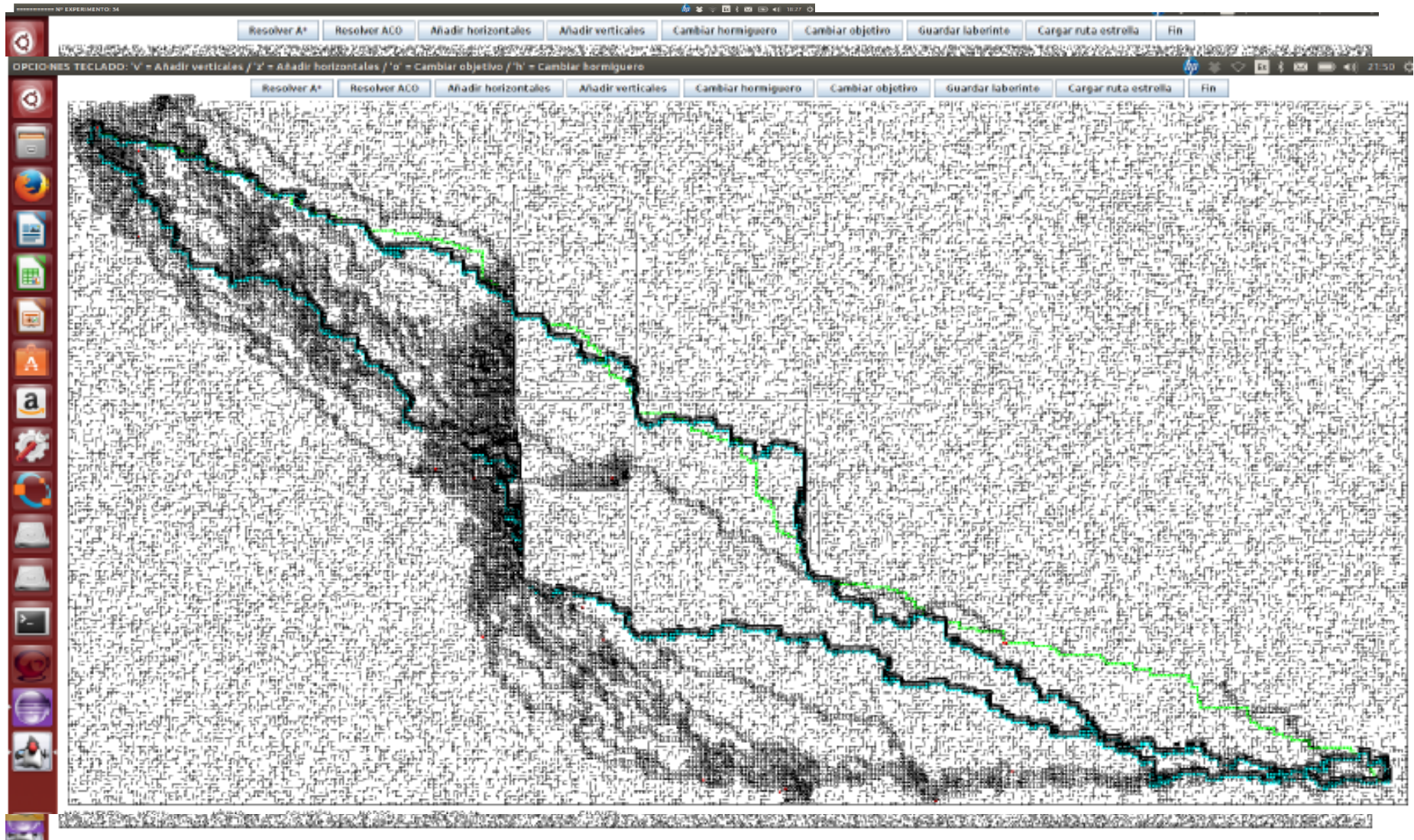
Penalizar vuelta atrás



Bloqueo del camino: Permitir vuelta atrás

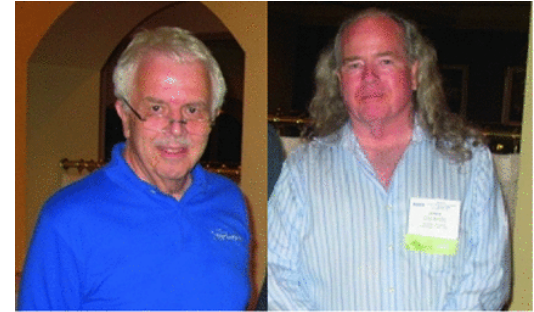


Ejemplos de Laberintos



Enjambre de Partículas

(Particle Swarm Optimization - PSO)



Russel Eberhart and James Kennedy

Kennedy & Eberhart, 1995,



Particle swarm optimization. *James Kennedy and Russell Eberhart.*

Proceedings of IEEE International Conference on Neural Networks, pg 1942–1948, 1995. IEEE Press.

<http://www.swarmintelligence.org/>

<http://www.particleswarm.info/>

Bibliografía

- Applying Particle Swarm Optimization Adıgüzel Mercangöz,
- Particle Swarm Optimization Xin-She Yang, in Nature-Inspired
- Performance comparison of five metaheuristic nature-inspired systems. Shonak Bansal. Artificial Intelligence Review 53:55
- A simplified multi-objective particle swarm optimization. Alghamdi Swarm Intelligence (2020) 14:83–116
- Application of particle swarm optimization algorithm in power system. Nazari-Heris, Mohammadi-Ivatloo. Handbook of Neural Computing
- Particle Swarm Optimization Algorithms. Introduction to Nature-Inspired Computing Lindfield, John Penny, 2017
- A survey of Particle Swarm Optimization techniques for solving the Traveling Salesman Problem, Larabi, Sainte. Artif Intell Rev (2015) 44:537–546.
- A Comprehensive Survey on Particle Swarm Optimization Algorithms Wang, Genlin. Mathematical Problems in Engineering, 2015

 [ACO Timetabling Tool.pdf](#)

 [Ant Colony Optimization Book.pdf](#)

 [Aplicaciones Algoritmo Hormigas.pdf](#)

 [Aplicaciones Metaheurística POO.pdf](#)

 [APPLICATION IN POWER SYSTEM PROBLEMS.pdf](#)

 [Comparison of 5 novel nature inspired metaheuristic.pdf](#)

 [From ants to whales metaheuristics for all tastes \(2020\).pdf](#)

 [Metaheuristic Review 2019.pdf](#)

 [multi-objective particle swarm optimization.pdf](#)

 [PARTICLE SWARM OPTIMIZATION.pdf](#)

 [Particle Swarm Optimization Algorithms.pdf](#)

 [PSO for engineering problems.pdf](#)

 [PSO-Survey.pdf](#)

 [PSO Viajante Multiobjetivo.pdf](#)

 [State of the art review of intelligent scheduling.pdf](#)

 [Survey PSO para timetabling.pdf](#)

Metaheurística bioinspirada, basada en la interacción social de las **bandadas de pájaros**, **bancos de peces**, etc. : Enjambres de Partículas.

- En un grupo (enjambre) se establecen diferentes jerarquías, de acuerdo a las características de cada individuo.
- Particularmente, existe un **líder**, reconocido y seguido por los demás individuos del grupo.
 - El líder tiene habilidades que le permiten ser guía para buscar alimento.
 - Los individuos de menor jerarquía confían en la capacidad del líder para dirigirlos.

En el proceso de búsqueda de alimento (búsqueda solución):

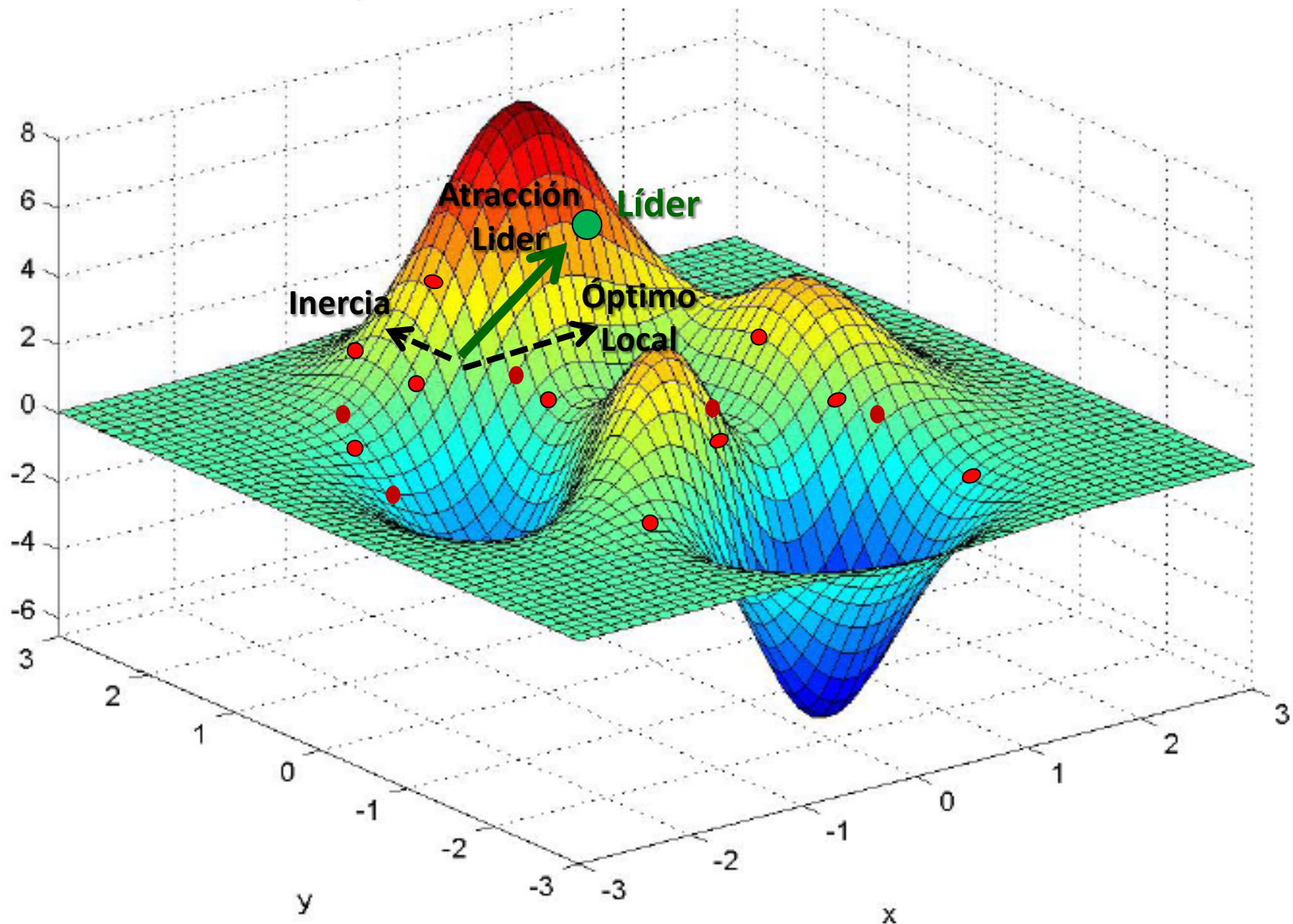
- El líder guía al grupo por donde sospecha encontrarán alimento.
- Todos los individuos siguen al líder, pero tratando también a su vez de localizar alimento y evitar colisiones.
- Eventualmente, comunican al grupo nuevas ubicaciones de alimento (existe una comunicación entre el conjunto de partículas del enjambre).



El grupo no es estático: Puede surgir un individuo con mejores capacidades que el actual, sustituyéndolo en el mando del grupo: **nuevo líder**.

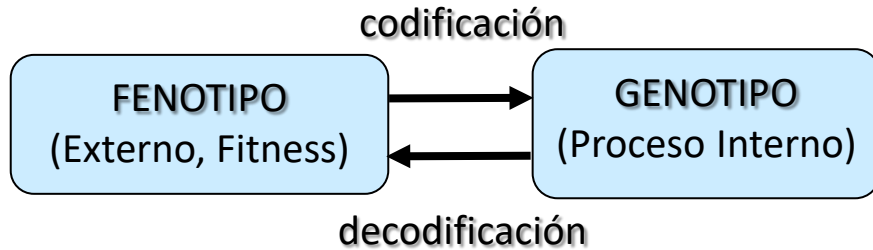
- El **proceso de selección** se basa en comparar el valor de aptitud (valor de la función objetivo) de *cada miembro* del grupo con el valor de aptitud del actual líder: el miembro del grupo con mejor evaluación será el nuevo líder si supera al líder actual.

Idea Básica: Cada **partícula** sobrevuela el espacio de soluciones n-dimensional en busca de soluciones óptimas.



Modelización del problema y Proceso

Cada partícula (individuo) del enjambre (o nube de partículas) representa una solución y realiza un proceso de búsqueda de mejores soluciones.



- La búsqueda de cada partícula_i en el espacio de soluciones se realiza en base al “conocimiento propio”, a la influencia del líder y al movimiento de la partícula:
 - i. *Solución propia: $Pbest_i$*
 - ii. *Solución aportada por el líder: $Gbest_i$*
 - iii. *Movimiento de la partícula: V_i*
- Se plantean unas reglas matemáticas para el movimiento de cada partícula.
- El objetivo final es que el enjambre de partículas converja hacia las mejores soluciones

MODELIZACIÓN

Partícula_i $\cong$ Solución_i

$\Rightarrow$

Conjunto de decisiones (o valores) sobre cada variable del problema:

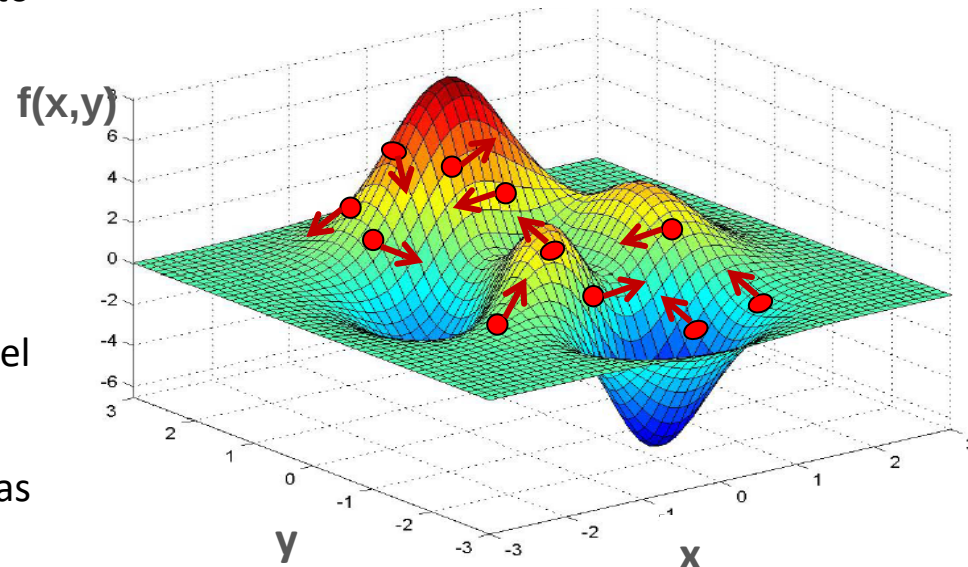
Partícula_i = [x_{i1} , x_{i2} , x_{i3} , ..., x_{in}]

Típicamente vector de Reales o Enteros



Espacio de Búsqueda $\cong$
Espacio de Decisión n-dimensional

Ejemplo 2-dimensional: Partícula_i: (x_i , y_i)



Esquema Básico Algoritmo PSO

Crear **enjambre inicial** de partículas (soluciones en el espacio de búsqueda) y **velocidades iniciales** (típicamente de manera aleatoria).

Soluciones : $\{ (Posición_i, Velocidad_i) \}$

Mientras no_fin (n^o evaluaciones, optimalidad, etc.)

1. Evaluar cada partícula_i:

Evaluación de Soluciones

- **Pbest_i**: Mejor solución encontrada por la partícula_i
- **V_i**: Velocidad la partícula_i (incluye dirección)
- **Gbest**: Mejor solución encontrada por el enjambre (líder).

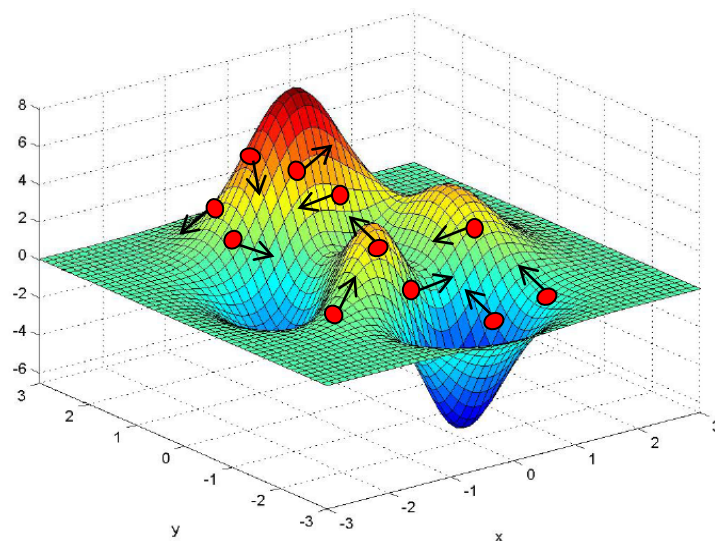
Búsqueda nuevas soluciones

2. Obtener función de vuelo de las partículas (**V'**_i):

Obtener nueva velocidad y dirección de cada partícula (**V'**_i) en base a posición del líder (**Gbest**) y al propio proceso de búsqueda local (**V_i**, **Pbest_i**).

3. Actualizar la posición de cada partícula (nueva solución).

Aplicar función de vuelo a la posición actual y obtener nueva posición (**X'**_i)



Para cada partícula_i:

Posición_i: Codificación de una solución S_i
Genotipo (n dimensional)

$[X_{i1}, X_{i2}, X_{i3}, \dots, X_{in}]$

Velocidad_i: Obtención de nueva solución
 $S'_i \in N(S_i)$
Típicamente n-dimensional

Pbest_i: Mejor solución de la partícula_i

Gbest: Mejor solución del enjambre.

Esquema Básico Algoritmo PSO

Crear enjambre inicial de partículas (soluciones en el espacio de búsqueda) de manera aleatoria y velocidades iniciales.

Soluciones : { (Posición_i, Velocidad_i) }

Mientras no_fin (*nº evaluaciones, optimalidad, etc.*)

1. Evaluar cada partícula_i:

Evaluación de Soluciones

- **Pbest_i**: Mejor solución encontrada por la partícula_i
- **V_i**: Velocidad la partícula_i (incluye dirección)
- **Gbest**: Mejor solución encontrada por el enjambre (líder).

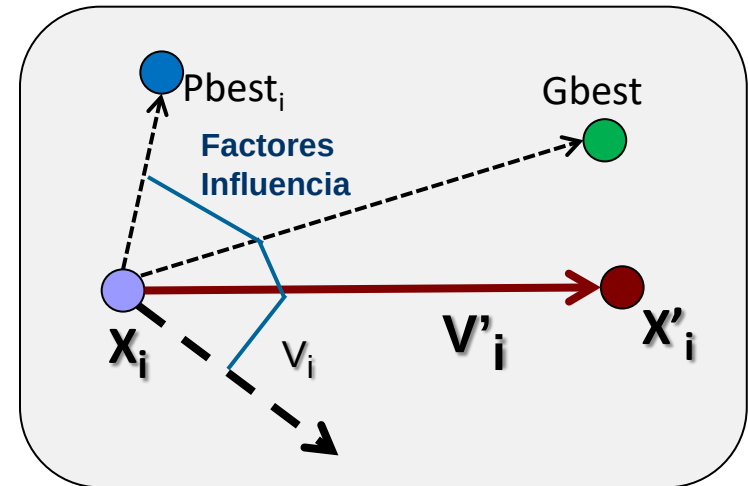
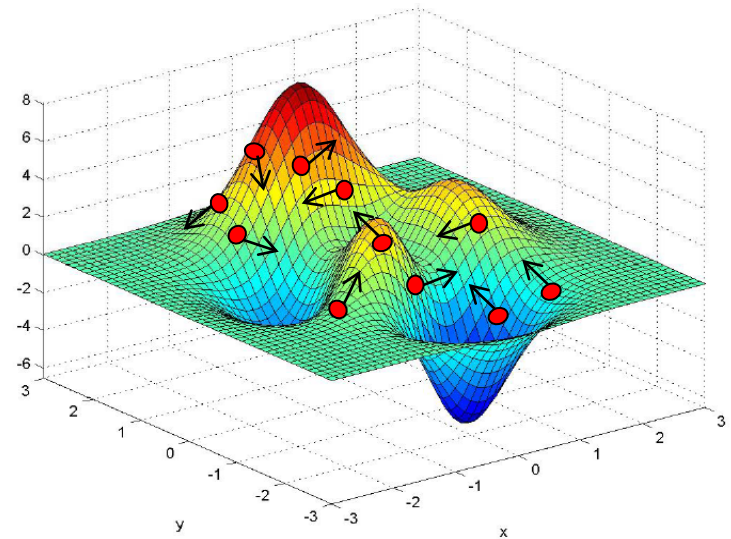
Búsqueda nuevas soluciones

2. Obtener función de vuelo de las partículas (**V'_i**):

Obtener nueva velocidad y dirección de cada partícula (**V'_i**) en base a posición del líder (**Gbest**) y al propio proceso de búsqueda local (**V_i**, **Pbest_i**).

3. Actualizar la posición de cada partícula (nueva solución).

Aplicar función de vuelo a la posición actual y obtener nueva posición ($X'_i = X_i \otimes V'_i$)



Cada partícula explora el espacio de soluciones a partir de su mejor posición y de la mejor posición global

Ecuación del Movimiento

Cada Partícula-*i* se caracteriza por (espacio *n*-dimensional):

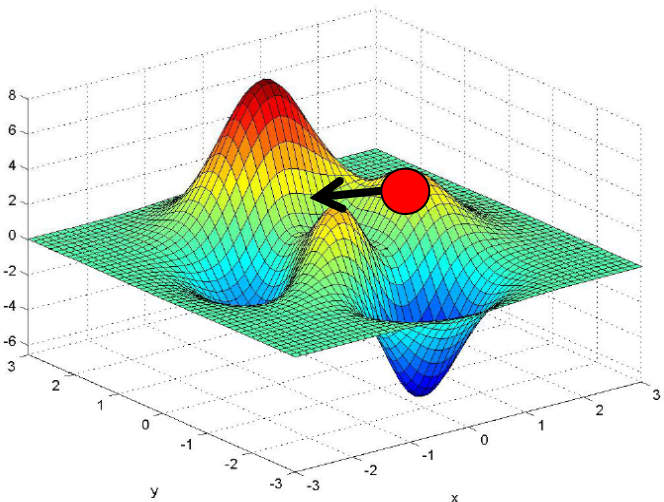
Posición $\mathbf{X}_i = [x_{i1}, x_{i2}, \dots, x_{in}]$

Velocidad $\mathbf{V}_i = [v_{i1}, v_{i2}, \dots, v_{in}]$

Mejor posición personal (en su evolución): $\mathbf{P}_i = [p_{i1}, p_{i2}, \dots, p_{in}]$

Además, mejor posición global alcanzada por todas las partículas:

$\mathbf{G} = [g_1, g_2, \dots, g_n]$

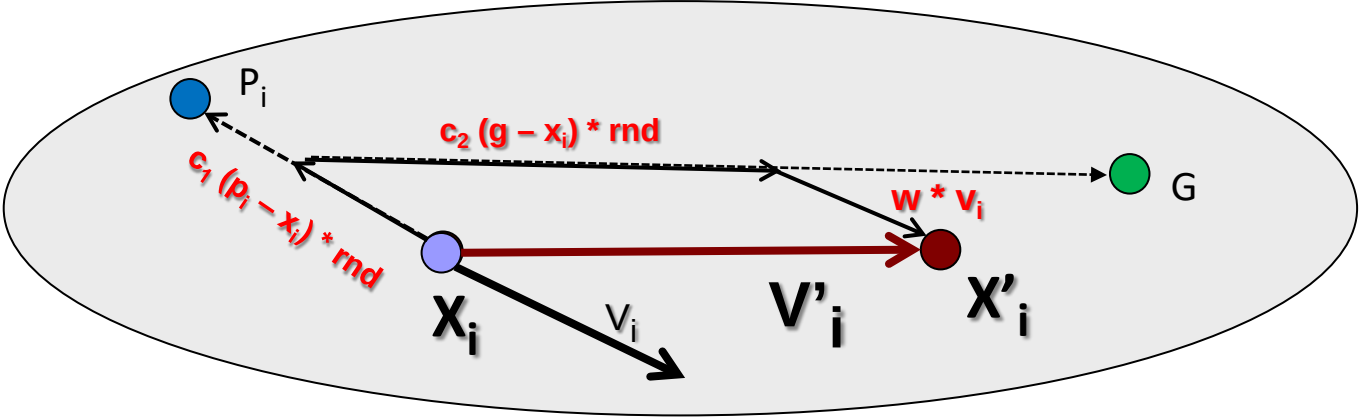


En cada iteración del proceso de búsqueda, cada Partícula-*i* actualiza su velocidad $\mathbf{V}'_i$, donde cada componente $[v'_{i1}, v'_{i2}, \dots, v'_{in}]$, se obtiene:

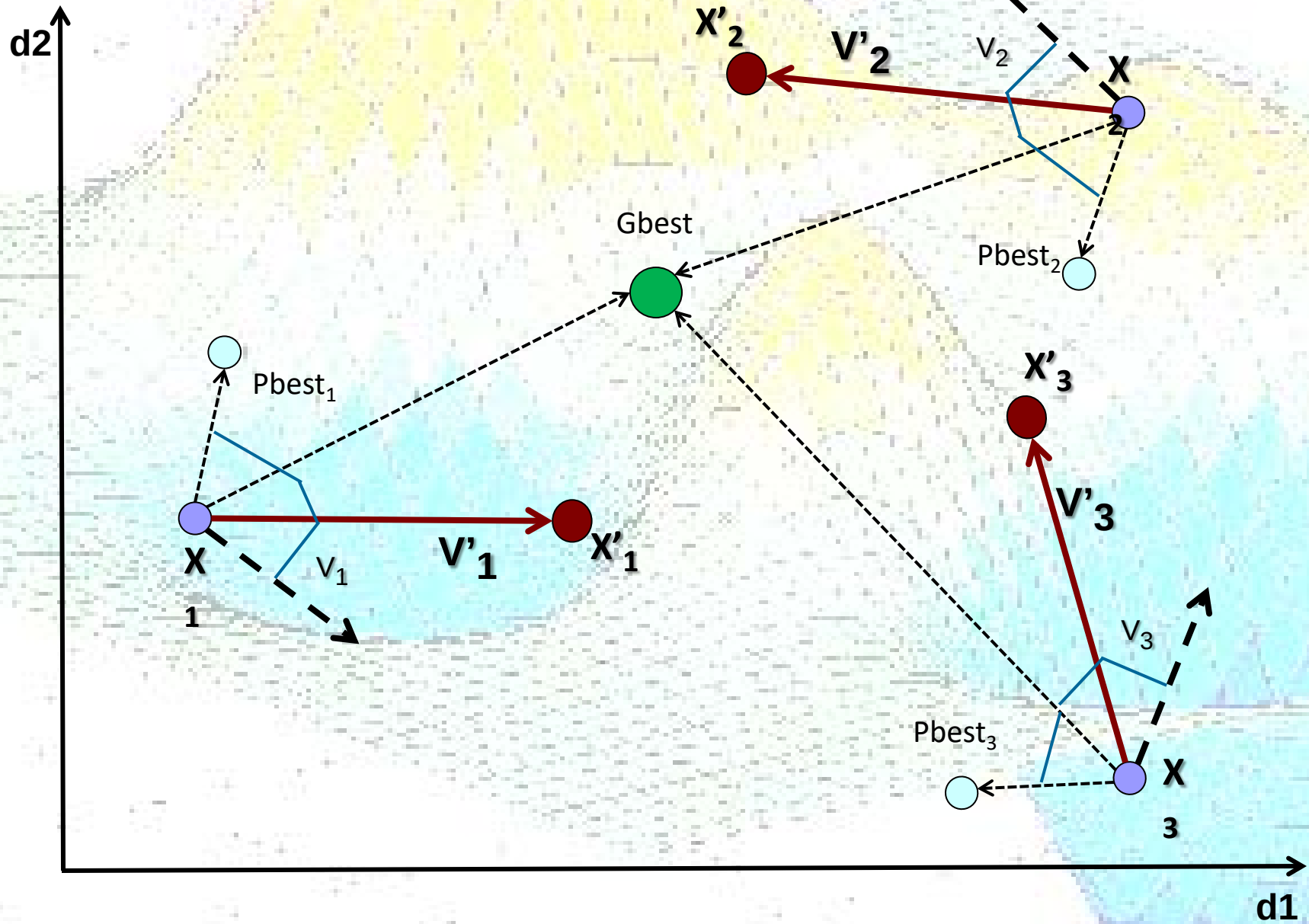
Típicamente: $w + c_1 + c_2 = 1$

$$\begin{aligned} \mathbf{V}'_{ik} = & \mathbf{w} * \mathbf{v}_{ik} \\ & + \mathbf{c}_1 (\mathbf{p}_{ik} - \mathbf{x}_{ik}) * \text{random}(0, 1) \\ & + \mathbf{c}_2 (\mathbf{g}_k - \mathbf{x}_{ik}) * \text{random}(0, 1) \end{aligned}$$

$w \in [0, 1]$: Inercia, o impacto velocidad previa V_i .
 c_1 : Parámetro cognitivo (experiencia individual)
 c_2 : Parámetro social (influencia enjambre, cooperación)



Cooperación en la búsqueda



Algorithm 1: classical PSO

input : Objective function $f : S \rightarrow \mathbb{R}$ to be minimized

output: $G \in \mathbb{R}^D$

Inicialización

// Initialization

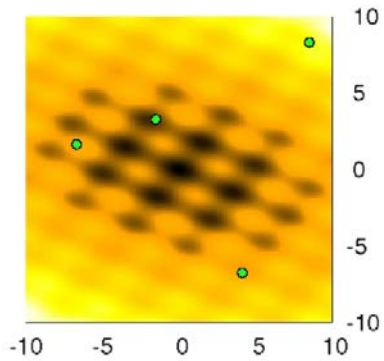
```
1 for  $n = 1 \rightarrow N$  do           Para cada partícula  
2   Initialize position  $X^n \in \mathbb{R}^D$  randomly;           Posición y Velocidad inicial  
3   Initialize velocity  $V^n \in \mathbb{R}^D$ ;           Mejor Posición Local inicial  
4   Initialize local attractor  $L^n := X^n$ ;  
5 Initialize  $G := \operatorname{argmin}_{\{L^1, \dots, L^N\}} f$ ;           Mejor Posición Global inicial
```

// Movement

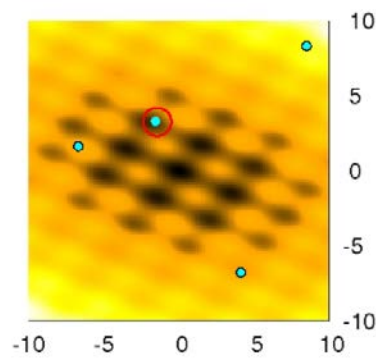
Optimización

```
6 repeat  
7   for  $n = 1 \rightarrow N$  do           Para cada partícula  
8     for  $d = 1 \rightarrow D$  do           Para cada dimensión           Actualiza Velocidad  
9        $V^{n,d} := \chi \cdot V^{n,d} + c_1 \cdot \operatorname{rand}() \cdot (L^{n,d} - X^{n,d}) + c_2 \cdot \operatorname{rand}() \cdot (G^d - X^{n,d})$ ;  
10       $X^{n,d} := X^{n,d} + V^{n,d}$ ;  
11      if  $f(X^n) \leq f(L^n)$  then  $L^n := X^n$ ;           Actualiza Óptimo Local  
12      if  $f(X^n) \leq f(G)$  then  $G := X^n$ ;           Actualiza Óptimo Global  
13 until Termination criterion holds;  
14 return  $G$ ;
```

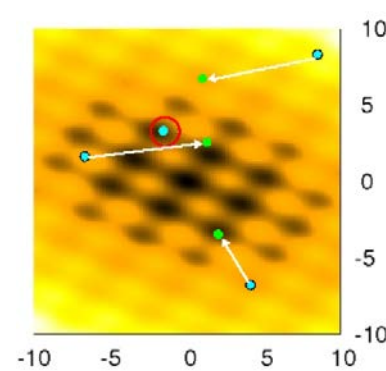
Ejemplo (M. Montes, U. L. Bruxelles)



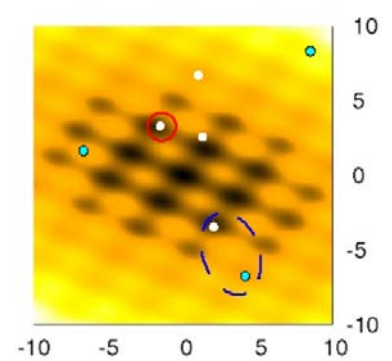
Creación de partículas y evaluación según $f(x)$



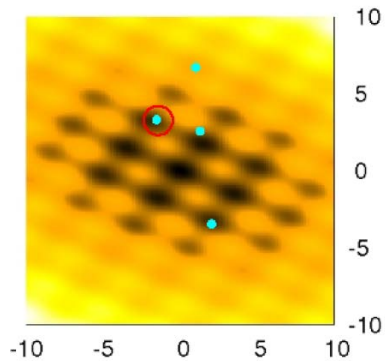
Determinación mejor partícula.



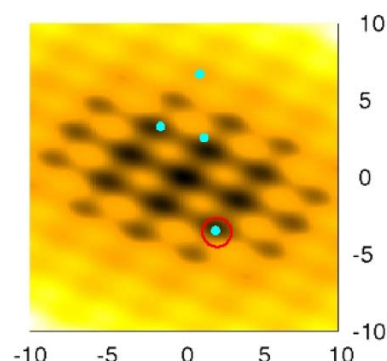
Actualización posición partículas según reglas movimiento.



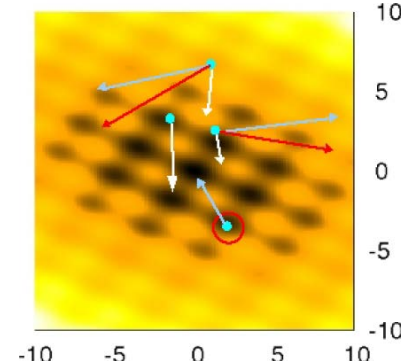
Evaluación nuevas posiciones.
Si nueva posición es mejor que la previa, actualizar posición.



Nuevas posiciones.



Determinación nueva mejor partícula.



Actualizar dirección y velocidades según reglas movimiento.

Repetir (3) hasta condición parada se cumpla.

Parámetros PSO

Como en toda metaheurística, es importante el ajuste de parámetros (diferentes variantes propuestas en PSO).

nº partículas (tamaño del enjambre)

Dependiente de la irregularidad de la hiper-superficie del fitness: *Cuanto más irregular, serán necesarias más partículas.*

$$\mathbf{v}'_{ik} = \mathbf{w} * \mathbf{v}_{ik} + \mathbf{c}_1 (p_{ik} - x_{ik}) * RND(0, 1) + \mathbf{c}_2 (g_k - x_{ik}) * RND(0, 1)$$

w (inercia).

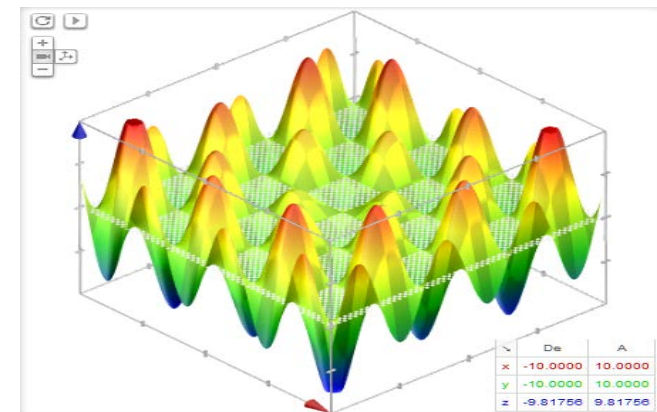
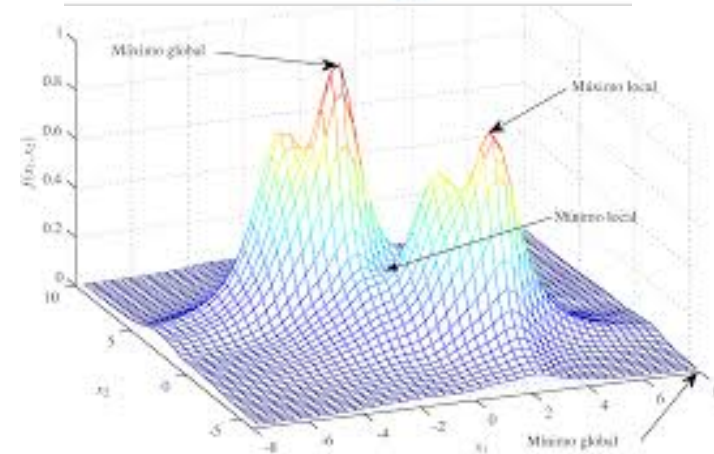
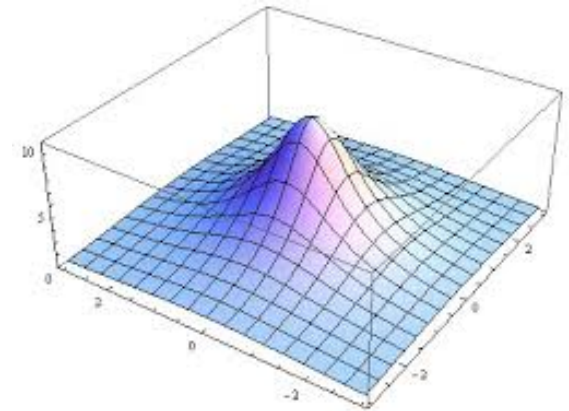
- Factor de *exploración* en búsqueda local
- Usualmente tiende a incrementarse en la búsqueda

c1: factor cognitivo personal (memoria)

- Controla el movimiento hacia su propio óptimo (*explotación local*)

c2: factor cognitivo social (cooperación)

- Factor de *explotación global*. Controla el movimiento hacia óptimo global.
- Un valor alto puede producir caer en óptimos locales.



Función de Cruce

La función de cruce (o de vuelo) obtiene la nueva posición X_{i+i} de la partícula, a partir de la posición actual X_i y la nueva velocidad V'_i :

$$V'_{ik} = c_2 * \text{RND}() * (g_k - x_{ik}) + c1 * \text{RND}() * (p_{ik} - x_{ik}) + w * v_{ik}$$

$$X_{i+i} = X_i \otimes V'_i$$

V'_i combina la mejor posición del enjambre (G , como información global), la mejor posición local (P_i , como información local) , y la velocidad anterior (V_i).

La operación de cruce es dependiente del diseño del sistema!
(dimensiones / #variables)

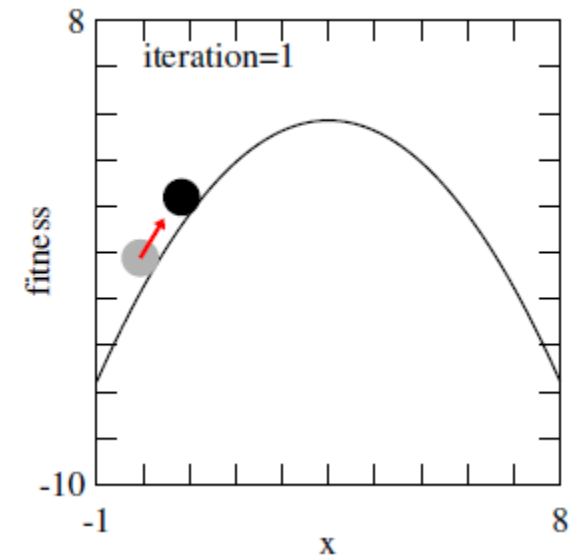
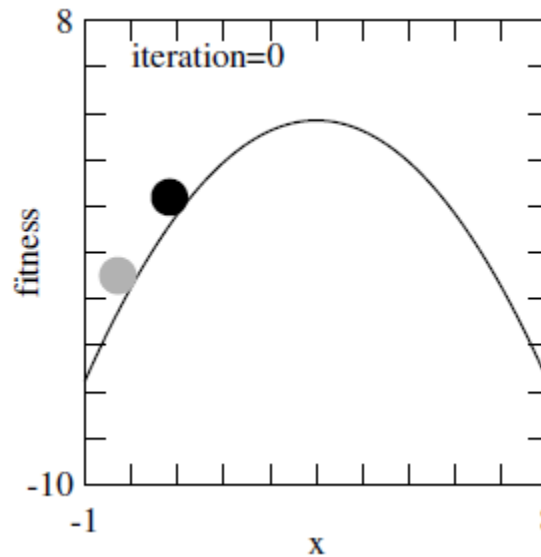
Ejemplo 1. Una dimensión

Variable: $x \in [-1, 8]$

Objetivo: maximizar $f(x)$

Nº partículas: 2

$W = 0.9$, $C1$, $C2 = 1$



Función de Vuelo v' , Actualización de la posición x' :

$$v_i(t+1) = 0.9v_i(t) + [b_i(t) - x_i(t)] + [B(t) - x_i(t)]$$

$$x_i(t+1) = x_i(t) + 0.2v_i(t+1)$$

*An Introduction to Metaheuristics for
Optimization, Chopard, Tomassini,
Springer (2018)*

Donde:

$b_i(t)$ is the particle-best,
 $B(t)$ is the global-best.

Inicialmente ($t=0$),

Partícula₁: $x_1(0)=0$, $b_1(0)=0$, $v_1(0)=0$

Partícula₂: $x_2(0)=1$, $b_2(0)=1$, $v_2(0)=0$

$B(0)=1$, $v_1(0)=v_2(0)=0$

Paso-1 ($t=1$),

Partícula₁: $x_1(1) = x_1(0) + 0.2 v_1(1) = 0 + 0.2 [0.9 * 0 + (0 - 0) + (1 - 0)] = 0.2$; $b_1(1)=0.2$

Partícula₂: $x_2(1) = x_2(0) + 0.2 v_2(1) = 1 + 0.2 [0.9 * 0 + (1 - 1) + (1 - 1)] = 1$; $b_2(1)=1$

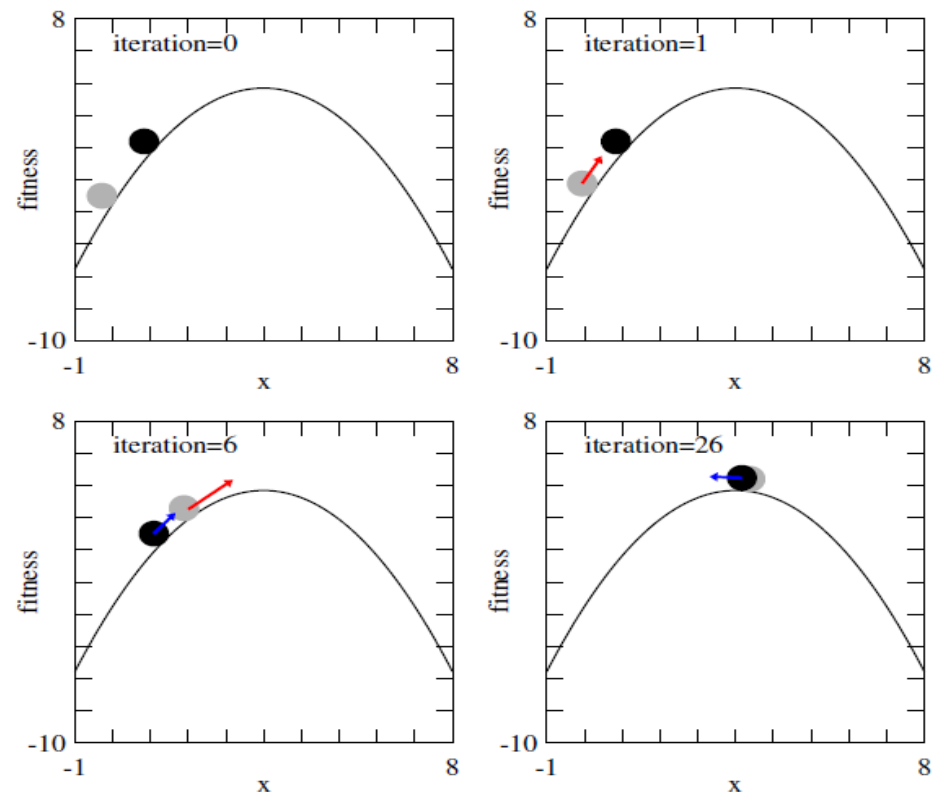
$$v_i(t+1) = 0.9v_i(t) + [b_i(t) - x_i(t)] + [B(t) - x_i(t)]$$

$$x_i(t+1) = x_i(t) + 0.2v_i(t+1)$$

$$x_1(1)=0.2, \quad b_1(1)=0.2, \quad v_1(1)=1.0$$

$$x_2(1)=1, \quad b_2(1)=1, \quad v_2(1)=0.0$$

$$B(1)=1$$



PSO example with two particles in the one-dimensional space. To better illustrate the evolution, particles are shown here moving on the fitness curve; actually, **they only move along the x axis**

Paso-2 (t=2),

Partícula₁: $x_1(2) = x_1(1) + 0.2 v_1(2) = 0,2 + 0.2 [0.9 * 1 + (0.2 - 0.2) + (1 - 0.2)] = 0,2 + 0,2 * 1.7 = 0.54;$
 $b_1(2) = 0.54$

Partícula₂: $x_2(2) = x_2(1) + 0.2 v_2(2) = 1 + 0.2 [0.9 * 0 + (1 - 1) + (1 - 1)] = 1 ; b_2(1)=1$

.... Paso 6,

Debido a la inercia, la Particula₁ arrastrará a la Particula₂

Ejemplo 2a. Dos dimensiones

Variable: $x_1, x_2 \in [-10, +10]$

Objetivo: maximizar $f(x,y) = 100 - (x^2 + y^2)$

Nº partículas: 2

$W = 0.9$, $C_1, C_2 = 1$

Inicialmente,

$x_1(0) = (6, 4)$, $b_1(0) = 100 - (6^2 + 4^2) = 48$

$x_2(0) = (3, 8)$, $b_2(0) = 100 - (3^2 + 8^2) = 27$

$B(0) = (6, 4)$,

$v_1(0) = v_2(0) = (0, 0)$

Función de Vuelo v' , Actualización de la posición x' :

$$v_i(t+1) = 0.9v_i(t) + [b_i(t) - x_i(t)] + [B(t) - x_i(t)]$$

$$x_i(t+1) = x_i(t) + 0.2v_i(t+1)$$

Paso-1,

Partícula₁: $x_1(1) = x_1(0) + 0.2 v_1(1) = (6, 4) + 0.2 [0.9 * (0, 0) + ((6, 4) - (6, 4))) + ((6, 4) - (6, 4))] = (6, 4)$;

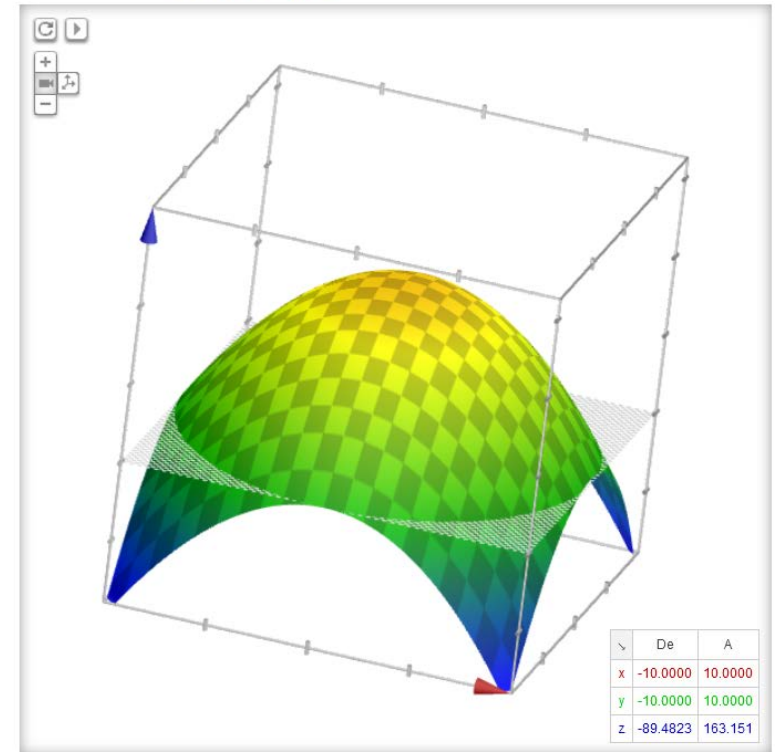
$b_1(1) = (6, 4)$

Partícula₂: $x_2(1) = x_2(0) + 0.2 v_2(1) = (3, 8) + 0.2 [0.9 * (0, 0) + ((3, 8) - (3, 8)) + ((6, 4) - (3, 8))] = (3, 8) + 0.2 (3, -4) = (3, 8) - (0.6, 0.8) = (2.4, 7.2)$ $f(2.4, 7.2) = 42.4$; **$b_2(1) = (2.4, 7.2)$**

$B(1) = (6, 4)$

Gráfico de $100 - x^2 + y^2$

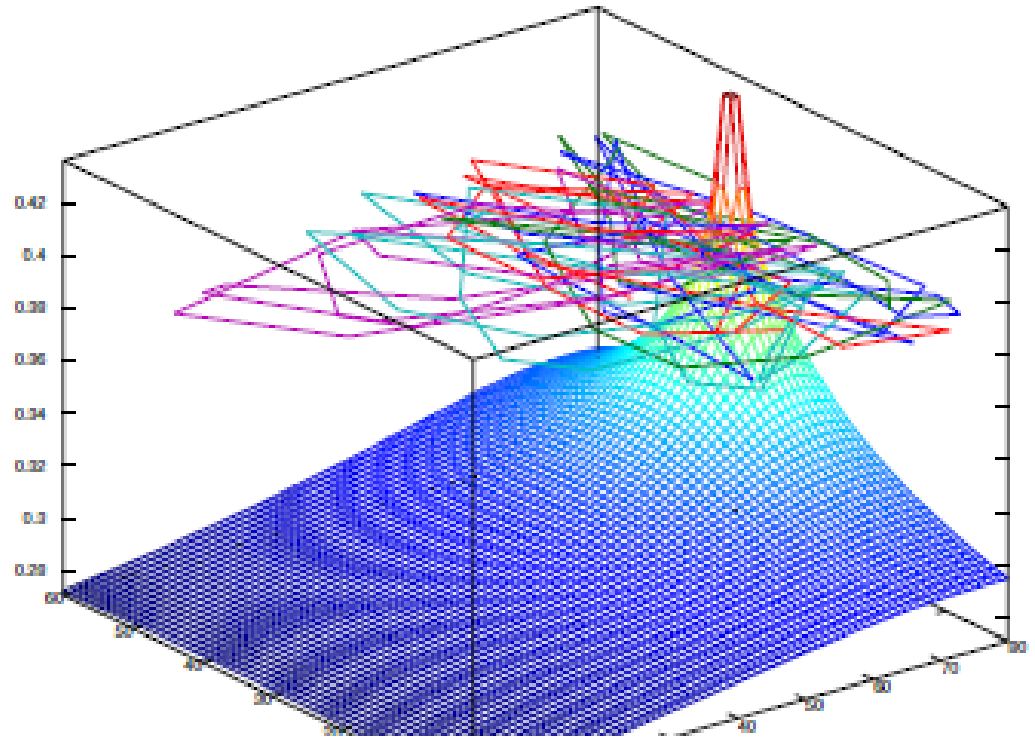
$z = 100 - (x^2 + y^2)$



Ejemplo 2b. Dos dimensiones

An example of PSO in 2D on a single-maximum fitness landscape.

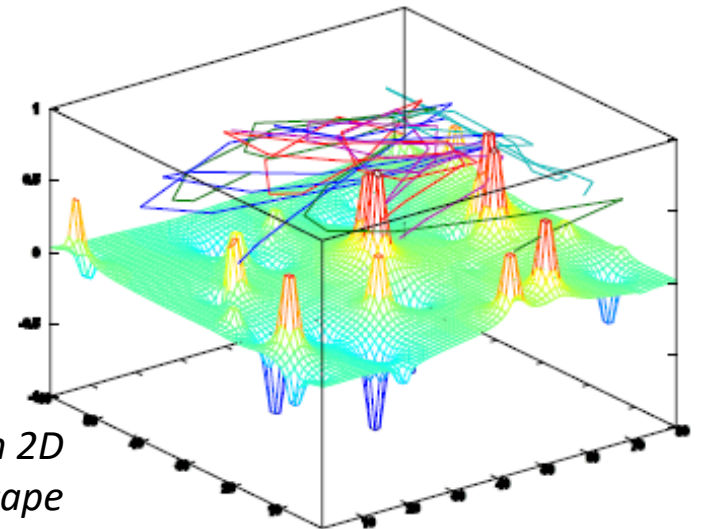
Global optimum at: (75, 36);
fitness value at the global optimum: 0.436



*With five particles, 50 iterations,
the best solution found was $B = (75; 39)$ $f(B) = 0.384$.*

With 20 particles, 100 iterations, the optimal solution was found

*An Introduction to Metaheuristics for Optimization,
Chopard, Tomassini, Springer (2018)*



*An example of PSO in 2D
with a multimodal fitness landscape*

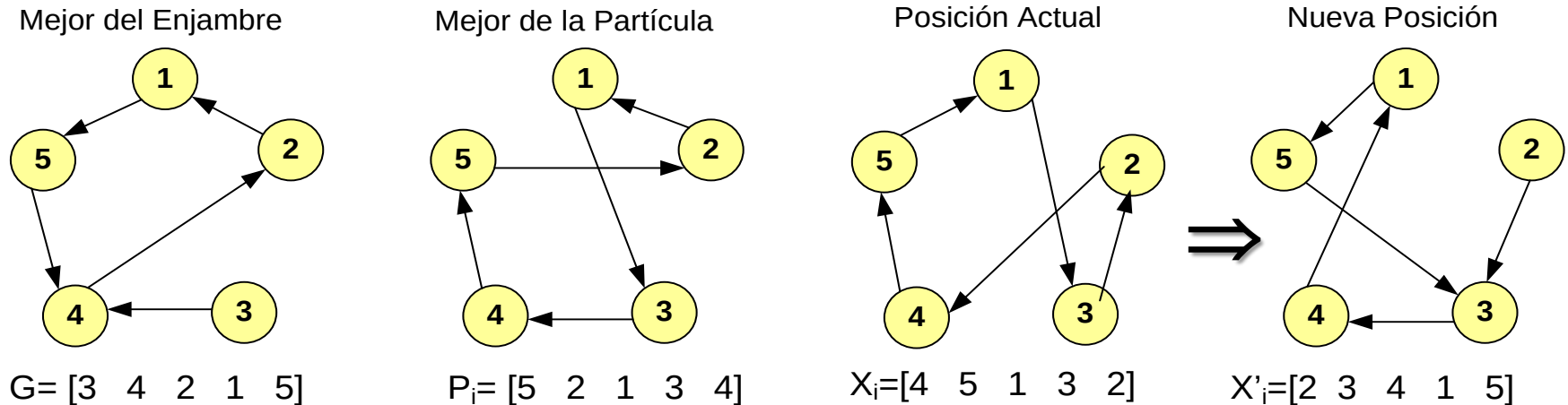
Función de Cruce

$$v'_{ik} = c_2 * \text{RND}() * (g_k - x_{ik}) + c1 * \text{RND}() * (p_{ik} - x_{ik}) + w * v_{ik}$$

$$X_{i+i} = X_i \otimes V'_i$$

La operación de cruce es dependiente del diseño del sistema, y tiene diferentes variantes.

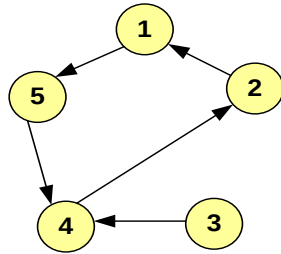
Ejemplo -3 TSP: Cada posición X_i de las partículas es un recorrido (*solución en un espacio de soluciones*)



Varias alternativas. Ejemplos en: *Particle Swarm Optimization Algorithm for the Traveling Salesman Problem*. E. Goldberg, M. Goldberg, G. Souza. In: 'Travelling Salesman Problem', 2008. *En Poliformat*

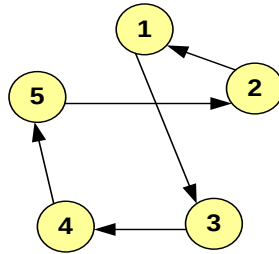
Ejemplo-3 TSP

Mejor del Enjambre



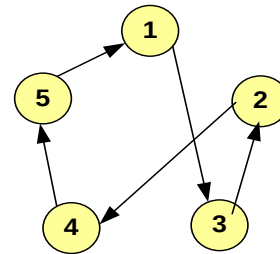
$G: [3 \ 4 \ 2 \ 1 \ 5]$

Mejor de la Partícula



$P_i: [5 \ 2 \ 1 \ 3 \ 4]$

Posición Actual



$X_i: [4 \ 5 \ 1 \ 3 \ 2]$

$$G = [3 \ 4 \ 2 \ 1 \ 5]$$

$$X_{i=} = [4 \ 5 \ 1 \ 3 \ 2]$$

$$G - X_i = [-1 \ -1 \ 1 \ -2 \ 3]$$

$$P_i = [5 \ 2 \ 1 \ 3 \ 4]$$

$$X_{i=} = [4 \ 5 \ 1 \ 3 \ 2]$$

$$P_i - X_i = [1 \ -3 \ 0 \ 0 \ 2]$$

Supongamos Velocidad Partícula:

$$V_i = [3 \ 5 \ 2 \ 6 \ -8]$$

$$V'_{ik} = c_2 * RND() * (g_k - x_{ik}) + c_1 * RND() * (p_{ik} - x_{ik}) + w * v_{ik}$$

Asumiendo $w=0,6$, $C1=0,3$, $C2=0,1$, resulta: $V'_{ik} = 0.1 * RND() * (g_k - x_{ik}) + 0.3 * RND() * (p_{ik} - x_{ik}) + 0.6 * v_{ik}$

$$V'_{i1} = 0.1 * RND() * (-1) + 0.3 * RND() * 1 + 0.6 * 3 = 2$$

$$V'_{i2} = 0.1 * RND() * (-1) + 0.3 * RND() * (-3) + 0.6 * 5 = 3$$

$$V'_{i3} = 0.1 * RND() * 1 + 0.3 * RND() * 0 + 0.6 * 2 = 1$$

$$V'_{i4} = 0.1 * RND() * (-2) + 0.3 * RND() * 0 + 0.6 * 6 = 3$$

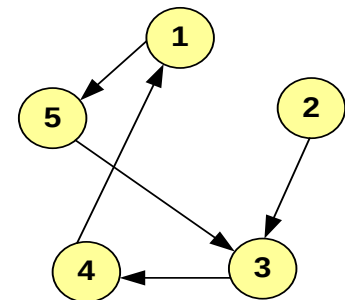
$$V'_{i5} = 0.1 * RND() * 3 + 0.3 * RND() * 2 + 0.6 * (-8) = -4$$

Luego, $V'_i = [2 \ 3 \ 1 \ 3 \ -4]$. Tal que: $X_{i+i} = X_i \otimes V'_i$

⊗: Representa los desplazamientos (swap) de cada nodo en el vector:

$$X_{i+i} = X_i \otimes V'_i = [4 \ 5 \ 1 \ 3 \ 2] \otimes [2 \ 3 \ 1 \ 3 \ -4] = [2 \ 3 \ 4 \ 1 \ 5]$$

Nueva Posición



$[2 \ 3 \ 4 \ 1 \ 5]$

Ejemplo-4 TSP

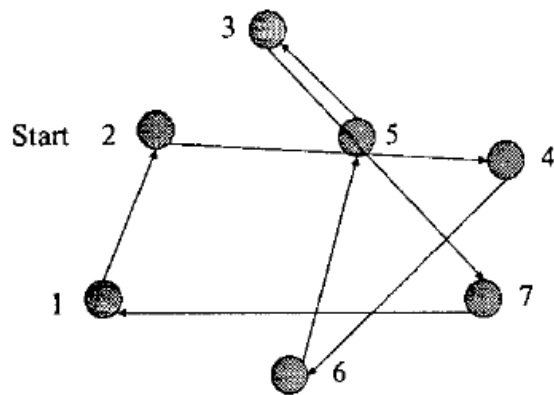
Aplicamos una nueva **Función de cruce** para obtener la nueva posición X_{i+1} de la partícula i :

$$X_{i+1} \leftarrow \text{RND}() \otimes [C_1(\%) X_i, C_2(\%) P_i, C_3(\%) G]$$

La función de cruce $\otimes$ representa la determinación probabilística (RND) de la nueva posición X_{i+1} en base a la posición actual (X_i), a la información local (P_i), y a la información global (G).

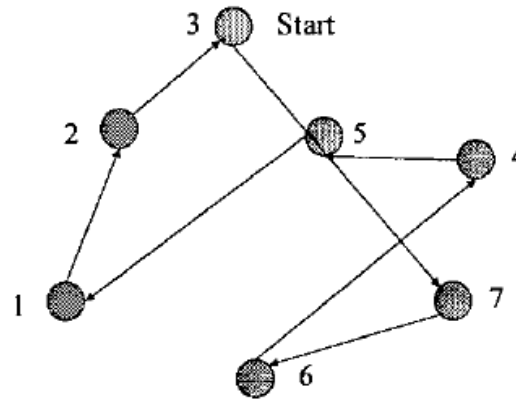
La determinación se basa en los valores (probabilidades) de C_1 , C_2 , C_3 , donde $C_1 + C_2 + C_3 = 100\%$.

Current Position



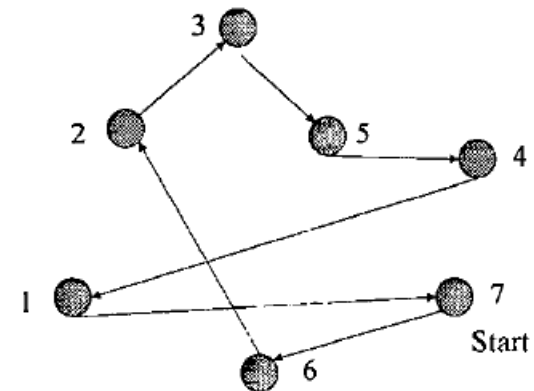
$$X_i = \{2, 4, 6, 5, 3, 7, 1\}$$

Global Best



$$G = \{3, 7, 6, 4, 5, 1, 2\}$$

Local Best



$$P_i = \{7, 6, 2, 3, 5, 4, 1\}$$

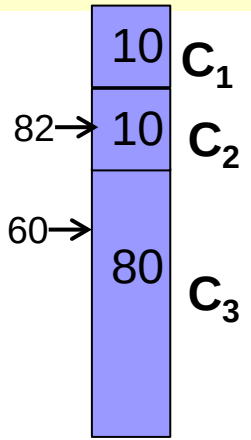
TSP for surveillance mission using particle swarm optimization , Ph. D. Thesis. Bany Secrest, Air force Institute, of Technology. Ohio, USA.

Aplicamos una nueva **Función de cruce** para obtener la nueva posición X_{i+i} de la partícula_i:

$$X_{i+i} \leftarrow \text{RND}() \otimes [C_1(\%) X_i, \quad C_2(\%) P_i, \quad C_3(\%) G]$$

Asumamos: $C_1= 10\%$, $C_2= 10\%$, $C_3= 80\%$ [C3], y la aleatoriedad RND resulta: {60, 82, 87,26, 94} **RULETA**

Estado Actual: $X_i = \{2,4,6,5,3,7,1\}$ $G = \{3,7,6,4,5,1,2\}$ $P_i = \{7,6,2,3,5,4,1\}$



PROCESO:

- 1ª Ciudad:** Por defecto, se elije como ciudad inicial la última ciudad de X_i : $X_{i+i}[1] = 1$
- 2ª Ciudad:** Por aleatoriedad, dado que $\text{RND}()=60$ es menor que C_3 , se elije la información global (G). Se elije **2**, ya que es la que sigue a **1** en G, elegida en el paso anterior $X_{i+i}[2] = 2$
- 3ª Ciudad:** Por aleatoriedad, dado que $\text{RND}()=82$ es mayor que C_3 , pero menor que C_3+C_2 , se elije la información local (P). Se elije **3**, ya que es la que sigue a **2** en P_i , elegida en el paso anterior $X_{i+i}[3] = 3$
- 4ª Ciudad:** Dado que $\text{RND}()=87 \in \{C_3, C_3+C_2\}$, se elije la información local (P). Se elije **5**, ya que es la que sigue a la ciudad **3**, elegida en el paso anterior $X_{i+i}[4] = 5$.
- 5ª Ciudad:** Dado que $\text{RND}()=26 < C_3$, se elegiría por la información global (G), pero **1** ya ha sido elegida. Se intenta por la información local (P) y se elije **4**, ya que es la que sigue a 5: $X_{i+i}[5] = 4$.
- 6ª Ciudad:** Por aleatoriedad, dado que $\text{RND}()=94 > C3+C2$, se elije la información previa (X). Se elije **7**, ya que es la penúltima ciudad del recorrido de X_i . $X_{i+i}[6] = 7$
- 7ª Ciudad:** La ciudad que queda es la 6. $X_{i+i}[7] = 6$

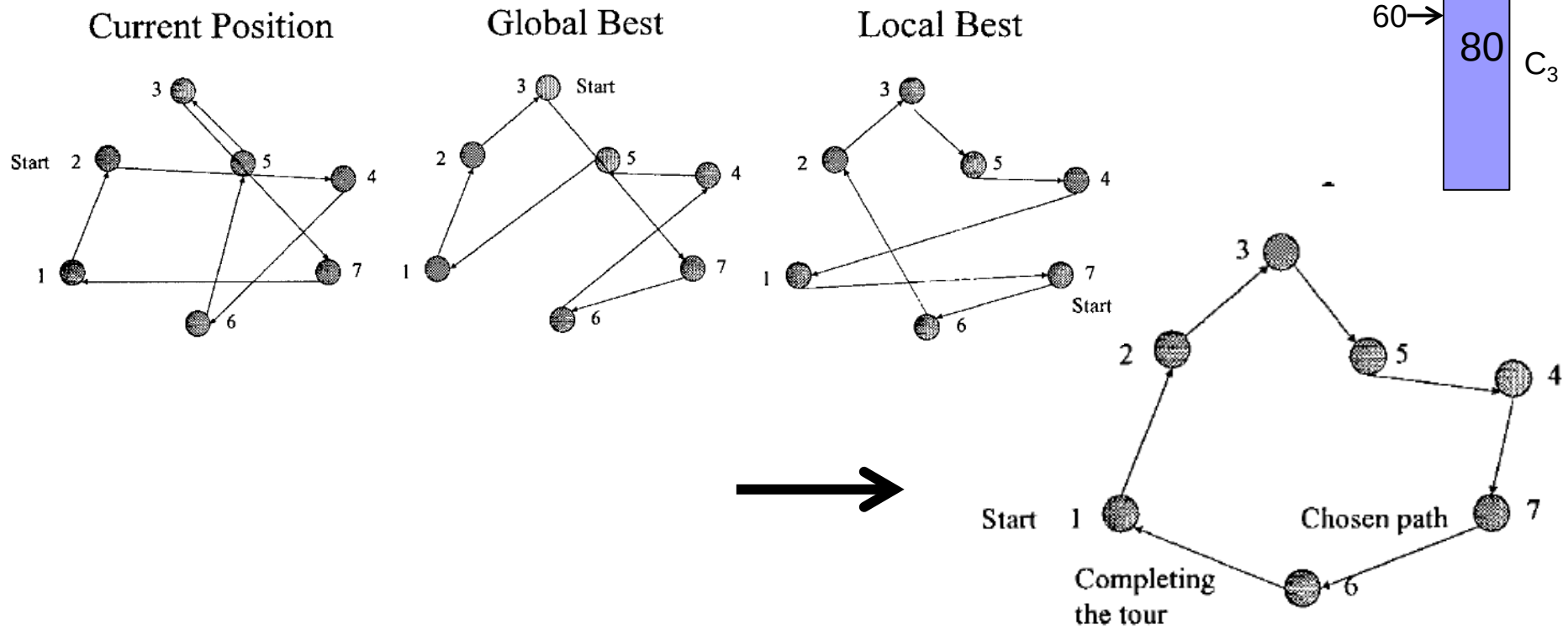
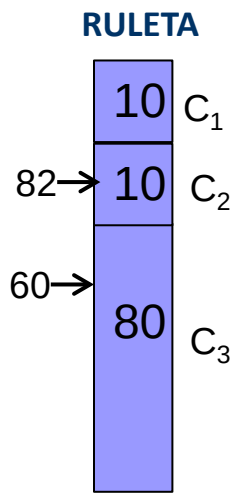
Función de cruce para obtener la nueva posición X_{i+1} de la partícula_i:

$$X_{i+1} \leftarrow \text{RND}() \otimes [C_1(\%) X_i, \quad C_2(\%) P_i, \quad C_3(\%) G,]$$

Donde la función $\otimes$ representa la selección probabilística (RND) de la nueva posición X_{i+1} en base a la posición actual (X_i), a la información local (P_i), y a la información global (G).

Estado Actual: $X_i = \{2, 4, 6, 5, 3, 7, 1\}$ $G = \{3, 7, 6, 4, 5, 1, 2\}$ $P_i = \{7, 6, 2, 3, 5, 4, 1\}$

Nuevo recorrido $X_{i+1} = \{1, 2, 3, 5, 4, 7, 6\}$



Resumen:

- Metaheurística de optimización basada en modelos de conductas sociales.
- Requiere un conjunto de ‘soluciones iniciales (partículas)’ que van ‘moviéndose’ en espacio de soluciones siguiendo unas ‘reglas matemáticas’ (inercia, propias, sociales).
- Combina *exploración* y *explotación*, ajustable con los parámetros del método:

¿¿ w (inercia), $c1$ (factor propio), $c2$ (factor social) ??

Todavía no se conoce bien la relación entre los parámetros y los conceptos de exploración y explotación.

- Asume muy pocas hipótesis del problema o del espacio de soluciones (posición y velocidad de cada partícula y la mejor posición global).
- Doble concepto de convergencia:
 - Convergencia hacia la ‘*mejor solución*’ (mejor posición global g)
 - Convergencia del enjambre hacia una *zona de ‘buenas soluciones’* (que pueden no incluir la óptima)
- Amplia utilización actual. Variantes y aplicaciones.

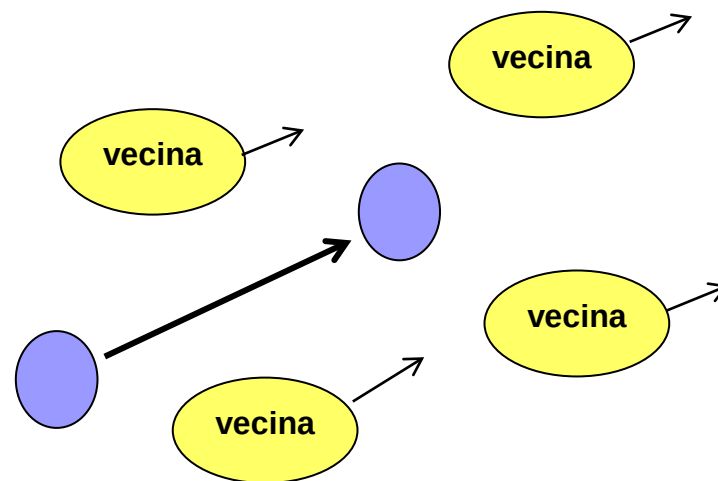
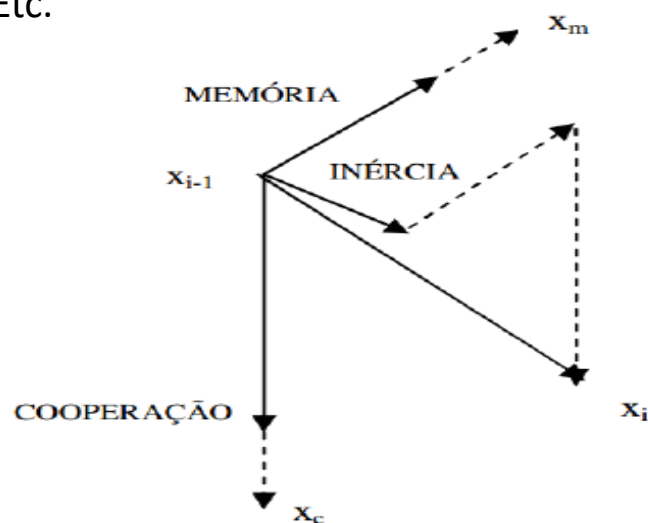
Existen muy diversas variantes PSO:

- Aplicación a problemas multi-objetivo (considerando Frontera de Pareto).
- Con o sin líder (únicamente *factor inercial y memoria*)
- Agrupación de partículas (mejores globales de cada grupo o líderes locales),
- Introducir una fuerza repulsiva entre las partículas (para evitar convergencia prematura),
- Perturbaciones momentáneas de las partículas ($\cong$ mejora local)
- Utilizar solo 'g' (mejor global): *accelerated particle swarm optimization* (APSO, Xin-She Yang 2021)):

$$v_i^{t+1} = v_i^t + \beta(g^* - x_i^t) + \alpha\epsilon_t \quad \text{simplificada como} \quad x_i^{t+1} = (1 - \beta)x_i^t + \beta g^* + \alpha\epsilon_t$$

donde ϵ es una distribución gaussiana $\{0,1\}$, y típicamente $\alpha \approx 0.1 \sim 1$ and $\beta \approx 0.1 \sim 0.7$, ($\alpha \approx 1$, $\beta \approx 0.5$)

- Incremento/decremento de la inercia (w), memoria (c1), cooperación social (c2),
- Etc.



Integrated Particle Swarm Optimization (iPSO)

- En PSO estándar, la actualización de cada partícula (X_i) se guía por dos posiciones significativas del espacio de búsqueda: P_i , G .
- Si la partícula actual se encuentra muy cerca de cualquiera de estas dos posiciones, sus capacidades de orientación se reducen o incluso se desvanecen.

iPSO incorpora una tercera partícula, llamada **partícula ponderada** (X^W):

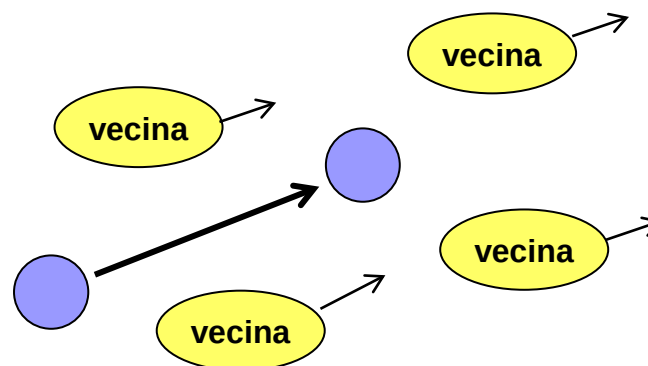
X^W es la posición ponderada media de las mejores posiciones de cada una de las M partículas del enjambre:

$$X^W = \sum_{j=1, M} c_j^w * P_j \quad ; \text{ siendo } c_j^w \text{ el factor de ponderación para cada partícula}$$

Esta nueva partícula se incorpora en una revisada función de cruce

(Interactive search algorithm: A new hybrid metaheuristic optimization algorithm.

Mortazavi, Toğan, Nuhoğlu. Engi.App. of Artificial Intelligence, 71, May 2018). En Poliformat



Hybrid Gravitational Search Algorithm & Particle Swarm Optimization (PSO – GSA)

Las partículas en un PSO clásico siguen unas reglas matemáticas de movimiento:

$$\mathbf{x}_i(t+1) = \mathbf{x}_i(t) + \mathbf{v}_i(t), \quad \text{donde} \quad \mathbf{v}_i = w * \mathbf{v}_i + C_1 (p_{ik} - \mathbf{x}_{ik}) * \text{RND} + C_2 (g_k - \mathbf{x}_{ik}) * \text{RND}$$

En el **método PSO, combinado con una Búsqueda Gravitacional (GSA)**, se asume que:

- Las partículas tienen '**masa**', tal que a mayor masa, generan mayor fuerza gravitacional. *La masa depende de la bondad (fitness) de la partícula.*
- Aparece una '**fuerza de atracción**' entre partículas, que depende de sus masas (fitness) y de su distancia relativa (en el espacio de soluciones).
- La fuerza de atracción provoca una '**aceleración**' (a_i) del movimiento de cada partícula, que depende de la combinación (Σ) de las fuerzas de atracción a la que está sometida por el resto de partículas.
- La nueva ecuación de velocidad es:

$$V_i(t+1) = w \times v_i(t) + C_1 \times a_i(t) + \left(\frac{C_1}{C_2} \right) \times (p_g - x_i)$$

Mejora:

El PSO clásico no considera el valor (absoluto) del fitness o la distancia entre (todas) las partículas en el movimiento. **El método PSO-GSA incluye el 'valor' del 'fitness' y 'distancia' en la metaheurística.**

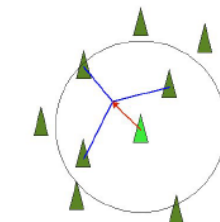
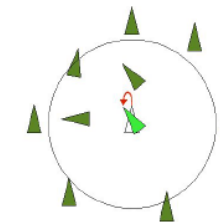
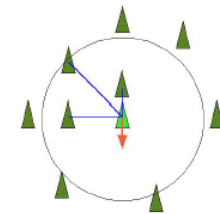
A novel hybrid gravitational search particle swarm optimization algorithm. Kanh, Ling.
Engineering Applications of Artificial Intelligence Volume 102, June 2021 **(En Poliformat)**

Modelado de movimientos de grupos, vida artificial, flocking

Flocking: Movimiento colectivo de una alto número de entidades autónomas (rebaños, bandadas, etc.)

Consideración de otros factores o reglas en el movimiento de las partículas:

- **Separación:** Cada partícula tiende a moverse lejos de sus vecinos si estos demasiado cerca. Mantiene distancia razonable, pero no demasiado lejos (*repulsión posición vecinos*).
- **Alineación:** Cada partícula sigue la *dirección media* del movimiento de sus vecinas e igualando su velocidad: búsqueda en el entorno local alineada con la búsqueda de sus partículas vecinas.
- **Cohesión:** Las partículas siguen una dirección global y mantienen una cohesión de grupo con sus vecinas, tendiendo ir al centro del enjambre. Cada partícula tiende a moverse hacia la *posición media* de su vecindad.



[DEMO](#)

Modelo inicial flocking: Flocks, herds and schools: A Distributed Behavioral Model, Craig W. Reynolds. *Computer Graphics*, 21(4), July 1987. **En Poliformat**

Acknowledgements: Me gustaría agradecer a las bandadas, rebaños y escuelas por existir.

Entorno Boids: <http://www.red3d.com/cwr/boids/>

Demo: <https://owenmcnaughton.github.io/Boids.js/>



PSO aplicable con buenos resultados en problemas multi-objetivo con variables reales.

- *When Evolutionary Computing Meets **Astro and Geoinformatics**.*
Chellym Mirchev. In Knowledge Discovery in Big Data from Astronomy and Earth Observation Elsevier (2020)
- *Analysis of the publications on the **applications** of particle swarm optimisation*
Riccardo Poli . Journal of Artificial Evolution and Applications, Hindawi (2008)
- *Application of particle swarm optimization algorithm in **power system problems**.*
Zargari, Heris, vatloo. Handbook of Neural Computation. Elsevier 2017
- *A survey of Particle Swarm Optimization techniques for solving university Examination **Timetabling Problem**.*
Zarabi . Artificial Intelligence Review, 44:537–546 Springer (2015)
- *A Comprehensive Survey on Particle Swarm Optimization Algorithm and Its **Applications***
Zhang, Wang, Ji. Mathematical Problems in Engineering, Article ID 931256 (2015)

PSO Tutorial: <http://www.swarmintelligence.org/tutorials.php>

PSO Software: <http://www.borgelt.net/psopt.html>

Nuevas Variantes y adaptaciones a tipos de problemas.

....Otras Metaheurísticas Sociales

Algoritmo inspirado en murciélagos (Bat Algorithm - BA)

A New Metaheuristic Bat-Inspired Algorithm. X.
S. Yang

Nature Inspired Cooperative Strategies for
Optimization, Studies in Computational
Intelligence, 284, (2010)

Cada murciélago virtual vuela **al azar** con una
velocidad v_i a la posición (solución) x_i con una
frecuencia de onda e intensidad A_i .

Cuando localiza una presa, **cambia su frecuencia
e intensidad** y la búsqueda se intensifica como
una búsqueda **local**, que continua hasta que se
cumpla el criterio de parada.

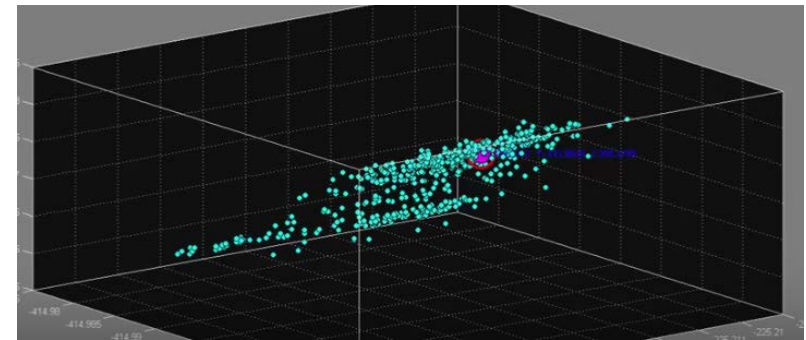
El proceso completo se repite iterativamente
hacia una nueva presa.

De esta forma, se asume que, mediante el ajuste
de la frecuencia, se puede controlar el balance
entre exploración y explotación.

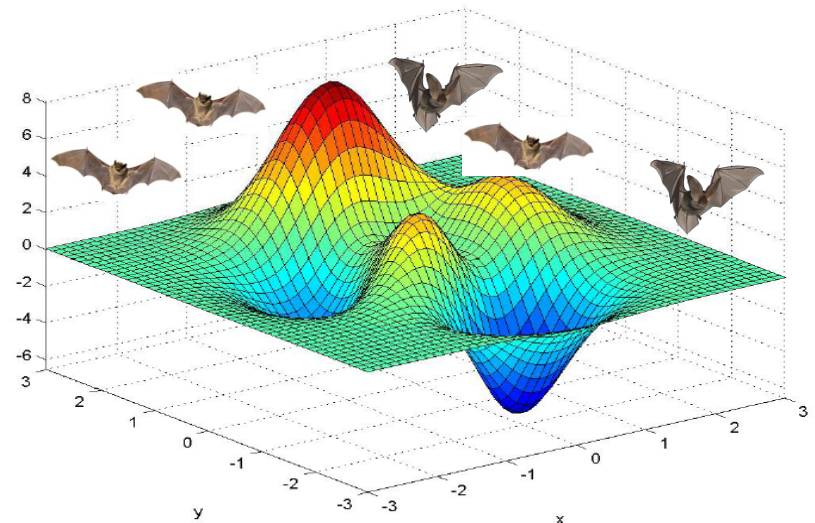
An improved discrete bat algorithm for symmetric and asymmetric
Traveling Salesman Problems

Eneko Osaba^{a,b,*}, Xin-She Yang^b, Fernando Diaz^a, Pedro Lopez-Garcia^a,
Roberto Carballedo^a

^a Deusto Institute of Technology (Deustotech), University of Deusto, Av. Universidades 24, Bilbao 48007, Spain
^b School of Science and Technology, Middlesex University, Hendon Campus, London, NW4 4BT, United Kingdom



La interacción entre partículas se basa en la
frecuencia e intensidad de los vecinos



- **Enjambre de Luciérnagas** (**Glow worm swarm optimization**, Krishnanand, Ghose, 2005),
- **Algoritmo de Luciérnagas** (**Firefly Algorithm** , X. S. Yang 2009)

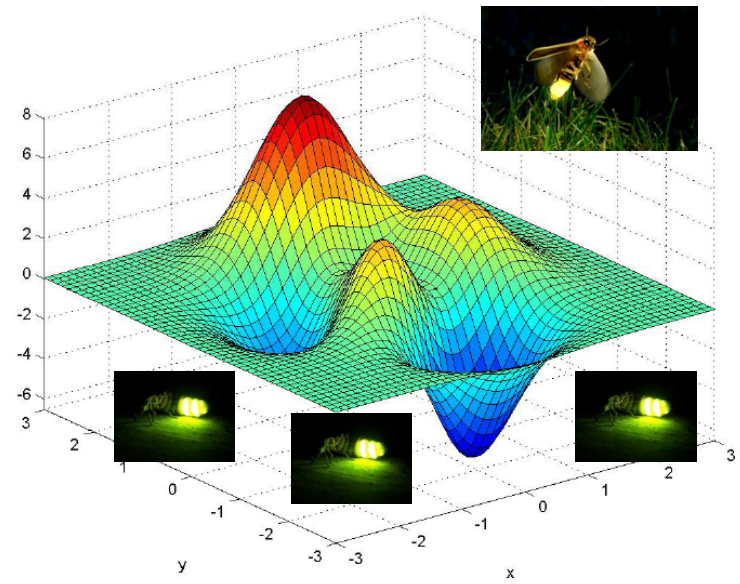


- Las luciérnagas basan su comportamiento social en la luminosidad que emiten (luciferina).
- En su cortejo nocturno, los machos vuelan en busca de pareja mientras emiten destellos de luz característicos de cada especie.
- Las hembras de la misma especie pueden responder con destellos específicos y así el apareamiento puede ocurrir.
- Con similar planteamiento, la **luminosidad de una luciérnaga depende de la calidad de la solución encontrada y la distancia desde donde las otras compañeras están buscando soluciones.**
- Cada luciérnaga selecciona, utilizando un mecanismo probabilístico, un vecino que tiene un valor más alto de luciferina que su propio y se mueve hacia él.

Aplicaciones: Diseño de redes ópticas, Canales de comunicación, Infraestructuras (puentes), etc.



La interacción entre partículas se basa en la luminosidad (intensidad y color)



A swarm optimization algorithm inspired in the behavior of the social-spider.

Cuevas, Cienfuegos, Zaldívar, Pérez-Cisneros. Expert Systems with Applications, 40 (16), (2013), pp. 6374-6384



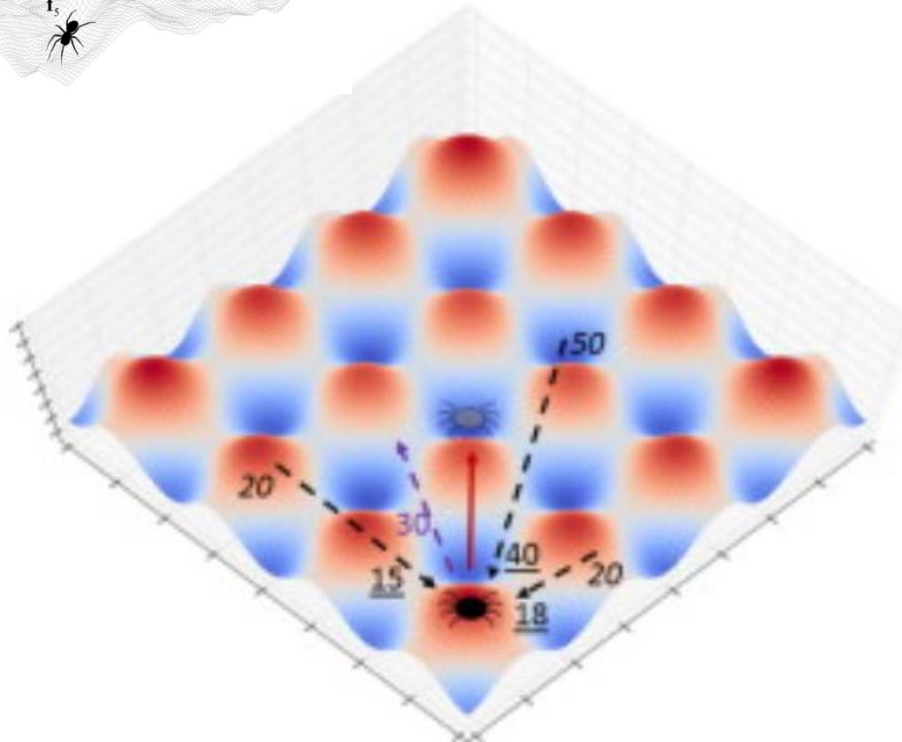
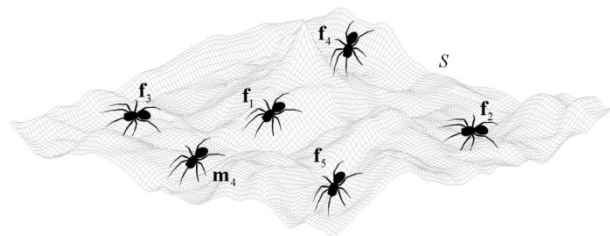
A social spider algorithm for global optimization

James J.Q. Yu , Victor O.K. Li

[Show more](#)

doi:10.1016/j.asoc.2015.02.014

[Get rights and content](#)



- Current position
- New position
- Received vibration
- 20 Vibration intensity at source
- 15 Attenuated vibration intensity
- Target vibration at last iteration
- 30 Target vibration intensity
- Spider Movement



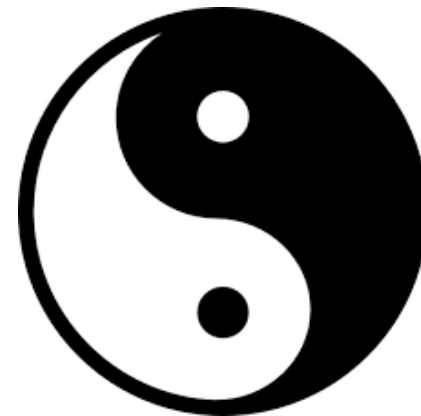
Yin-Yang-pair Optimization: A novel lightweight optimization algorithm

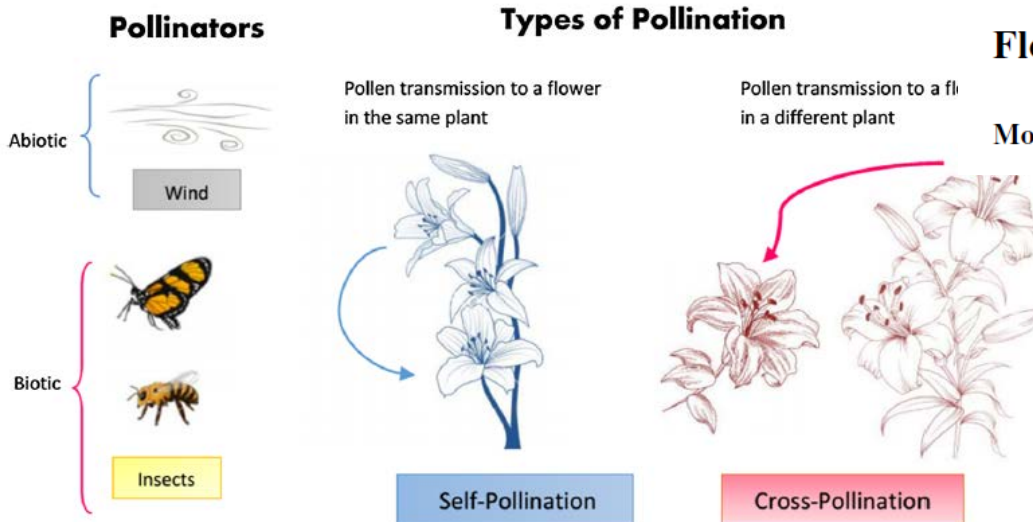


Varun Punnathanam, Prakash Kotecha*

Department of Chemical Engineering, Indian Institute of Technology Guwahati, Guwahati 781039, Assam, India

*Based on the YinYang philosophy of
balance between conflicting forces:
Exploration and Exploitation.*





Flower pollination algorithm: a comprehensive review

Mohamed Abdel-Basset¹ · Laila A. Shawky¹

Metaheurística que toma su metáfora en la proliferación de flores en las plantas.

```

1: Initialize parameters with switching probability  $p \in [0,1]$  ;
2: Generate initial population of flowers randomly;
3: Evaluate initial population and find the current best solution  $gbest$ ;
4: while (stopping criterion not satisfied) do
5:   For each flower
6:     if  $rand() < p$ 
7:       Global pollination :  $x_i^{t+1} = x_i^t + L(x_i^t - gbest)$  ; // Based on Lévy step
8:     else
9:       Select two random solutions  $x_j^t$  and  $x_k^t$  ;
10:      Local pollination :  $x_i^{t+1} = x_i^t + \epsilon(x_j^t - x_k^t)$ ;
11:    end if
12:    Evaluate new solutions;
13:    Update solutions with better new ones;
14:  end for
15:  Keep the current best solution;
16: end while
    
```

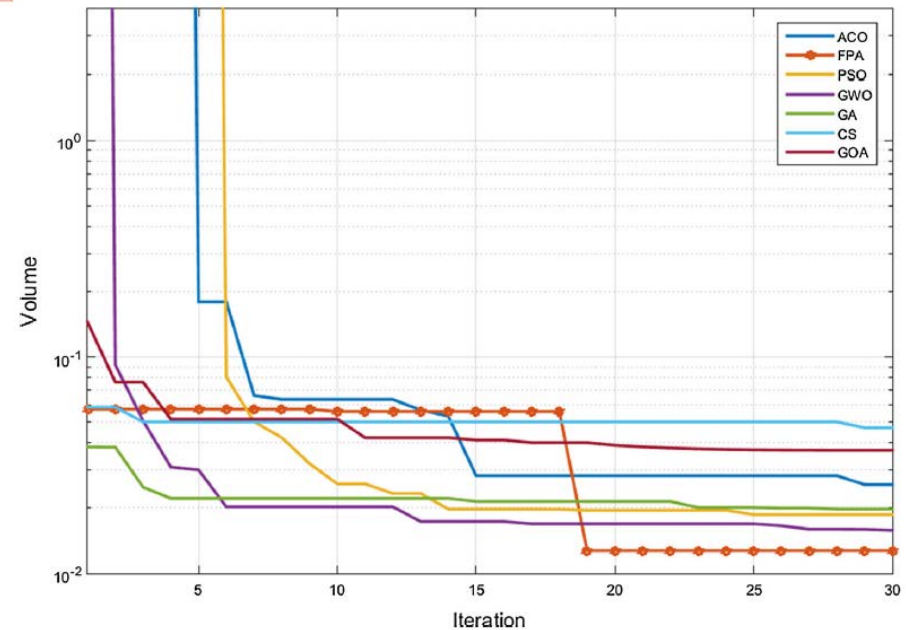


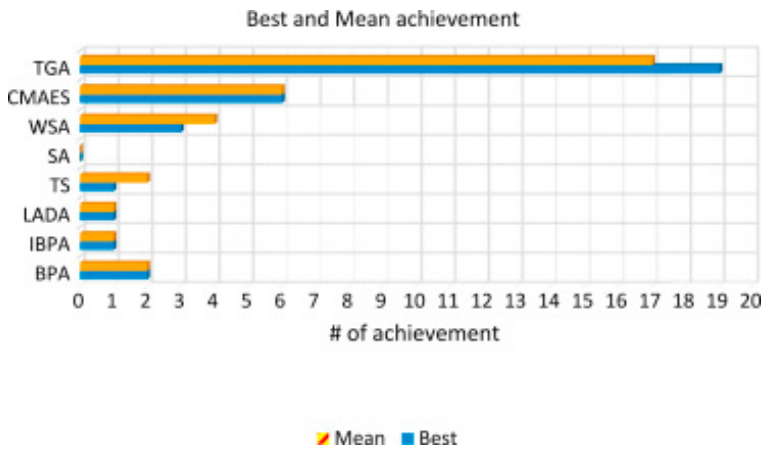
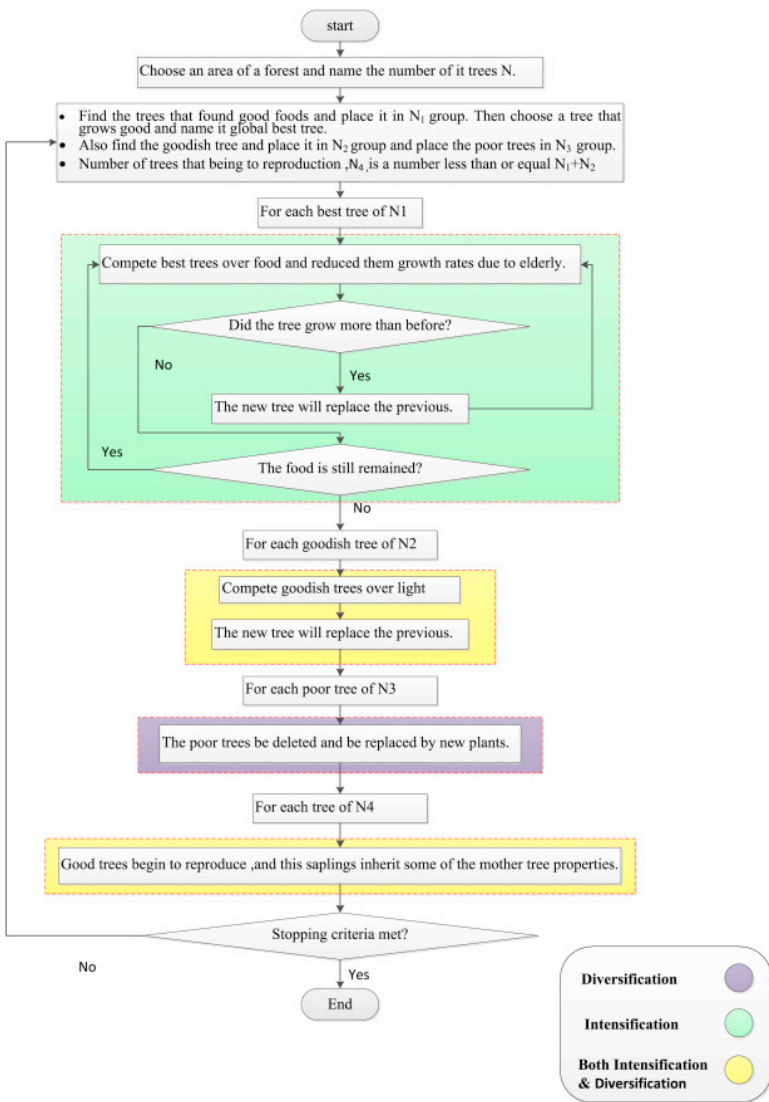
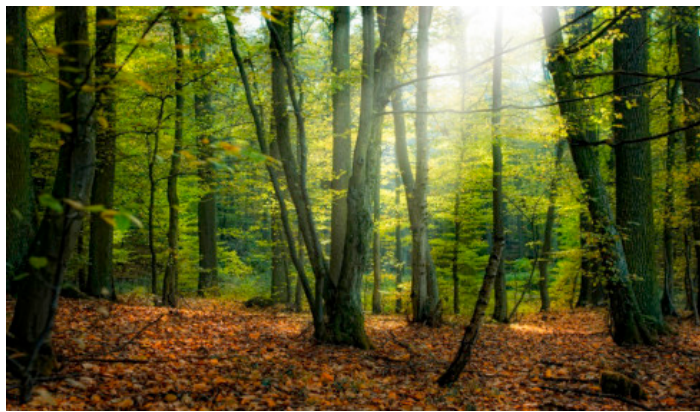
Fig. 4 Convergence of FPA and other metaheuristics

Metaheurística basada en la conducta de los árboles

- Competencia para obtener luz y comida.



Tree Growth Algorithm (TGA): A novel approach for solving optimization problems



A comparison of novel metaheuristic algorithms on color aerial image multilevel thresholding.

Kurban, Durmus, Karakose.

Engineering Applications of Artificial Intelligence.
Vol.105, Oct.2021

Multilevel color image thresholding can be considered as an optimization problem that an algorithm should determine the optimum threshold values to obtain a perfectly segmented image. In recent years, metaheuristic algorithms become popular in several fields including image thresholding by their advantage of flexible structure. The motivation of this study relies on the fact that using same algorithm to solve any particular problem do not guarantee the best results as stated in no-free lunch theorem in optimization. Therefore, six novel nature inspired algorithms such as **equilibrium optimization (EO), political optimizer (PO), turbulent flow of water-based optimization (TFWO), henry gas solubility optimization (HGSO), marine predators algorithm (MPA), and slime mould algorithm (SMA)** are chosen to be compared by determining the multilevel color image thresholding values.



A comparison of novel metaheuristic algorithms on color aerial image multilevel thresholding

Rifat Kurban^{a,*}, Ali Durmus^b, Ercan Karakose^c

Show more

+ Add to Mendeley + Share + Cite

<https://doi.org/10.1016/j.engappai.2021.104410>

Get rights and content

Abstract

Color image thresholding is a well-known approach for image segmentation. An objective function, based on image entropy, is defined by threshold numbers and locations in the color histogram. Multiple classes of images can be created with multilevel thresholding. The main problem with thresholding techniques is to decide the threshold color values for each image. For a human operator, it is very hard to determine the specific threshold values of the images to be segmented. From this perspective, multilevel color image thresholding can be considered as an optimization problem that an algorithm should determine the optimum threshold values to obtain a perfectly segmented image. In recent years, metaheuristic algorithms become popular in several fields including image thresholding by their advantage of flexible structure. The motivation of this study relies on the fact that using same algorithm to solve any particular problem do not guarantee the best results as stated in no-free lunch theorem in optimization. Therefore, six novel nature inspired algorithms such as equilibrium optimization (EO), political optimizer (PO), turbulent flow of water-based optimization (TFWO), henry gas solubility optimization (HGSO), marine predators algorithm (MPA), and slime mould algorithm (SMA) are chosen to be compared by determining the multilevel color image thresholding values. These algorithms, which are used for the first time to solve this problem, are also compared statistically with extensive experiments. Aerial test images obtained by *zenon* are used in the experiments. Kapur's entropy and between-class variance (Otsu's method) objectives are maximized by metaheuristic algorithms. Experimental results are evaluated with structural similarity (SSIM), peak-signal noise ratio (PSNR), blind/referenceless image spatial quality evaluator (BRISQUE), perception-based image quality evaluator (PIQE), natural image quality evaluator (NIQE), and computing CPU time consumption of the algorithms. Another problem of thresholding is that the question of how to compare the results objectively. Many image quality metrics may give incompatible results. Correlation analysis of the average ranking results show that image quality metrics used in this study are compatible to each other. Extensive experiments show that MPA and TFWO outperformed SMA, EO, PO and HGSO in terms of PSNR, SSIM, BRISQUE, PIQE, NIQE, and CPU time consumption for multilevel color aerial image thresholding.

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/343251814>

Coronavirus Optimization Algorithm: A Bioinspired Metaheuristic Based on the COVID-19 Propagation Model

Article in *Big Data* - July 2020

DOI: 10.1089/big.2020.0051

CITATIONS

33

READS

658

9 authors, including:



Francisco Martínez-Álvarez

Universidad Pablo de Olavide

139 PUBLICATIONS 2,464 CITATIONS

SEE PROFILE



Gualberto Asencio Cortés

University of Pablo de Olavide

51 PUBLICATIONS 516 CITATIONS

SEE PROFILE



José Francisco Torres

Universidad Pablo de Olavide

13 PUBLICATIONS 252 CITATIONS

SEE PROFILE



David Gutiérrez-Avilés

Universidad Pablo de Olavide

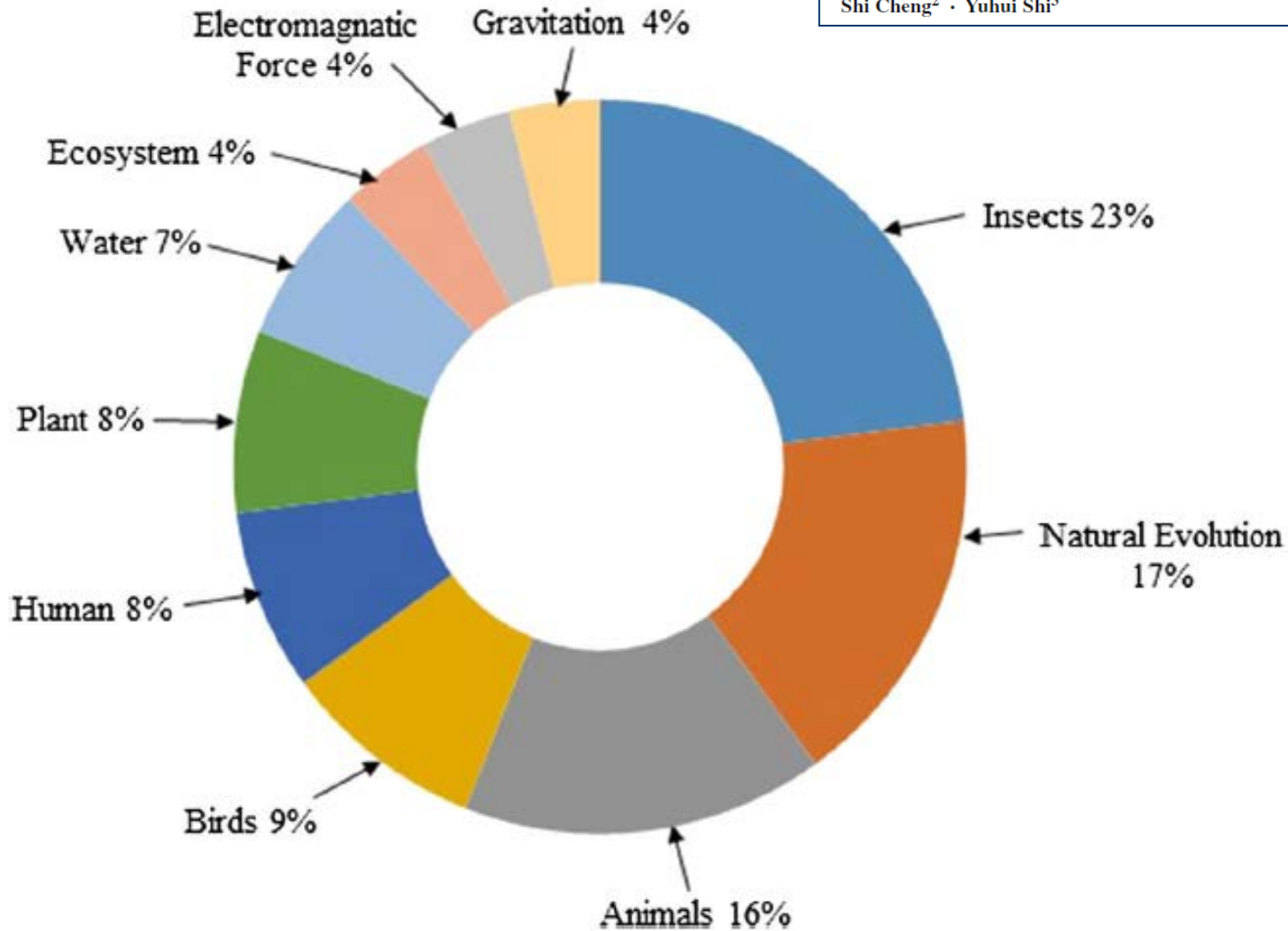
24 PUBLICATIONS 162 CITATIONS

SEE PROFILE

En resumen..... Metaheurísticas Bio-Inspiradas

Metaheuristic research: a comprehensive survey

Kashif Hussain¹  · Mohd Najib Mohd Salleh¹ ·
Shi Cheng² · Yuhui Shi³



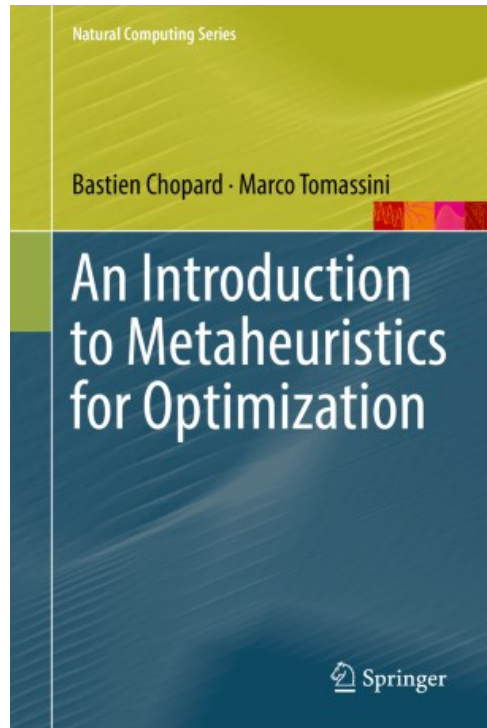
Reference

Algorithm name

Reference

Algorithm name

Robbins and Monro (1951)	Stochastic Approximation Method	Wierstra et al. (2008)	Natural Evolution Strategies
Hooke and Jeeves (1961)	Pattern Search (PS)	Yang (2009)	Firefly Algorithm (FA)
Rastrigin (1963)	Random Search (RS)	Teodorović (2009)	Bee Colony Optimization (BCO)
Matyas (1965)	Random Optimization	He et al. (2009)	Group Search Optimizer (GSO)
Fogel et al. (1966)	Evolutionary strategy (ES)	Yang and Deb (2009)	Cuckoo Search (CS)
Hastings (1970)	Metropolis–Hastings algorithm	Rashedi et al. (2009)	Gravitational Search Algorithm (GSA)
Kernighan and Lin (1970)	Graph Partitioning Method	Husseinzadeh Kashan (2014)	League Championship Algorithm (LCA)
Holland (1975)	Genetic Algorithm (GA)	Kadioglu and Sellmann (2009)	Dialectic Search
Glover (1977)	Scatter Search	Yang (2010a)	Bat Algorithm (BA)
Kirkpatrick et al. (1983)	Simulated Annealing (SA)	Lourenço et al. (2010)	Iterated Local Search (ILS)
Glover (1986)	Tabu Search (TS)	Shah-Hosseini (2011)	Galaxy-based Search Algorithm (GbSA)
Farmer et al. (1986)	Artificial Immune System (AIS)	Tamura and Yasuda (2011b) ; Tamura and Yasuda (2011a)	Spiral Optimization (SO)
Moscato (1989)	Memetic Algorithm	Rajabioun (2011)	Cuckoo Optimization Algorithm (COA)
Koza (1992)	Genetic Programming (GP)	Rao et al. (2011)	Teaching-Learning-Based Optimization (TLBO)
Dorigo et al. (1996)	Ant Colony Optimization (ACO)	Alsheddy (2011)	Guided Local Search (GLS)
Fonseca and Fleming (1993)	Multi-Objective GA (MOGA)	Gandomi and Alavi (2012)	Krill Herd (KH) Algorithm
Battiti and Brunato (2010)	Reactive Search Optimization (RSO)	Hajiaghaei-Keshteli and Aminnayeri (2014)	Keshtel Algorithm (KA)
Srinivas and Deb (1994)	NSGA for Multi-Objective Optimization	Civicioglu (2012)	Differential Search Algorithm (DSA)
Eberhart and Kennedy (1995)	Particle Swarm Optimization (PSO)	Alatas (2012)	Artificial Chemical Reaction Optimization Algorithm (ACROA)
Hansen and Ostermeier (2001)	CMA-ES	Husseinzadeh Kashan (2013)	Optics Inspired Optimization (OIO)
Storn and Price (1997)	Differential Evolution (DE)	Husseinzadeh Kashan et al. (2015)	Grouping Evolution Strategies (GES)
Rubinstein (1997)	Cross Entropy Method (CEM)	Sadollah et al. (2013)	Mine Blast Algorithm (MBA)
Hansen et al. (1997)	Variable Neighborhood search (VNS)	Hatamlou (2013)	Black Hole (BH)
Taillard and Voss (1999)	Partial Optimization Metaheuristic Under Special Intensification Conditions (POPMUSIC)	Civicioglu (2013a)	Artificial Cooperative Search Algorithm (ACS)
Geem et al. (2001)	Harmony Search (HS)	Kaveh and Mahdavi (2015)	Colliding Bodies Optimization (CBO)
Hanseth and Aanestad (2001)	Bootstrap Algorithm (BA)	Gandomi (2014)	Interior Search Algorithm (ISA)
Larrañaga and Lozano (2002)	Estimation of Distribution Algorithms (EDA)	Salimi (2014)	Stochastic Fractal Search (SFS)
Deb et al. (2002)	NSGA-II for Multi-Objective Optimization	Cheng and Prayogo (2014)	Symbiotic Organisms Search (SOS)
Liu and Passino (2002)	Bacterial Foraging behavior Optimization (BFO)	Zheng (2015)	Water Wave Optimization (WWO)
Nakrani and Tovey (2004)	Bees Optimization	Dogan and Ölmez (2015)	Vortex Search Algorithm (VSA)
Krishnanand and Ghose (2005) ; Krishnanand and Ghose (2006)	Glowworm Swarm Optimization (GSO)	Wang et al. (2015)	Elephant Herding Optimization (EHO)
Basturk and Karaboga (2006)	Artificial Bee Colony Algorithm (ABC)	Baykasoğlu and Akpinar (2017)	Weighted Superposition Attraction (WSA)
Pham et al. (2005)	Bees Algorithms (BA)	Mirjalili (2016b)	Dragonfly algorithm
Haddad et al. (2006)	Honey-bee Mating Optimization (HMO)	Liang et al. (2016)	Virus Optimization Algorithm
Shah-Hosseini (2009)	Intelligent Water Drops (IWD)	Mirjalili (2016a)	Sine Cosine Algorithm (SCA)
Atashpaz-Gargari and Lucas (2007)	Imperialist Competitive Algorithm (ICA)	Ebrahimi and Khamenechi (2016)	Sperm Whale Algorithm (SWA)
Mucherino and Seref (2007)	Monkey Search (MS)	Mirjalili et al. (2017)	Salp Swarm Algorithm



Artif Intell Rev (2019) 52:2191–2233
<https://doi.org/10.1007/s10462-017-9605-z>

Metaheuristic research: a comprehensive survey

Kashif Hussain¹  · Mohd Najib Mohd Salleh¹ ·
Shi Cheng² · Yuhui Shi³

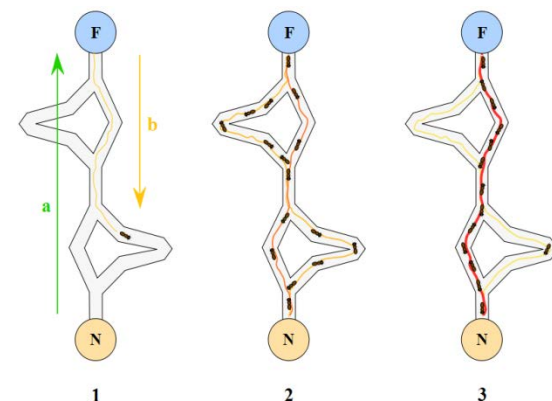
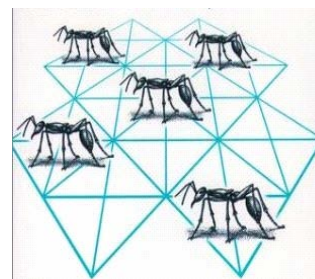
Artificial Intelligence Review (2020) 53:501–593
<https://doi.org/10.1007/s10462-018-9667-6>

A state of the art review of intelligent scheduling

Mohammad Hossein Fazel Zarandi¹ · Ali Akbar Sadat Asl¹ ·
Shahabeddin Sotudian² · Oscar Castillo³

Published online: 19 November 2018
© Springer Nature B.V. 2018

- Sistemas **bio-inspirados**
- Basados en el comportamiento colectivo de sistemas **descentralizados y auto-organizados** (hormigas, abeja, pájaros, peces, etc.)
- Las partículas representan **soluciones o procesos de búsqueda**, que inciden en:
 - **Exploración**: Generación de soluciones nuevas
 - **Explotación**: Mejora de la solución en su vecindad
- Las **partículas interactúan** entre si compartiendo soluciones encontradas (para que otras partículas utilicen/mejoren las buenas soluciones. La interacción se basa en **reglas muy simples**).
- La interacción entre las partículas permite un **comportamiento global** de la búsqueda más inteligente y adaptativo que la suma de comportamientos individuales.

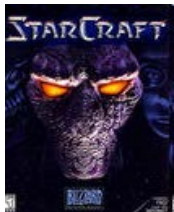
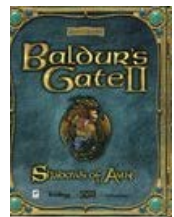


Path-finding (Obtención de sendas en laberintos)

Objetivo: encontrar el mejor camino (habitualmente, el más corto) entre dos puntos a través de un mapa con obstáculos. Problemas de laberinto, *Aplicaciones para videojuegos*, etc.

Algoritmos típicos: Dijkstra (muy costoso en laberintos complejos),
Algoritmos A (equivalente a Dijkstra, con $h(n)=0$),
A* sobre-informados, etc.

Metaheurísticas: Grasp, **inteligencia de enjambre (algoritmo de las hormigas),** ...

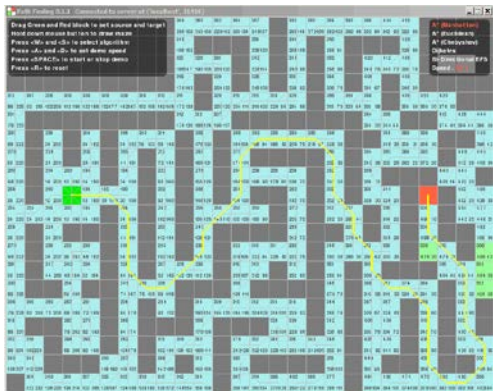


<http://www.movingai.com/benchmarks/>

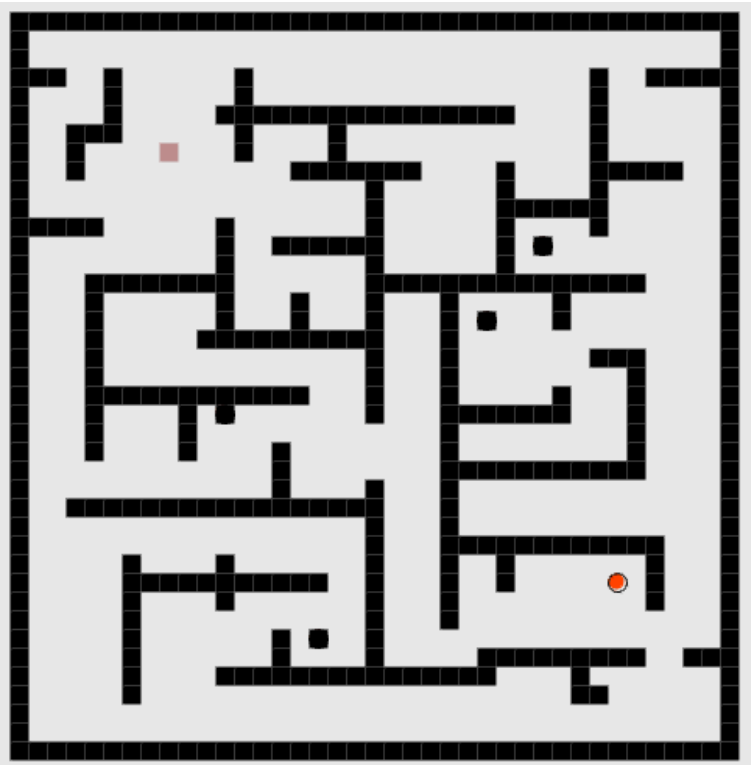
Benchmarks for Grid-Based Path finding.

IEEE Trans. on Comp. Int. and AI in games (2012)

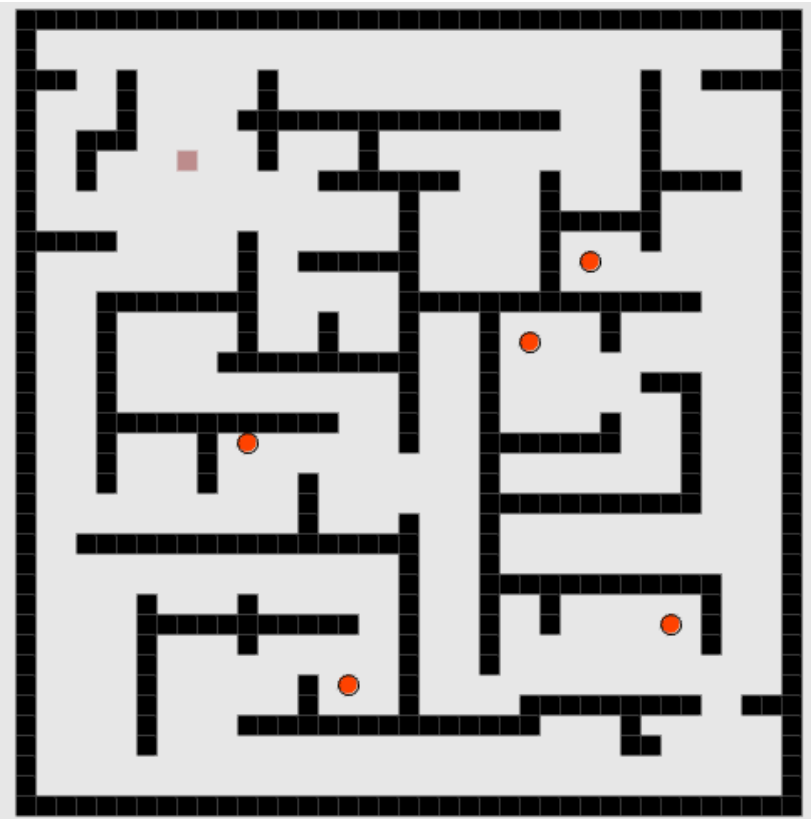
Algoritmos Heurísticos A

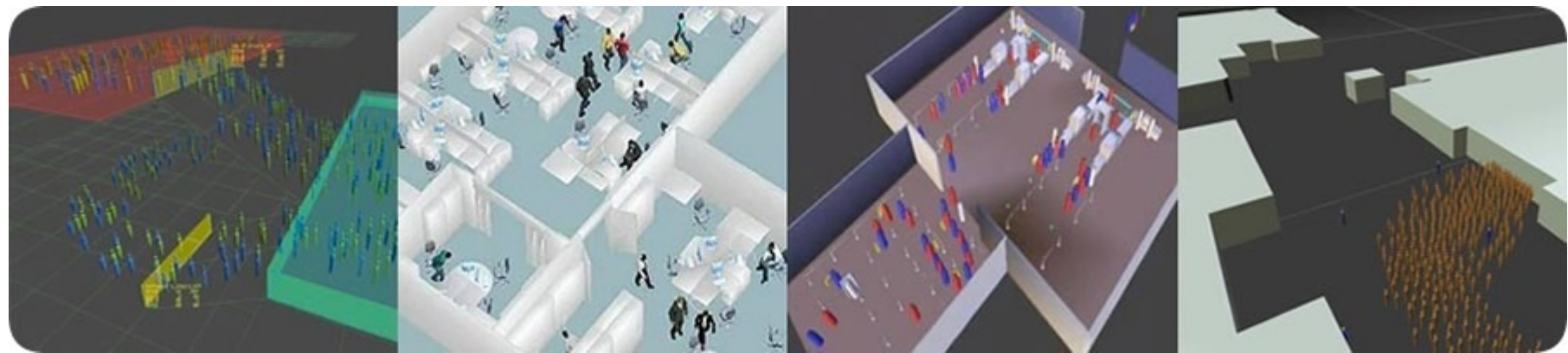


Laberinto Simple
(Alg. Hormigas)



Laberinto con
Múltiples Objetivos





<http://massivesoftware.com>



Simulación de colectivos coordinados
(movimiento de masas)



Flocking: Modelado de movimientos de grupos, vida artificial.

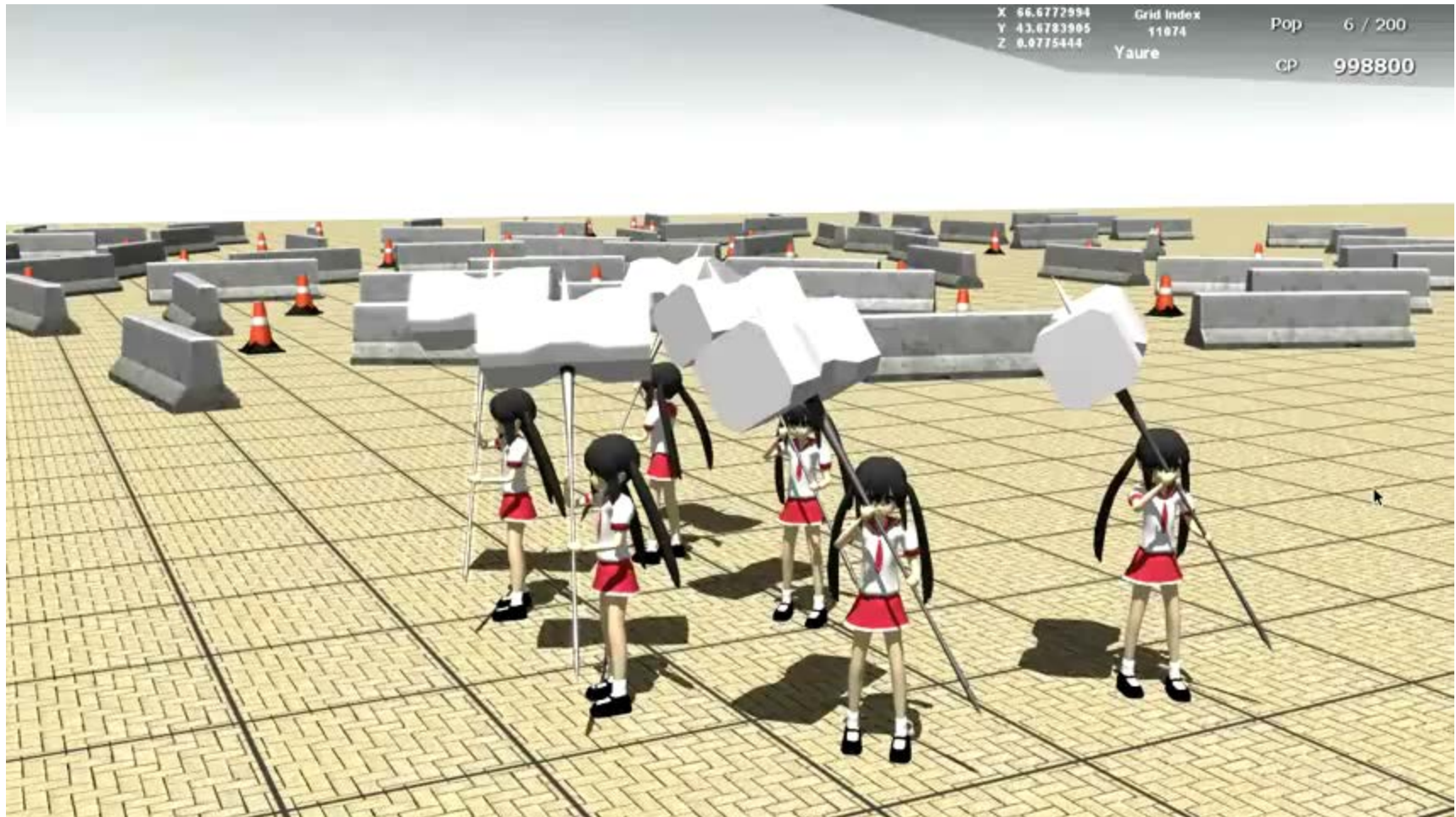
Las partículas se desplazan en una dirección global, persiguiendo una meta y evitando colisiones:

- Cada partículas mantiene una distancia con sus vecinos (*separación*).
- Cada partículas sigue la dirección media del movimiento de sus vecinas (*alineación*).
- Cada partículas mantiene una cohesión de grupo con sus vecinas (*cohesión*).
- Puede, o no, haber un líder en el grupo

Aplicaciones: Collective animal behaviour, movimiento de drones, juegos sociales, simulación en películas, etc.



Movimiento de individuos (*swarm behaviour*)



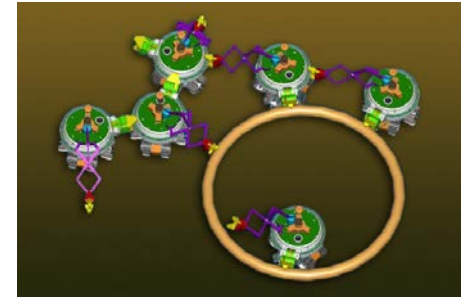
Swarm Robotics

Enfoque inspirado en el comportamiento colectivo de los animales sociales y que se centra en la interacción de múltiples robots.

Hace hincapié en el control descentralizado, comunicación limitada entre agentes, la información local, el seguimiento de un comportamiento global y la robustez del grupo.

Los sistemas de enjambre robóticos difieren de otros sistemas multi-robot en:

- Son robots autónomos ubicados en un determinado medio ambiente,
- El enjambre tiene un gran número de robots, posiblemente agrupados en pequeños grupos de robots homogéneos,
- Los robots son relativamente simples, tienen sensores locales y sus habilidades de comunicación son limitados.
- La coordinación entre los robots se distribuye tal que el sistema es tolerante a fallos, debido a la redundancia de robots cada uno de los agentes en el sistema no es esencial y puede ser sustituido por otro agente.
- Son fácilmente escalable, permitiendo que más agentes para ser agregados o eliminados de acuerdo a las demandas de la tarea



Científicos de Harvard crean un enjambre cooperativo de 1.000 robots

Objetivo: Aplicar técnicas de Inteligencia Colectiva.

*The main objective of the **Swarm-bots** is the design and implementation of **self-organising** and **self-assembling artefacts**. This novel approach finds its theoretical roots in recent studies in **swarm intelligence**: self-organising and self-assembling capabilities shown by social insects and other animal societies.*

The main tangible objective is the construction of at least one of such artefact (swarm-bot). That is, an artifact composed of a number of simpler, insect-like, robots(s-bots), built out of relatively cheap components, capable of self-assembling and self-organising to adapt to its environment.

Simulación de Escenarios
(Lausanne Bruselas)



Swarm-Bots



Hardware: <https://www.k-team.com/mobile-robotics-products/kilobot>

Ejemplo de resolución

(encontrar senda optima y transportar objeto)



Dificultades y Colaboración



Distintos algoritmos/heurísticas de búsqueda/optimización

Algoritmo de las Hormigas

Búsqueda en Haz

Enfriamiento Simulado

GRASP

Algoritmos Genéticos

Enjambre de Partículas

BÚSQUEDA DISPERSA

Algoritmos Meméticos

Búsqueda Tabú

Algoritmo A*

Algoritmo de las Abejas

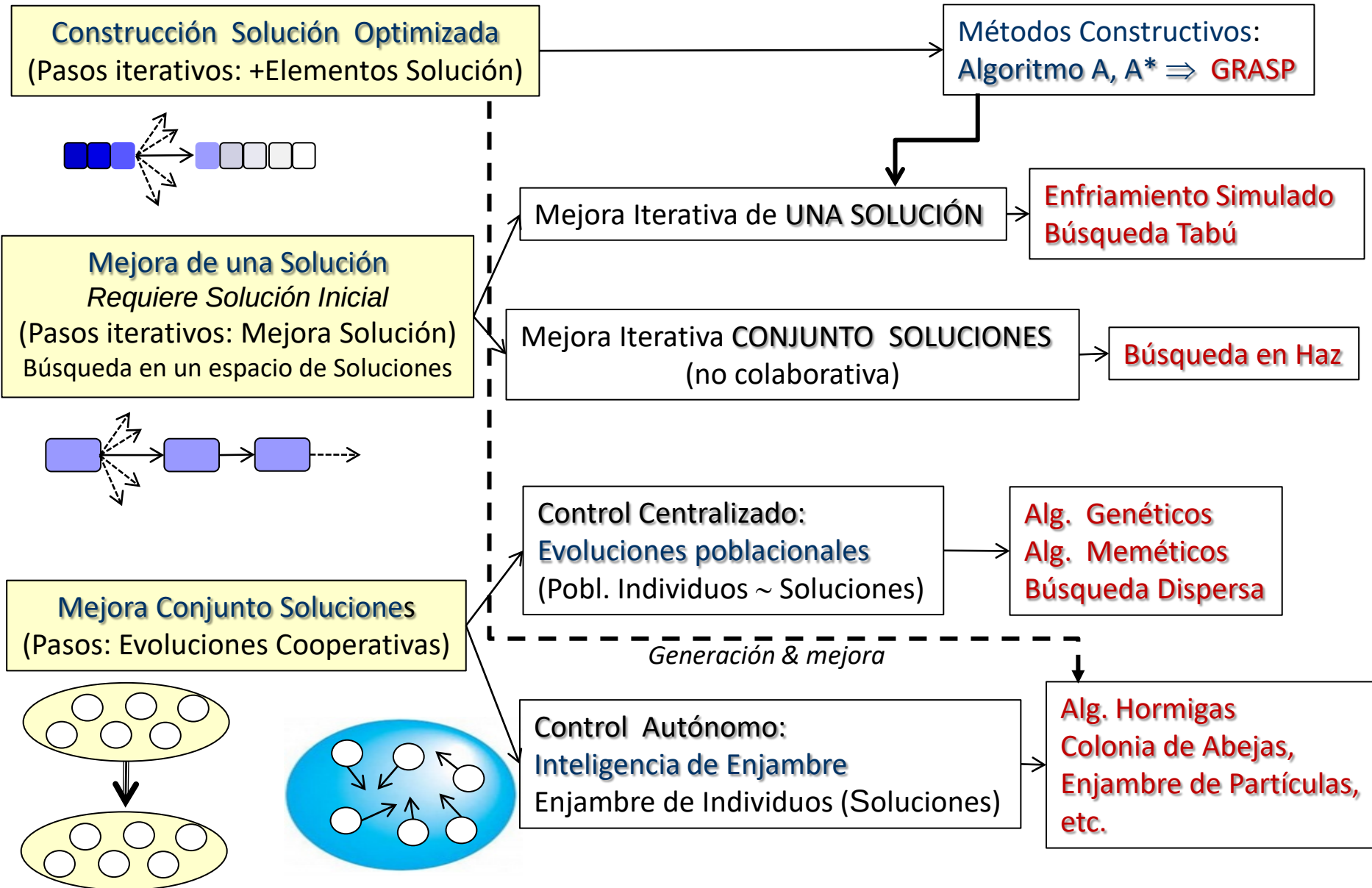
Heurística $h1(n)$

Heurística $h2(n)$

Heurística $h3(n)$

Cual es mejor?

Métodos Metaheurísticos. s.





No Free Lunch Theorems for Optimization

David H. Wolpert and William G. Macready

'No free lunch' Theorem

*(No free lunch theorems for optimization, Wolpert, D.H.; Macready, W.G.;
IEEE Transactions on Evolutionary Computation, 1, 1997)*

*"The computational cost of finding a solution,
averaged over all problems in the class, is the same for any solution method."*

*"For any algorithm, any elevated performance over one class of problems
is exactly paid for in performance over another class."*

PROBLEMA

Mejora Metaheurística
Enfriamiento Simulado,
(Haz, Tabú)

Híbridos: Constructivos + Mejora Local
GRASP

Evolutivos
Algoritmos Genéticos
(Búsqueda dispersa, Alg. Meméticos)

Inteligencia Social
Alg. Hormigas, Enjambre Partículas

No Free Lunch Theorems for Optimization

David H. Wolpert and William G. Macready



Contents lists available at SciVerse ScienceDirect

Information Sciences

journal homepage: www.elsevier.com/locate/ins



A survey on optimization metaheuristics

Ilhem Boussaïd^a, Julien Lepagnot^b, Patrick Siarry^{b,*}

^aUniversité des sciences et de la technologie Houari Boumediene, Electrical Engineering and Computer Science Department, El-Alia BP 32 Bab-Ezzouar, 16111 Algiers, Algeria

^bUniversité Paris Est Créteil, LISSI, 61 avenue du Général de Gaulle 94010 Créteil, France

Artif Intell Rev (2019) 52:2191–2233

<https://doi.org/10.1007/s10462-017-9605-z>

Metaheuristic research: a comprehensive survey

Natural Computing Series

Bastien Chopard · Marco Tomassini

An Introduction to Metaheuristics for Optimization

Springer

Machine Learning & Metaheuristics

PROBLEMA

Mejora Metaheurística
Enfriamiento Simulado,
(Haz, Tabú)

Híbridos: Constructivos + Mejora Local
GRASP

Evolutivos
Algoritmos Genéticos
(Búsqueda dispersa, Alg. Meméticos)

Inteligencia Social
Alg. Hormigas, Enjambre Partículas

Machine Learning



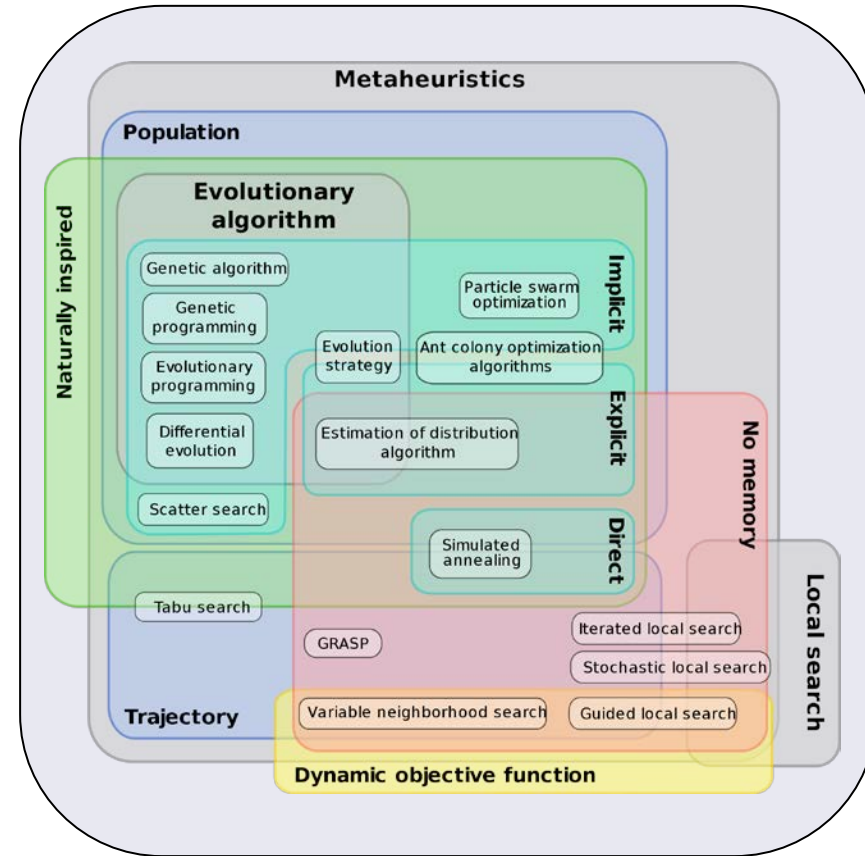
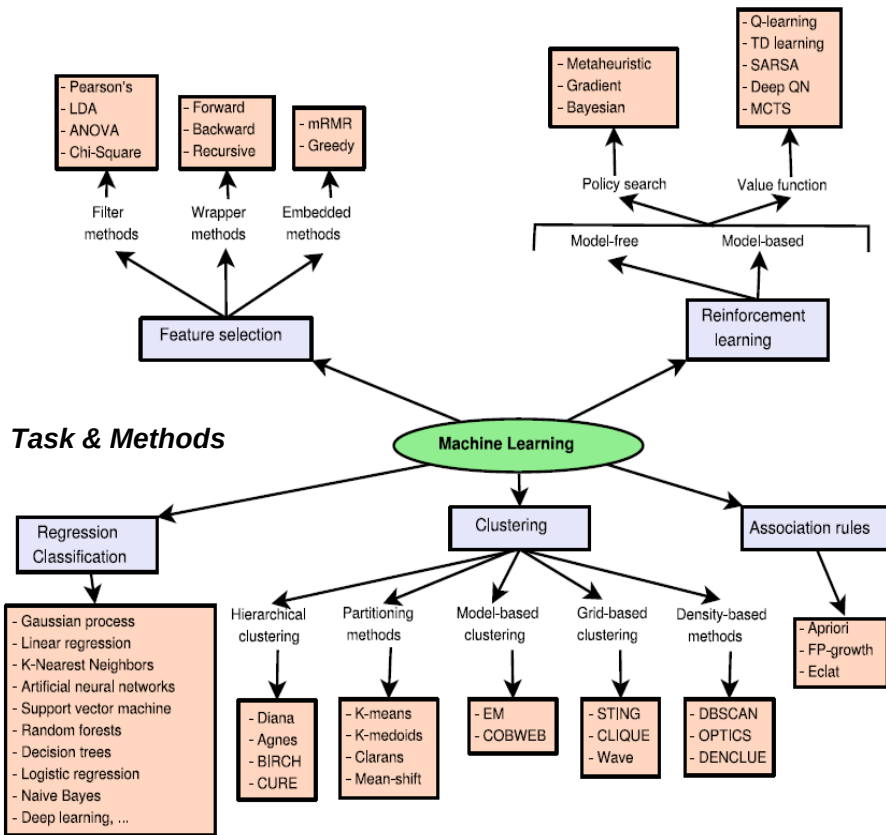
Metaheuristics



Machine Learning into Metaheuristics: A Survey and Taxonomy,
ACM Computing Surveys. Vol. 54, Iss. 6 (2021)

Aplicación “Machine Learning” a “Metaheurísticas”

Aplicación del aprendizaje automático (ML) para diseñar metaheurísticas más eficientes y efectivas.

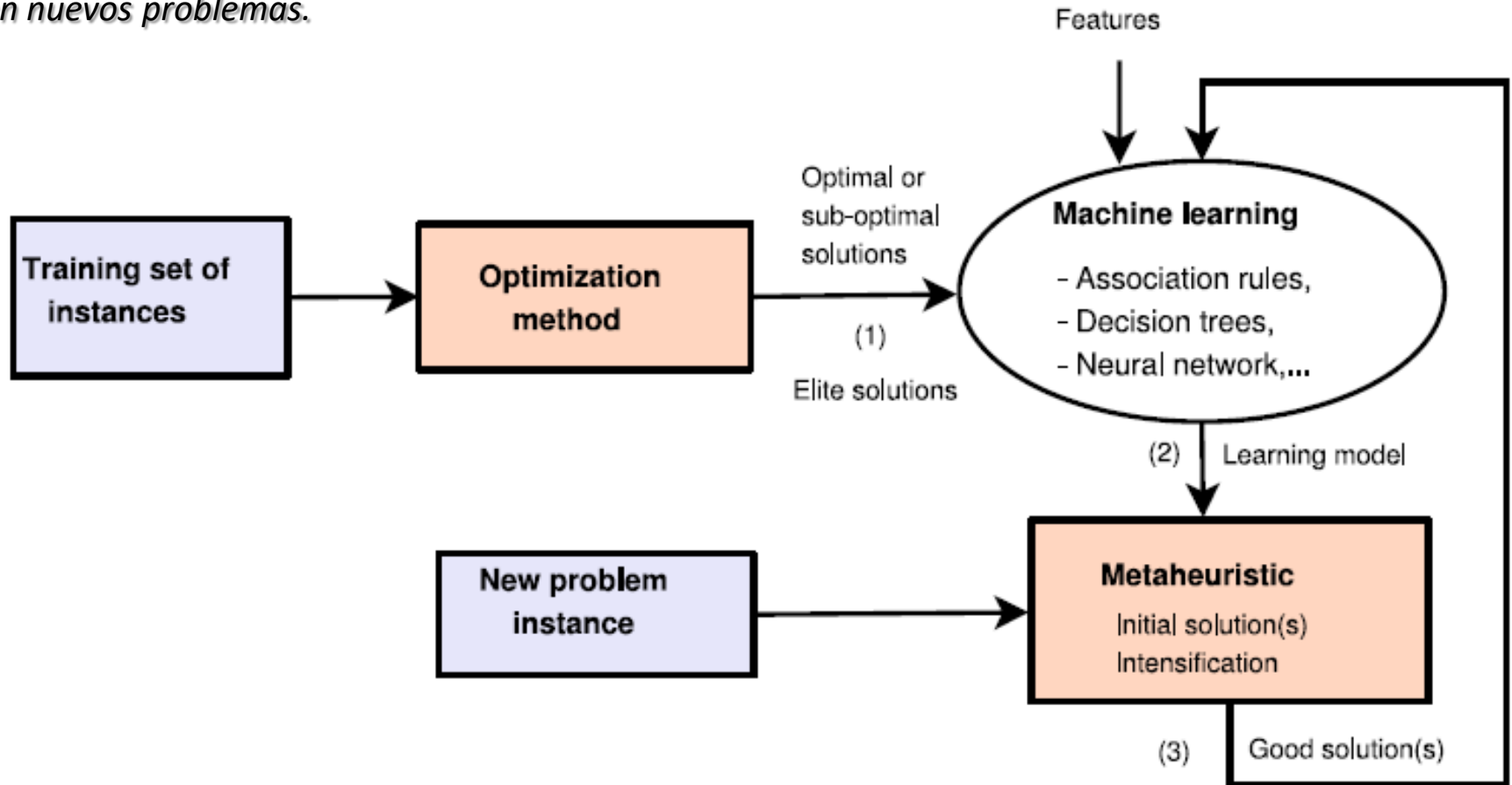


Aprendizaje de características/patrones/propiedades de buenas soluciones en un dominio de problemas,

para ayudar a la metaheurística a:

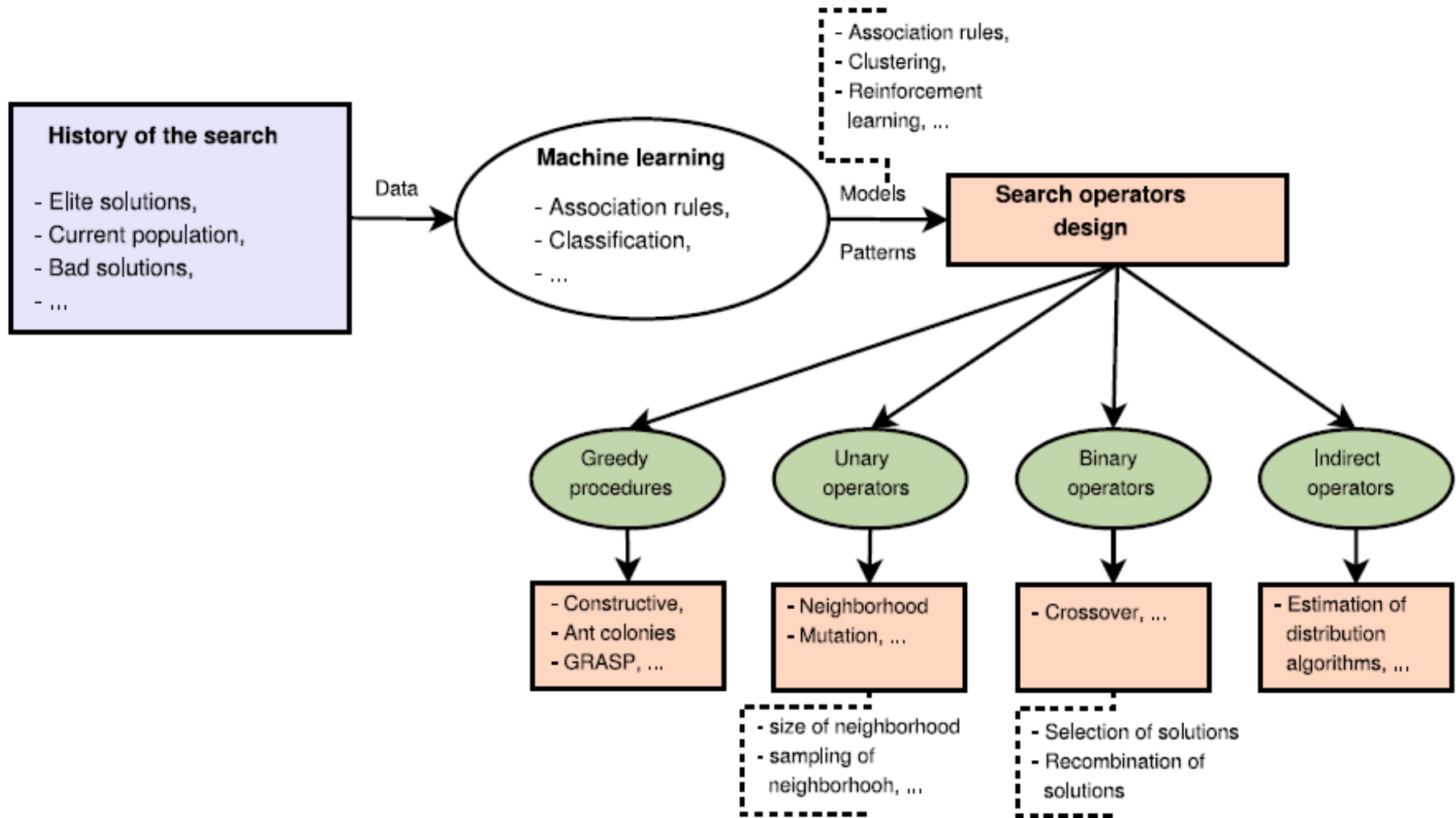
- (i) *buscar en regiones prometedoras del espacio de búsqueda, o*
- (ii) *Inicialización de mejores soluciones*

en nuevos problemas.

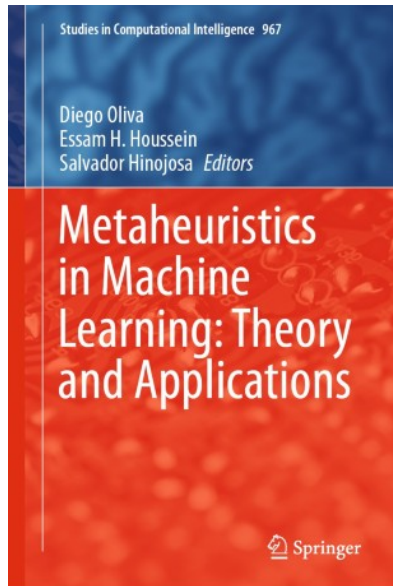
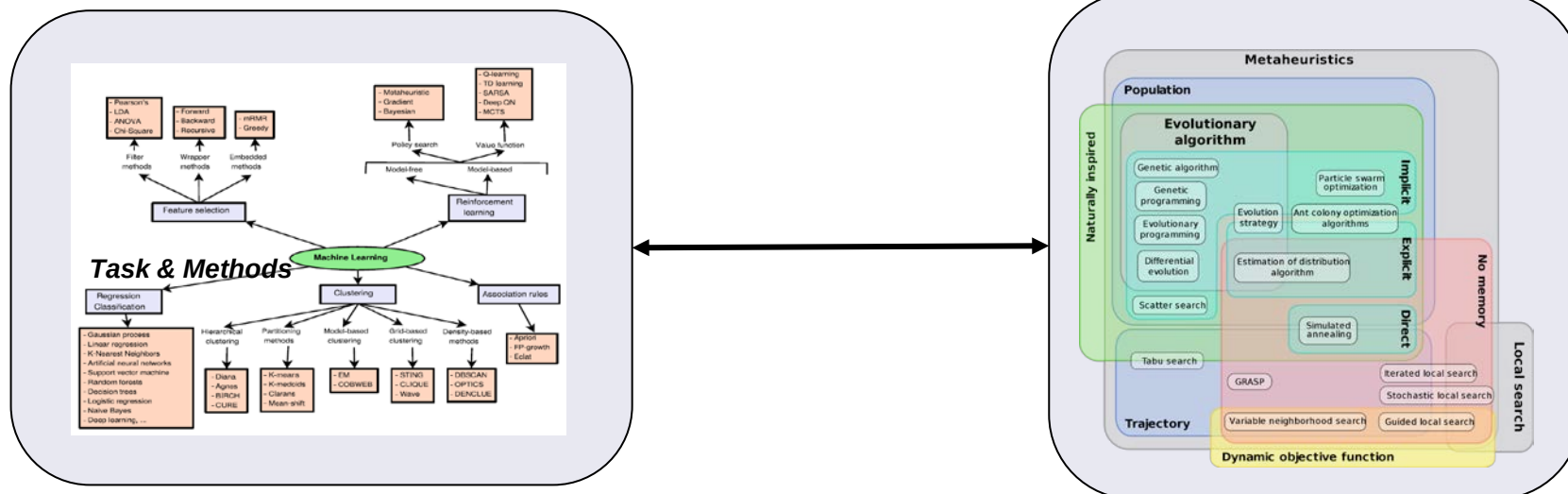


Parameter Tuning

Extracción de conocimiento de búsquedas previas para la obtención de buenos operadores en nuevos problemas: *parametrización de la metaheurística dependiente del problema (tamaño, tipología, etc.)*



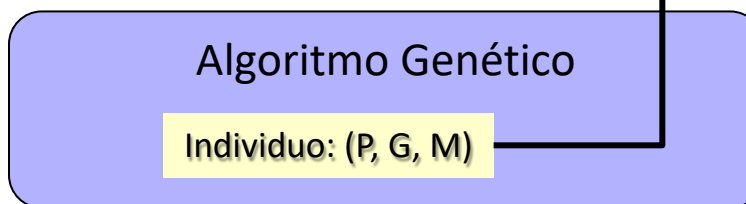
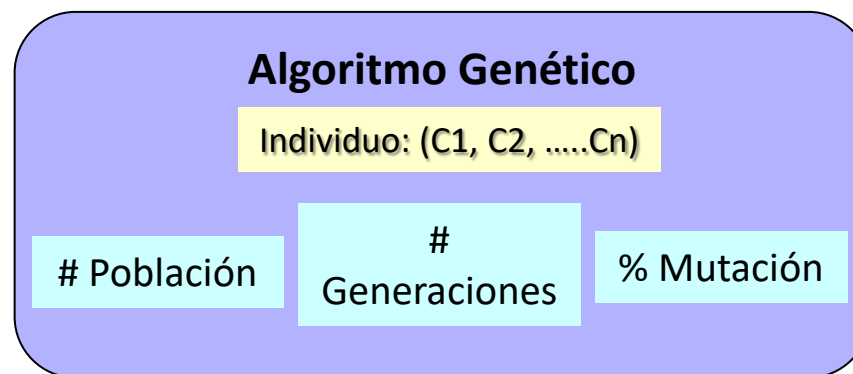
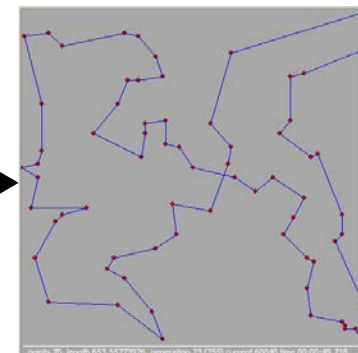
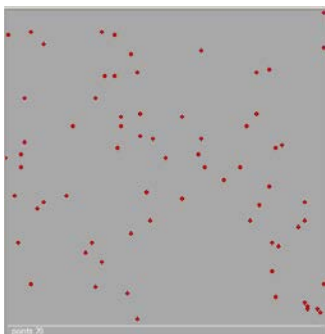
Aplicación de Metaheurísticas a Machine Learning



Metaheuristics in Machine Learning: Theory and Applications. Oliva, Houssein, Hinojosa. Springer (2021)

- Ajuste de la arquitectura del sistema de aprendizaje.
- Mejora del entrenamiento, solidez y velocidad.
- Ajuste de parámetros, coeficientes RNA,

Aplicación de Metaheurísticas a Metaheurísticas



fitness

Objetivo:
Mejor (P, G, M)

- Un AG es un proceso estocástico (no garantía obtención de buenas soluciones).
- Ante un tipo de problemas, hay que establecer sus parámetros, en función de su tipología, tamaño, etc.
- Esto se hace mediante un análisis razonado del problema, esperando encontrar los parámetros que darán buenas soluciones (sin garantía).

Se puede plantear un proceso de optimización metaheurístico de los parametros de un AG, que obtenga los mejores parámetros ante similar clase de problemas: AG sobre AG

Tema- 5 Inteligencia de Enjambre (Swarm intelligence)

(Inteligencia Social, Inteligencia Colectiva, Metaheurísticas Bio-Inspiradas)

- Algoritmo de las Hormigas (*Ant Colony Optimization - ACO*),
- Enjambre de partículas (*Particle Swarm Optimization - PSO*)
- Otras variantes



<http://www.swarmintelligence.org/>



Particle Swarm Optimization

Introduction

Particle swarm optimization (PSO) is a population based stochastic optimization technique developed by Dr. Eberhart and Dr. Kennedy in 1995, inspired by social behavior of bird flocking or fish schooling.

PSO shares many similarities with evolutionary computation techniques such as Genetic Algorithms (GA). The system is initialized with a population of random solutions and searches for optima by updating generations. However, unlike GA, PSO has no evolution operators such as crossover and mutation. In PSO, the potential solutions, called particles, fly through the problem space by following the current optimum particles.

Each particle keeps track of its coordinates in the problem space which are associated with the best solution (fitness) it has achieved so far. (The fitness value is also stored.) This value is called *pbest*. Another "best" value that is tracked by the particle swarm optimizer is the best value, obtained so far by any particle in the neighbors of the particle. This location is called *best*. When a particle takes all the population as its topological neighbors, the best value is a global best and is called *gbest*.

About PSO
People
Bibliography
Tutorials
Conferences
Links
Forum
Guestbook
About Author

